

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное
образовательное учреждение высшего образования
«Череповецкий государственный университет»

Институт (факультет)	информационных технологий
Направление подготовки	
(специальность)	09.04.04 Программная инженерия
Выпускающая кафедра	Математическое и программное обеспечение ЭВМ

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Название работы	Программное обеспечение для анализа контента и формирования рекомендаций социальной сети на основе искусственных нейронных сетей
-----------------	--

Студента	Викторова Даниила Алексеевича Ф.И.О.
----------	---

ДОПУСТИТЬ К ЗАЩИТЕ

Директор института (декан факультета)	(Ершов Е.В.)
Заведующий выпускающей кафедрой	(Ершов Е.В.)
Руководитель выпускной квалификационной работы	(Ершов Е.В.)
Консультант по технико-экономическому обоснованию разработки	(Виноградова Л.Н.)
Нормоконтролер	(Виноградова Л.Н.)
Выпускник	(Викторов Д.А.)

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное
образовательное учреждение высшего образования
«Череповецкий государственный университет»

АННОТАЦИЯ
выпускной квалификационной работы студента

по теме:

Программное обеспечение для анализа контента и формирования рекомендаций социальной
сети на основе искусственных нейронных сетей

Студент

<u>Викторов Даниил Алексеевич</u>	
фамилия, имя, отчество	подпись

Руководитель работы

<u>ФГБОУ ВО «Череповецкий государственный университет», кафедра МПО ЭВМ</u>
место работы
<u>Доктор технических наук, профессор</u>
должность
<u>Ершов Евгений Валентинович</u>
фамилия, имя, отчество

(Перечислить основные вопросы, которые рассматривались; результаты работы)

Данная работа посвящена разработке и исследованию рекомендательной системы для
социальной сети, основанной на использовании искусственных нейронных сетей. Основная
цель работы — улучшение качества рекомендаций контента для сети путем интеграции
современных методов анализа изображений и поведенческих данных. В работе была
разработана система рекомендаций, включающая основные компоненты: модель
машинного обучения VisionTransformer для преобразования изображений в векторные
представления и гибридный алгоритм, использующий метод K-Nearest Neighbors и метод
коллаборативной фильтрации для формирования рекомендаций на основе визуального
анализа и оценки схожести пользователей и их предпочтений. Комплексное решение
показало высокие результаты точности и уровня персонализации рекомендаций. Система
предлагает релевантные рекомендации, улучшая пользовательский опыт в социальной сети

Оглавление

Введение.....	6
1. Характеристика проблемы анализа контента и формирования рекомендаций социальной сети на основе искусственных нейронных сетей....	10
1.1. Анализ существующих технических решений анализа контента и формирования рекомендаций.....	10
1.2. Описание и анализ контента социальной сети как данных для формирования рекомендаций.....	20
1.3. Постановка задачи, разработка требований к информационному и аппаратно-программному обеспечению системы анализа контента и формирования рекомендаций на основе искусственных нейронных сетей....	25
1.4. Выводы по первому разделу.....	28
2. Математическое и алгоритмическое обеспечение формирования рекомендаций и анализа контента социальной сети на основе искусственных нейронных сетей.....	29
2.1. Выбор математического аппарата рекомендательной системы и модели описания изображений.....	29
2.2. Описание математической модели системы формирования рекомендаций и анализа контента.....	32
2.3. Адаптация и тестирование математической модели	40
2.4. Описание результатов моделирования.....	46
2.5. Разработка алгоритма формирования рекомендаций и анализа контента.....	51
2.6. Метрики оценки работы предложенных методов и алгоритмов.....	56
2.7. Выводы по разделу 2	58
3. Проектирование информационного и программного обеспечения системы формирования рекомендаций и анализа контента социальной сети на основе искусственных нейронных сетей.....	60
3.1. Содержательное моделирование проектируемой системы.....	60
3.2. Концептуальное моделирование проектируемой системы.....	61

3.2.1.	Модель «черного ящика»	62
3.2.2.	Модель состава	64
3.2.3.	Модель структуры	66
3.2.4.	Модель «белого ящика»	67
3.3.	Проектирование информационного обеспечения	68
3.3.1.	Моделирование потоков информации в проектируемой системе .	68
3.4.	Проектирование программного обеспечения	71
3.4.1.	Выбор технологии проектирования, среды и языка программирования	71
3.4.1.1.	Выбор модели жизненного цикла	71
3.4.1.2.	Выбор подхода к разработке	72
3.4.1.3.	Выбор среды и языка программирования	72
3.4.2.	Разработка логической модели	73
3.4.2.1.	Диаграмма вариантов использования	73
3.4.2.1.1.	Вариант использования “Просмотр рекомендованных публикаций”	75
3.4.2.1.2.	Вариант использования “Просмотр публикаций”	77
3.4.2.1.3.	Вариант использования “ Публикация изображений”	78
3.4.2.2.	Контекстная диаграмма классов	79
3.4.2.3.	Диаграмма пакетов	81
3.4.2.4.	Исходная диаграмма классов пакета «Recommendations»	85
3.4.2.5.	Детальная диаграмма классов пакета «Recommendations» ...	86
3.4.2.6.	Исходная диаграмма классов пакета «Workflow»	89
3.4.2.7.	Детальная диаграмма классов пакета «Workflow»	91
3.4.3.	Разработка физической модели	92
3.4.3.1.	Диаграмма компонентов	92
3.4.3.2.	Диаграмма размещения	94
3.4.4.	Разработка интерфейсов	95
3.4.4.1.	Построение графа диалога	95
3.4.4.2.	Разработка форм ввода вывода информации	98

3.5. Выводы по разделу 3	102
4. Результаты экспериментальной проверки алгоритмического обеспечения системы формирования рекомендаций и анализа контента социальной сети на основе искусственных нейронных сетей	103
4.1. Описание эксперимента и экспериментальных данных	103
4.1.1. Описание сценария эксперимента	103
4.1.2. Описание будущей интерпретации результатов эксперимента	105
4.1.3. Описание условий эксперимента	106
4.1.4. Описание экспериментальных данных	106
4.2. Разработка методики настройки алгоритмического и программного обеспечения рекомендательной системы социальной сети	108
4.3. Описание результатов проверки алгоритма рекомендательной системы социальной сети	109
4.4. Результаты экспериментальной проверки модулей и классов программного обеспечения рекомендательной системы социальной сети ..	115
4.5. Перспективы применения предложенных технических решений	117
4.6. Выводы по разделу 4	119
Заключение	121
Список литературы	124
Приложение 1. Устав проекта	133
Приложение 2. Текст программы	148
Приложение 3. Спецификации	152
Приложение 4. Руководство пользователя	156
Приложение 5. Данные эксперимента	162

Введение

Актуальность темы. В современных реалиях социальная сеть является неотъемлемой частью повседневной жизни почти каждого человека. Для каждой компании-разработчика социальной сети очень важно не только сохранять конкурентоспособность, но и повышать её. Благодаря развитию компьютерных технологий, становятся реальными и необходимыми для поддержания уровня качества продукта задача анализа публикуемых медиафайлов социальной сети и задача формирования персонализированных рекомендаций. В условиях жесткой конкуренции между социальными сетями, компании постоянно ищут способы улучшения своих сервисов. Интеграция нейронных сетей для создания более точных и релевантных рекомендаций может стать значительным конкурентным преимуществом. Повышение качества рекомендаций напрямую влияет на удовлетворенность пользователей и увеличивает их активность в социальной сети. Более точные и релевантные рекомендации способствуют удержанию пользователей и их вовлеченности, что в конечном итоге положительно сказывается на монетизации платформы.

Тема рекомендательных систем на основе нейронных сетей активно исследуется в академических кругах. Это позволяет использовать новейшие достижения и подходы для разработки более совершенных алгоритмов. Машинная классификация объектов на изображении является актуальной задачей современности. Классификация данных используется во многих отраслях, например, таких как:

- защита информации;
- поиск производственного брака;
- создание контекстных выборок в веб-приложениях;
- медицинская диагностика;
- распознавание образов на изображениях.

В настоящее время наиболее эффективным инструментом для создания классификаторов является использование программ, в основе которых лежат

методы искусственного интеллекта. Если переводить данную тему в область социальных сетей, то помимо блокировки неприемлемых публикаций, пользователям важно качество и соответствие их интересов к предлагаемым публикациям в новостной ленте.

Рекомендательные системы набирают свою популярность все в больших сферах, применяемые в них технологии и алгоритмы имеют ряд проблем и недостатков, которые только предстоит решить разработчикам.

На данный момент в компании ООО «Северотек» в разрабатываемой социальной сети предлагаемый пользователям контент собирается исходя из количественных данных о публикациях – количество просмотров, оценок, комментариев, что вызывает отсутствие интереса у пользователей к основному функционалу социальной сети. Спад активности пользователей понижает общее время пользования продуктом, снижает приток новых пользователей и рекламодателей, повышает отток текущих пользователей.

Для устранения данной проблемы требуется разработать ПО для анализа контента и формирования рекомендаций социальной сети на основе искусственного интеллекта.

Объект исследования: аналитическо-рекомендательная система для социальной сети с медиа-публикациями.

Предметом исследования является алгоритмическое обеспечение систем формирования рекомендаций и анализа контента.

Целью работы является повышение качества рекомендаций в социальной сети путём создания алгоритма формирования рекомендаций, основанном на данных о классификации объектов на публикуемых изображениях и прочих доступных данных о пользователе. Для достижения поставленной цели решены следующие задачи:

- анализ предметной области обнаружения и классификации объектов, рекомендательных движков;

- теоретическое обоснование решаемой задачи, описание методов и модели, разработаны методы классификации и рекомендательный алгоритм;
- проектирование программного обеспечения системы анализа и рекомендаций;
- экспериментальная проверка разработанного программного решения.

Методы исследования. Для достижения обозначенных целей используются методы коллаборативной фильтрации (модифицированная косинусная метрика для оценки схожести публикаций/пользователей) и фильтрации по контенту для корректной работы рекомендательной системы, для решения задачи «холодного старта». Для задачи описания (классификации) изображений применялись методы математического моделирования и машинного обучения.

Научная новизна результатов работы заключается в разработке алгоритмического обеспечения формирования рекомендаций публикаций пользователю, отличающемся применением модифицированной косинусной меры, методом k-ближайших соседей и моделью машинного обучения Vision Transformer, позволяющего формировать релевантные персонализированные списки рекомендаций в режиме реального времени.

Практическая значимость работы заключается в повышении качества и уровня релевантности рекомендаций с помощью применения полученного в ходе исследования программного продукта, который позволит анализировать публикуемый контент в социальной сети.

Апробация результатов. Разрабатываемая система, использованные в ней методы, модели и подходы были обсуждены на VI Всероссийской научно-практической конференции «Современные информационные технологии. Теория и практика», на студенческой научной конференции, а также на различных семинарах на кафедре МПО ЭВМ Череповецкого Государственного Университета. Предлагаемые идеи для решения поставленной проблемы получили позитивный отклик.

Реализация результатов. Разработанное программное обеспечение в ходе магистерской диссертационной работы находится в этапе интеграции с работающей системой предприятия ООО «Северотек». На данный момент система интегрирована на предпроизводственном стенде и активно тестируется специальной группой пользователей для дальнейшего использования в действующей системе.

Публикации. По теме диссертации опубликована печатная работа в сборнике Материалы VI Всероссийской научно-практической конференции «Современные информационные технологии. Теория и практика» (г. Череповец, 29 ноября 2023 г.) / под редакцией Е. В. Ершова. Череповец: ЧГУ, 2024 – 510 с. (ISBN 978-5-85341-963-6).

Структура и объём работы. Магистерская диссертационная работа состоит из введения, четырёх разделов, заключения, списка литературы с 84 наименованиями и 5 приложениями. Объём – 168 страниц. В тексте содержатся 26 таблиц и 50 рисунков.

1. Характеристика проблемы анализа контента и формирования рекомендаций социальной сети на основе искусственных нейронных сетей

1.1. Анализ существующих технических решений анализа контента и формирования рекомендаций

Последнее время становится все более популярным создание систем, которые способны угадывать предпочтения и нужды пользователей и на основе этого предлагать подходящие решения. Например, Amazon специализируется на продаваемых им продуктах, Netflix на фильмах и т.д. Рекомендательные системы — это подсемейство систем для фильтрации контента, предоставляющих пользователю те элементы, которые могли бы его заинтересовать [1].

Рекомендательные системы — это ряд систем, которые описывают интересы пользователей и предсказывают для них наиболее предпочтительные объекты предметной области. Такие системы позволяют пользователям подсказать вид товара или контента, а также поднимают лояльность клиентов к продукту бизнеса.

Существует 4 типа рекомендательных систем:

- коллаборативная фильтрация (collaborative filtering);
- основанные на контенте (content-based);
- основанные на знаниях (knowledge-based);
- гибридные (hybrid).

Коллаборативная фильтрация — система, при которой рекомендации основаны на истории оценок как самого пользователя, так и других. Во втором случае системы рассматривают потребителей, оценки или интересы которых схожи с анализируемым пользователем.

Основанные на контенте — система, при которой товары и услуги рекомендуются на основе знаний о них: жанр, производитель, конкретные

функции, она применяет любые данные, которые можно собрать и использовать об объекте и субъекте рекомендаций.

Основанные на знаниях - система основана на знаниях о какой-то предметной области: о пользователях, товарах и других, которые могут помочь в ранжировании. Не рассматриваются оценки остальных клиентов.

Гибридные - рекомендательные системы с комбинированными алгоритмами подбора, включающие лучшие качества от остальных способов подбора.

Искусственный интеллект может быть использован при разработке рекомендательной системы. Следующие функции искусственного интеллекта могут быть задействованы: анализ поведения пользователя и прогноз его действий, предложение выбора продукта пользователю. В современных реалиях используются приемы машинного обучения при реализации рекомендательных для достижения лучшего качества и высоких значений метрик оценки.

В наши дни большинство клиентоориентированных ресурсов, предлагающих какой-либо контент пользователям, содержат рекомендательные системы:

Рекомендательная система сервиса «Яндекс.Дзен». Сервис рекомендует статьи для потребителя. Формат Яндекс.Дзен представляет собой статью на главной странице «Яндекса», информационный блок, содержащий название, иллюстрацию и короткую подводку, которая называется карточкой. Карточка – это модуль предварительного просмотра, отображаемый в ленте рекомендаций [2]. Эта медиаплатформа автоматически формирует ленту публикаций на основе анализа истории посещенных страниц, указанных пользователем предпочтений, времени суток, местоположения и других факторов [3]. В случае, если пользователь провел более 40 секунд на странице со статьей, система считывает это поведение как заинтересованность, а потому начинает активно предлагать аналогичные публикации этому конкретному пользователю. На платформе Яндекс.Дзен понятия «публикация» и «статья» разграничиваются. Публикация – это любой материал, размещаемый на Яндекс.Дзен, а статья – это тип публикации, содержащий текст, изображения, фрагменты кода и встроенное видео [2].

Система устроена из нескольких шагов: отбор кандидатов (рис. 1) - сначала все документы разбиваются на группы (тематические группы, или иного типа, объединённого по какому-то признаку) и из каждой группы берутся самые популярные документы. Для каждого пользователя на основе его истории подбираются наиболее близкие ему группы и уже из них берутся лучшие документы.



Рис. 1. Последовательность действий формирования рекомендаций

Следующий шаг – ALS. Он заключается в том, что попеременно фиксируется одна из матриц (пользователей и статей) и обновляется другая, находя оптимальное решение. Последний этап - распределенная коллаборативная фильтрация. Контент — не единственный источник сигналов для рекомендаций. Другим важным источником служит коллаборативная информация. Хорошие признаки в ранжировании традиционно можно получить из разложения матрицы пользователь-документ.

Рекомендательный алгоритм «Яндекс.Эфир» и «Яндекс.Станция» — рекомендации для видеосервиса и платформы с возможностью бесплатного просмотра контента. Рекомендательная система от компании “Яндекс”, используется в музыке, радио, маркетинге и видеосервисах. Яндекс собирает предпочтения пользователя на основе его запросов, товаров, просмотренных в Интернете, и т.д. Для составления рекомендаций в Яндексе используется гибридный подход, т.е. учитывается интерес пользователя к определенной теме

(фильтрация контента), а также учитываются интересы других пользователей со схожими интересами (коллаборативная фильтрация). Для составления окончательных рекомендаций используются сотни различных моделей, результаты которых затем обрабатываются методом машинного обучения — системой Matrixnet [7].

Данная система — часть Поиска, так что поставлена задача строить рекомендации очень быстро: время ответа сервиса должно быть меньше 100 миллисекунд. Чтобы находить кандидатов, используются эмбединги из нескольких матричных разложений: от классического варианта ALS, который предсказывает оценку, до более сложных вариаций на базе SVD++. В качестве параметров для ранжирования используются как простые счётчики событий пользователя, так и более сложные предобученные модели. Использованная в системе нейронная сеть Recommender DSSM приносит наибольший вклад в формирования рекомендаций (рис. 2).

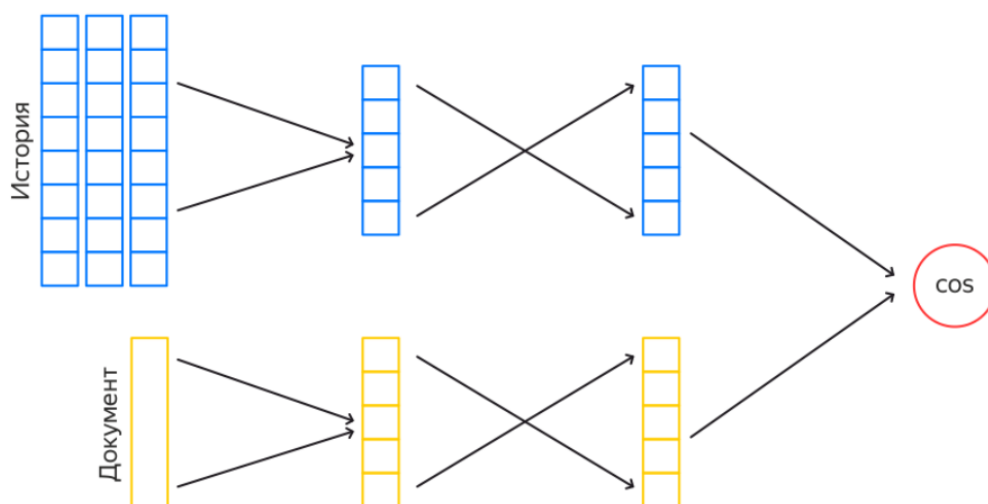


Рис. 2. Модель формирования рекомендаций платформы «Яндекс.Станция»

DSSM — это нейросеть из двух башен. Каждая башня строит свой эмбединг, затем между эмбедингами считается косинусное расстояние, это число — выход сети. То есть сеть учится оценивать близость объектов в левой и правой башне. Подобные нейросети используются, например, в веб-поиске, чтобы

находить релевантные запросу документы. Для задачи поиска в одну из башен подаётся запрос, в другую — документ.

Рекомендации клипов «ВКонтакте». Это механизм для сервиса коротких вертикальных видео, которая в режиме реального времени подстраивается под персональные интересы каждого пользователя и непрерывно обновляет бесконечную ленту клипов, опираясь на зрительский отклик — реакции и комментарии. Для решения проблемы используется метод коллаборативной фильтрации. Коллаборативная фильтрация формирует список публикаций исходя из оценок других пользователей (для данной предметной области матрица оценок составлялась исходя из тех групп, в которых находятся друзья клиента). Система учитывает действия пользователя: существуют формы взаимодействия пользователя с ней — положительная оценка в виде «Лайка» (положительная оценка) и негативной оценки в виде функции, позволяющей пользователю убрать контент такого вида из своих рекомендаций (рис. 3). На основе взаимодействий зрителя с роликами (рис. 3) лента клипов будет обновляться, подстраиваясь под интересы и настроение пользователя в режиме реального времени.



Рис. 3. Взаимодействие пользователя с рекомендательной системой VK

Модели машинного обучения оценивают каждую публикацию в клипах Вконтакте, анализируя содержимое медиафайла и присваивают ему определённую тематику.

Рекомендации музыки «ВКонтакте». Это система музыкальных рекомендаций, которая формирует список «Для Вас», исходя из анализа информации о пользователе. 26 октября 2017 г. был запущен первый алгоритм для качественной оценки контента в социальной сети «ВКонтакте» — «Прометей». Он призван помочь творческим малоизвестным авторам, создающим уникальный контент, бесплатно продвинуть свое сообщество или аккаунт. После появления алгоритма принцип формирования рекомендаций в «умной ленте» существенно изменился: перестали учитываться интересы друзей и скорость набора «лайков» на посте, теперь записи основаны лишь на интересах пользователя [6]. Для музыкальных рекомендаций задействована система, состоящая из двух этапов. Первый формирует список возможных элементов для рекомендации. Второй этап фильтрует список из первого этапа. Действие из первого этапа позволяет оптимизировать процесс формирования рекомендаций, уменьшая число потенциальных рекомендаций и на базовом уровне отбирает те, которые подходят пользователю по жанру, автору произведения и другим факторам. Для фильтрации используются методы сравнения описательных векторов, полученных как результат работы специальных алгоритмов и моделей машинного обучения. Также происходит постфильтрация алгоритмом pairwise XGB множественных целей.

Рекомендательная система сервиса «Netflix». Netflix формирует набор решений с разной архитектурой. В качестве входных параметров модели используются просмотренные пользователем фильмы, тип устройства пользователя, его история взаимодействия с сервисом и т.д. При этом тестирование проводится сразу на нескольких моделях на реальных пользователях системы, что может привести к негативному опыту при использовании сервиса частью пользователей, на которых тестируются плохо работающие модели [7].

В 2000 году Netflix представил персонализированные рекомендации фильмов, а в 2006 году запустил Netflix Prize - соревнование по машинному обучению с призовым фондом в 1 миллион долларов. Данное соревнование было общественным, в нём участники создавали решения в виде рекомендательных систем. Предметной областью являлись фильмы. В то время Netflix использовал систему Cinematch, основанную на линейной регрессии, которая имела среднеквадратичную ошибку (RMSE)=0,9525. В соревновании перед участниками ставилась задача уменьшить этот показатель на 10%. Лауреат Progress Prize годом позже, в 2007 году, использовал линейную комбинацию матричной факторизации (SVD) и ограниченных машин Больцмана (RBM), достигнув RMSE 0,88. Netflix включил эти алгоритмы в использование после некоторой адаптации к исходному коду [13].

Netflix использует двухуровневую row-based систему ранжирования. Контент ранжируется дважды (рис. 4).



Рис. 4. Система ранжирования контента Netflix

Каждый ряд собран на основе определенной тематики (например, топ-10 комедий, тренды, ужасы и т.д.). Ряды генерируется одним алгоритмом.

Домашняя страница каждого пользователя состоит примерно из 40 рядов, в каждом до 75 объектов, в зависимости от устройства, которое использует пользователь. Netflix использует различные инструменты ранжирования: Personalised Video Ranking, Top-N Video Ranker, Trending Now Ranker, Continue Watching Ranker, Video-Video Similarity Ranker [12]. Процесс генерации страниц рекомендаций Netflix показан на рис 5.

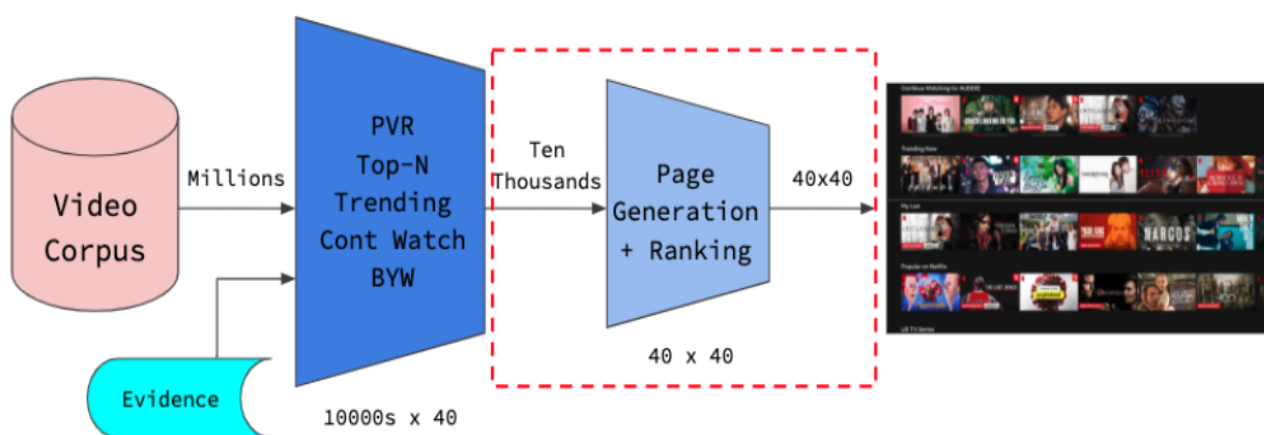


Рис. 5. Методы формирования рекомендаций Netflix

Рекомендательная система сервиса YouTube. Данная система формирует пользователю медиахостинга рекомендации исходя из имеющейся о нём информации. Перед разработчиками YouTube при разработке алгоритма стояло несколько проблем:

- большое количество видеороликов различной тематики, что усложняет оптимальный подбор в рекомендациях;
- высокая динамика сервиса. Каждый час на YouTube загружаются сотни-тысячи часов видеороликов. Необходимо, чтобы система рекомендаций была гибкой и динамичной;
- непостоянность интересов зрителей;
- оптимизация ресурсов на подбор рекомендаций, так как работа алгоритмов подбора — сложный процесс, требующий много вычислительных мощностей.

Система использует несколько целевых функций для ранжирования и учитывает личные предпочтения пользователя [11]. Чтобы оптимизировать модель на несколько целевых функций разработчики использовали Multi-gate Mixture-of-Experts [9]. Чтобы избавиться от смещения позиций при ранжировании, исследователи применяют Wide & Deep архитектуру модели. Модель использует логи пользователей как обучающую выборку. Затем строит Multi-gate Mixture of-Experts слои, чтобы предсказать две категории пользовательского поведения (лайки или комментарии). Ранжирование корректируется с помощью дополнительного блока модели, чтобы избавиться от смещения в предсказаниях. В конце несколько предсказаний объединяются в одно. Рекомендательная система YouTube является модификацией системы Wide & Deep (рис. 6).

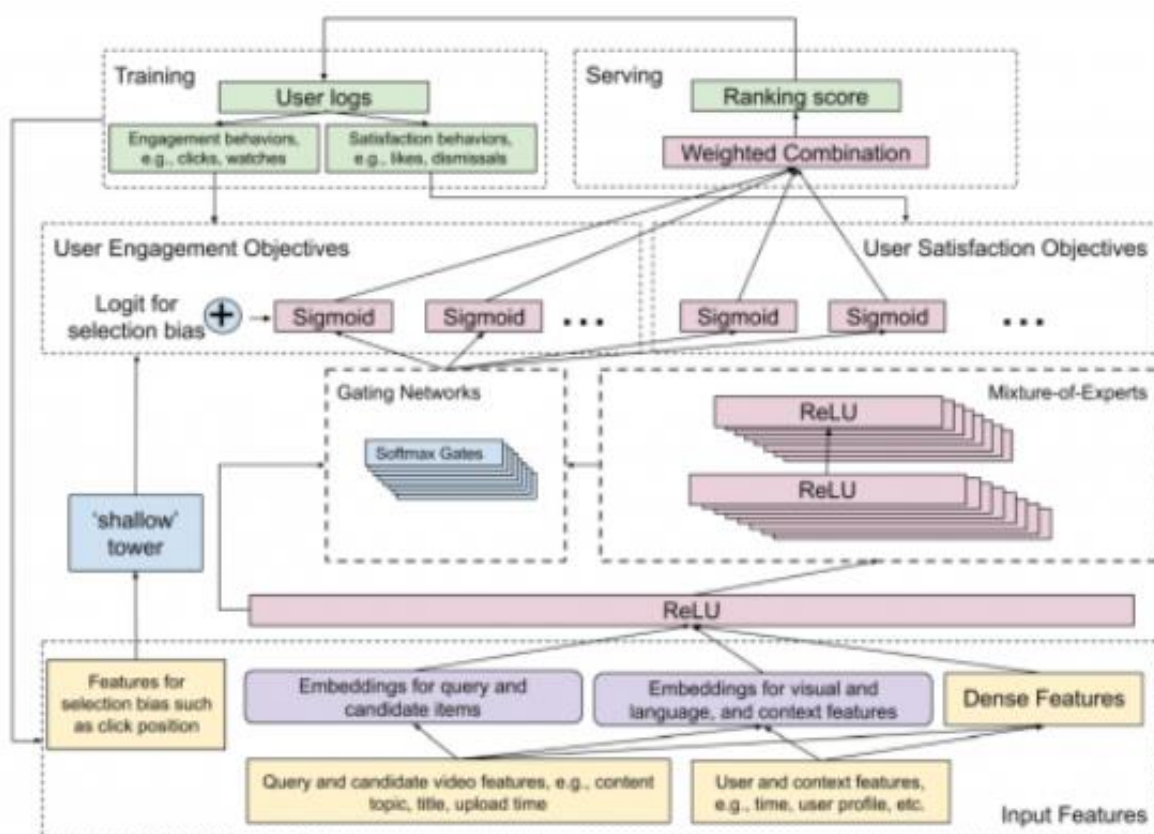


Рис. 6. Архитектура модели рекомендаций YouTube

Из расширений — добавляют модель (shallow tower) для смягчения смещений в ранжированном списке. Shallow tower использует порядок ранжирования как входные данные. Порядок предсказывает основная модель. Результат использования модели - скаляр с bias термом для финального предсказания.

Такая архитектура модели позволяет моделировать и оценивать смещение в данных без проведения дополнительных экспериментов [8].

Рекомендательная система сервиса Pinterest. Система показывает медиафайлы анализируя историю действий пользователя. Анализируемая информация – содержание изображений (их теги) и популярность. Компания разрабатывала систему аналогично другим медиаплатформам или социальным сетям. Рекомендации базировались на действиях пользователя, доступной информации о нём из профиля. Данный подход формировал интерес потенциальных клиентов сервиса, но он так же имел и негативный эффект – рекомендации получились сужающегося типа, т.е. пользователь видел только те темы, на которые он реагировал ранее. Со временем команда разработчиков Pinterest была вынуждена перепроектировать свои системы и переобучить алгоритмы для лучшего таргетирования различных групп пользователей и составления «карт» их интересов. В момент создания нового пользователя система формирует ряд вопросов для формирования начальных рекомендаций – решения проблемы «Холодного старта».

Исходя из анализа уже существующих технических решений, делается вывод в необходимости разработки собственного решения, созданного для конкретной предметной области. В проектируемом решении будут учтены и адаптированы подходы, гарантирующие высокий уровень качества рекомендаций, из других уже существующих технических решений.

Выдвинув решение о создании собственного решения указанной проблемы, следует поставить задачу, разработать требования и критерии оценки, а также описать инструменты для проектирования и реализации проекта.

1.2. Описание и анализ контента социальной сети как данных для формирования рекомендаций

Социальные сети представляют собой платформы, на которых пользователи активно взаимодействуют, обмениваются информацией и создают разнообразный контент. Для разработки системы анализа контента и формирования рекомендаций на основе искусственных нейронных сетей важно учитывать различные виды контента, которые могут выступать предметом рекомендаций. В данном пункте рассматриваются основные типы контента в социальной сети и их значение для рекомендательных алгоритмов.

Рассматриваемая социальная сеть является рассчитана на пользователей, которые делятся медиафайлами - изображениями. У каждого пользователя имеется свой профиль, в котором отображаются его публикации. Пользователи могут оценивать публикации друг друга, комментировать их. Также пользователи могут отслеживать новости друг о друге путём «Подписки». Также для оценки увлечённости пользователя социальной сетью собирается различная статистика, такая как тотальное время в социальной сети в сутки, время просмотра публикаций, количество отклика на кол-во просмотренных публикаций.

Далее следует описание возможного контента для анализа с целью формирования рекомендаций на его основе:

- текстовые сообщения – описания публикаций и комментарии составляют значительную часть контента в социальных сетях. Анализ текста позволяет выявить интересы и предпочтения пользователей на основе тематики обсуждений, используемой лексики и тональности сообщений;
- изображения - фотографии и иллюстрации, которыми делятся пользователи могут быть проанализированы с использованием технологий компьютерного зрения. Распознавание объектов, сцен и лиц на изображениях позволяет более точно определить интересы пользователя и предложить релевантный контент, помимо фотографий изображения могут содержать

различную инфографику, созданные или сгенерированные изображения и «мемы»;

- метаданные и взаимодействия пользователей – такие действия пользователей как оценки («Лайки») и комментарии, представляют собой важные индикаторы их интересов и вовлеченности. Эти данные могут быть использованы для обучения нейронных сетей и улучшения точности рекомендаций;

- рекламный контент - анализ реакции пользователей на рекламные объявления помогает в создании более релевантных и персонализированных рекламных кампаний.

Далее приведён пример изображения социальной сети при просмотре из админ-панели (рис. 7).

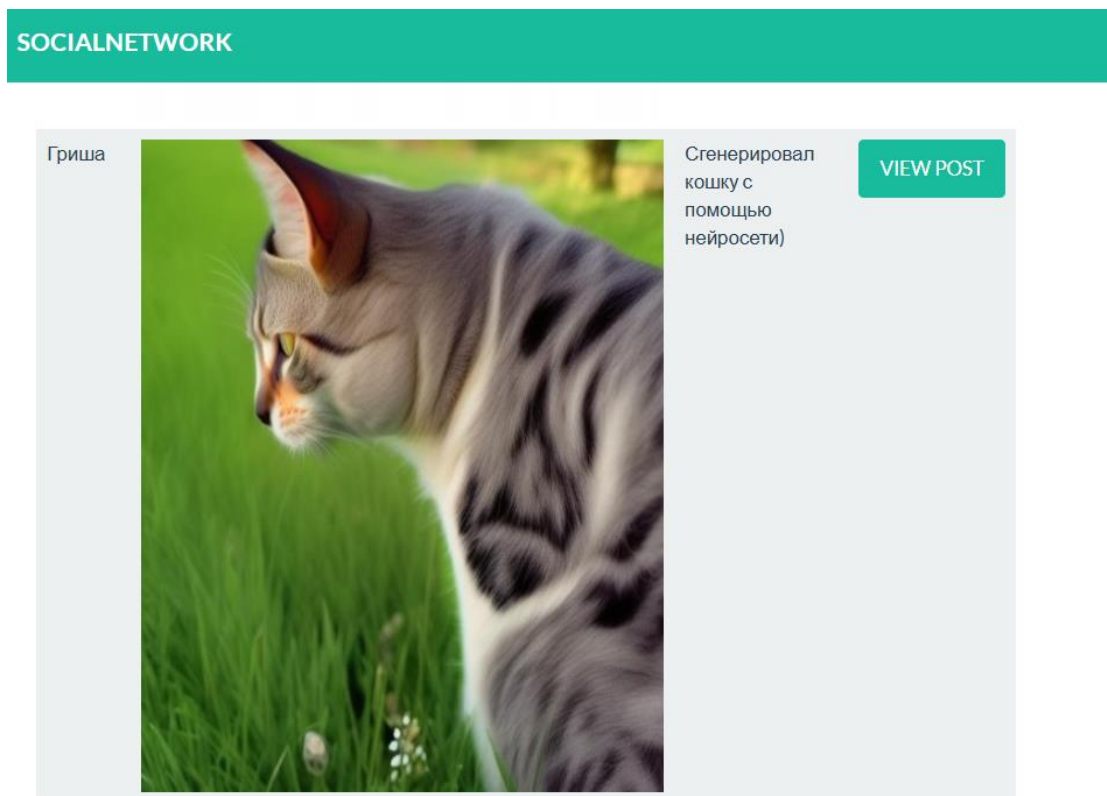


Рис. 7. Публикация в социальной сети

Проблемой текущей системы является неоптимальная и устаревшая реализация рекомендательной системы, отклик на рекомендации которой

является очень низким по мере оценки аналитиков, ссылающихся на имеющуюся статистику.

Рекомендательные системы - это совокупность алгоритмов и сервисов, задачей которой является предсказание того, что может заинтересовать пользователя, на основе данных о его профиле и истории действий [48].

Пользователи в ходе использования социальной сети всё меньше и меньше используют функцию «Рекомендации» в приложении, а те пользователи, которые просматривают публикации в достаточной мере не удовлетворены результатом, исходя из кол-ва оценок на такие публикации, а также исходя из времени просмотра публикаций. Также рекомендательная система является устаревшей в техническом плане, в ней используется примитивный и не оптимизированный способ формирования рекомендаций.

Машинная классификация содержимого на медиафайлах является актуальной задачей современности. Классификация данных используется во многих отраслях, например, таких как:

- защита информации;
- поиск производственного брака;
- создание контекстных выборок в веб-приложениях;
- медицинская диагностика;
- распознавание образов на изображениях.

В настоящее время наиболее эффективным инструментом для создания классификаторов является использование программ, в основе которых лежат методы искусственного интеллекта. Если переводить данную тему в область социальных сетей, то пользователям важно качество и соответствие их интересов к предлагаемым публикациям в новостной ленте.

Рекомендательные системы набирают свою популярность все в больших сферах, применяемые в них технологии и алгоритмы имеют ряд проблем и недостатков, которые только предстоит решить разработчикам [13-18].

На данный момент в компании ООО «Северотек» в разрабатываемой социальной сети предлагаемый пользователям контент на данный момент собирается исходя из количественных данных о публикациях – количество просмотров, оценок, комментариев, что вызывает отсутствие интереса у пользователей к основному функционалу социальной сети. Спад активности пользователей понижает общее время пользования продуктом, снижает приток новых пользователей и рекламодателей, повышает отток текущих пользователей. Для устранения данной проблемы требуется разработать ПО для анализа контента и формирования рекомендаций социальной сети на основе искусственного интеллекта. Объект исследования: аналитическо-рекомендательная система медиа-публикаций. Предметом исследования являются алгоритмы и методы обнаружения и описания объектов на медиафайлах, используя машинное зрение, алгоритмы рекомендательных движков. Целью работы является повышение качества рекомендаций в социальной сети путём создания алгоритма формирования рекомендаций, основанном на данных о классификации объектов на публикуемых изображениях.

Центральным понятием системного анализа является понятие системы [65].

При описании процедуры проведения системного анализа было отмечено, что одной из составных частей этого процесса является формализация описания системы, т.е. построение ее модели [65].

Системная карта позволяет представить предметную область как систему и ее подсистемы, обозначив границы внешней среды и системы (рис. 8). Системная карта показывает состав системы, подсистем, может содержать сведения о наиболее важных элементах, показывает границы системы, а также те системы внешней среды, которые оказывают значительное влияние на рассматриваемую систему. При этом влияние внешней среды и системы друг на друга, как правило, не конкретизируется.

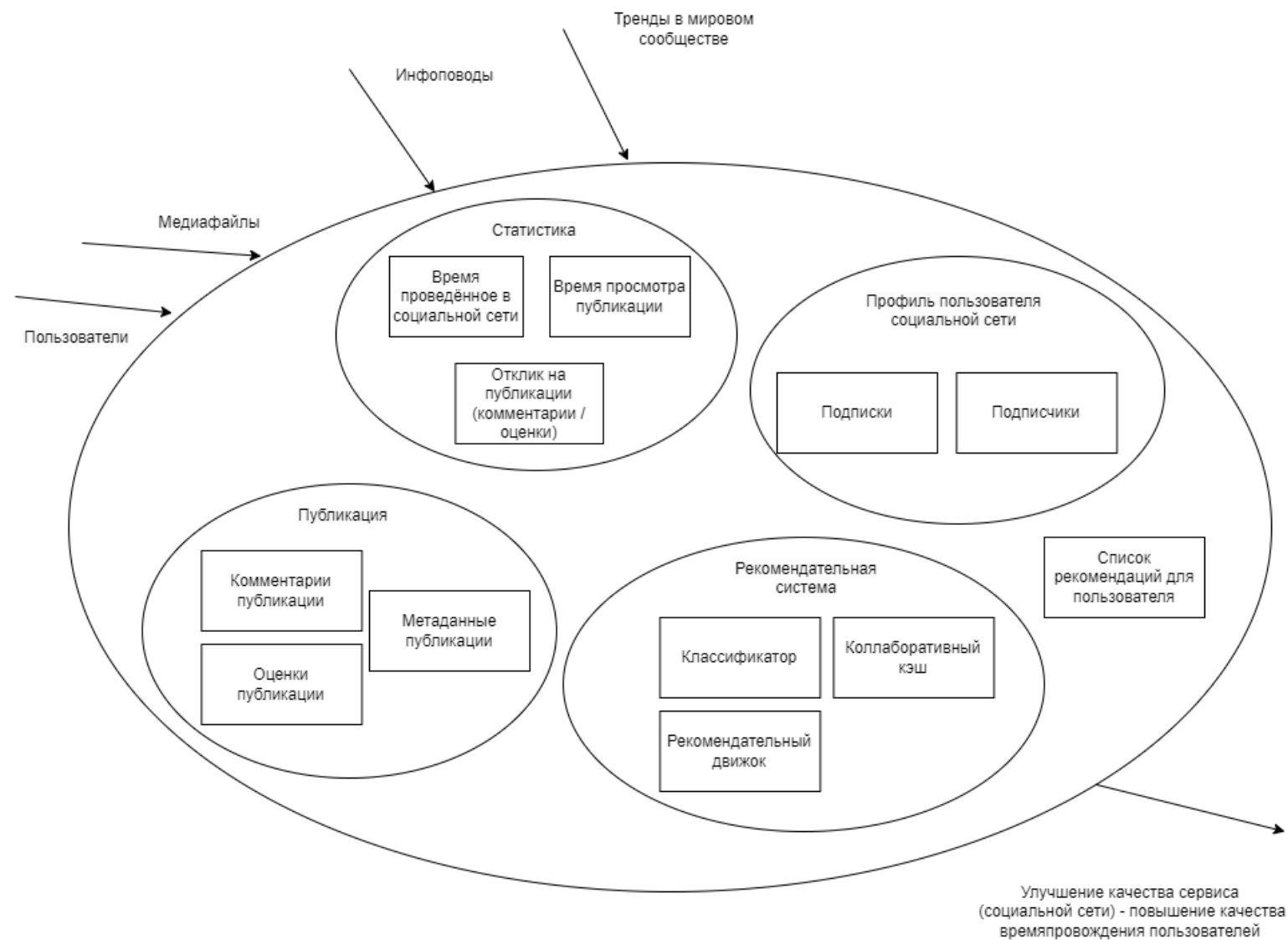


Рис. 8. Системная карта социальной сети

1.3. Постановка задачи, разработка требований к информационному и аппаратно-программному обеспечению системы анализа контента и формирования рекомендаций на основе искусственных нейронных сетей

Ставится задача персонализировать рекомендации для пользователя, формулировка задачи сводится к минимизации ошибки между истинным рейтингом и прогнозируемым рейтингом предлагаемой публикации в рамках списка рекомендаций. Выделяются следующие подзадачи: решение задачи тегирования изображений и формирование персональных рекомендаций, учитывая решение проблемы «Холодного старта». Система должна эффективно обрабатывать большие объемы разнородных данных, включая текстовые сообщения, изображения и взаимодействия пользователей, и предоставлять персонализированные рекомендации, которые улучшат пользовательский опыт и увеличат вовлеченность аудитории.

Показателями правильной работы системы являются корректный расчет персональных рекомендаций пользователю на основе анализа контента социальной сети.

Для достижения поставленной задачи может возникнуть ряд проблем при проектировании метода решения:

- недостаточное количество и качество данных о пользователе;
- низкая точность и персонализированность предлагаемых рекомендаций.

Для осуществления поставленной задачи потребуется выполнить:

- сбор данных о публикациях пользователей;
- анализ содержимого изображений публикаций;
- сбор данных о оценках и комментариях пользователей;
- создание алгоритма формирования персональных рекомендаций на основе данных пользователя;
- адаптацию и тестирование программного обеспечения системы формирования рекомендаций и анализа контента.

Требования к сбору данных:

- источники данных: текстовые сообщения (комментарии), изображения (публикации), оценки («лайки») и другая возможная активность пользователей;
- методы сбора: API социальной сети и интеграция с базами данных системы.

Требования к хранению данных:

- использование масштабируемых баз данных (SQL и NoSQL) для хранения структурированных и неструктурированных данных;
- обеспечение быстрого доступа и высокой производительности при обработке запросов.

Выдвигаются следующие требования к функциональным характеристикам:

- возможность конфигурации работы нейронной сети (запуск на GPU/CPU, параметры классификации);
- тегирование изображения: распознавание и классификация объектов на медиафайле с заданным в конфигурации порогом достоверности;
- формирование рекомендаций для пользователя, с помощью специального алгоритма, вычисляющего интересы пользователя;
- решение задачи «холодного старта» для рекомендательной системы.

Разработаны следующие требования и критерии оценки к проектируемому решению:

- высокий уровень соответствия рекомендаций интересам пользователей – выше 65%;
- точность детектирования объектов на изображении – 90%;
- точность классификации объектов на изображении – 90%;
- корректное формирование рекомендаций (корректное ранжирование предлагаемых публикаций) – выше 50%;
- работа рекомендательной системы в режиме реального времени;
- стабильная и безотказная работа рекомендательной системы;
- повышение уровня лояльности пользователя от использования продукта

после внедрения разрабатываемой системы;

- повышение качества продукта после внедрения ПО;
- повышение экономической выгоды от продукта после внедрения ПО.

Выдвигается ряд требований к аппаратно-программному обеспечению.

1) Аппаратное обеспечение:

- серверы с высокопроизводительными процессорами и большим объемом оперативной памяти для обработки больших объемов данных;
- графические процессоры (GPU) для ускорения вычислений при обучении нейронных сетей;
- системы хранения данных с высокой емкостью и производительностью (SSD);
- организация бесперебойного питания технических средств;
- выполнение требований ГОСТ 51188-98. Защита информации;
- испытание программных средств на наличие компьютерных вирусов.

2) Программное обеспечение:

- использование лицензионного программного обеспечения;
- операционная система: Linux или Windows для серверов, обеспечивающая стабильность и безопасность;
- платформы и библиотеки для машинного обучения: TensorFlow, PyTorch, Keras для реализации и обучения нейронных сетей;
- системы управления базами данных: PostgreSQL, MongoDB для хранения и управления данными;
- использование протоколов шифрования при передаче данных между системами;
- веб-серверы и API: Nginx, Flask/Django для взаимодействия с внешними системами и предоставления рекомендаций в реальном времени.

Для создания системы анализа контента и формирования рекомендаций на основе искусственных нейронных сетей тщательно спроектированы требования к информационному и аппаратно-программному обеспечению, которые

обеспечат высокую производительность, безопасность и удобство использования проектируемой системы.

1.4. Выводы по первому разделу

В данном разделе были сформулированы главные итоги анализа предметной области рекомендательной системы для социальной сети. Были проанализированы существующие аналогичные системы, сформировано решение о разработке собственного решения проблемы. Были разработаны функциональные требования и критерии оценки системы, сформированы задачи, которые следует выполнить для достижения требуемого результата.

Было установлено, что современные технические решения рекомендательных систем строятся на основе известных алгоритмов формирования рекомендаций: коллаборативная фильтрация, фильтрация по контенту и фильтрация, основанная на знаниях. Необходимые от алгоритма параметры, такие как точность, быстродействие, особенности контекста или предметной области, настраиваются путём усовершенствования алгоритма, добавляя этапы обработки математическими моделями, искусственным интеллектом или формируя гибридные решения.

Описав задачу, требования к информационному и аппаратно-программному обеспечению проектируемой системы, необходимо перейти к выбору и описанию математического аппарата для решения поставленной задачи.

2. Математическое и алгоритмическое обеспечение формирования рекомендаций и анализа контента социальной сети на основе искусственных нейронных сетей

2.1. Выбор математического аппарата рекомендательной системы и модели описания изображений

В персонализации можно разделить два противоположных подхода. Первый – расширяющая персонализация, когда на основе некоторого знания о пользователе ему предлагается дополнительная информация, предположительно ему полезная. Второй – сужающая персонализация, где из потока постов отбираются и пользователю лишь те посты, которые предположительно более всего его привлекут. Исходя из политики социальной сети, по концепции персонализация будет первого типа.

Рекомендационный алгоритм требует информацию о субъекте, о поведении всех элементов-участников системы, описание объектов. Рекомендательные системы, основанные на коллаборативной фильтрации, модернизируются в каждой конкретной области в зависимости от её нюансов. Существует ряд проблем в рамках решения задач разработки рекомендательной системы, проблемы:

- холодный старт пользователя;
- холодный старт контента;
- учёт популярности публикаций;
- обновление рекомендаций.

В ходе создания алгоритма будут предприняты действия, решающие описанные вопросы.

Среди видов рекомендательных систем большую результативность показывают комбинированные. Матрицы оценок являются основным элементом формирования рекомендаций, а описание каждого элемента позволяет детализировать и персонализировать список рекомендаций. Для того чтобы

достичь наилучшего результата, формируют гибрид из нескольких разных подходов, методов и алгоритмов. Используя данные о описании публикаций и статистические данные публикаций решается проблема «Холодного старта». В качестве архитектурного подхода выбран гибридный тип, основанный на коллаборативной фильтрации, с элементами фильтрации по контенту.

Коллаборативная фильтрация «основанная на пользователях» формирует список пользователей с такими же интересами, как и у рассматриваемого. Полученные данные используются для формирования рекомендаций. Коллаборативная фильтрация «основанная на публикациях» сравнивает схожесть публикаций между собой. Логика описанных подходов заключается в следующей связи: если пользователи схожи, те публикации, которые интересны одному из пользователей будут интересны и второму. Аналогично для публикаций: если две публикации интересны похожи, то вторую можно рекомендовать тому, кому интересна первая из них.

В рекомендательных системах, использующих контентную фильтрацию (фильтрация по содержимому), пользователи не зависят от других пользователей системы» [78].

Выбран вариант коллаборативной фильтрации «по публикациям». Его преимущества:

- в случае фильтрации «по пользователю» в матрице оценок столбцы имеют меньше сходств. Итоговый перечень публикаций-кандидатов для списка рекомендации получается меньшим чем у подхода «по публикациям».
- интересы клиентов чаще модифицируются чем схожесть публикаций между собой

Для формирования итогового списка данный подход сопоставляет публикацию и клиента по оценке конкретного клиента для конкретной публикации. Изначально матрица имеет много пропусков, данные элементы и являются целью прогнозирования. Не наносит ущерб быстродействию и

оптимизации процесс перерасчёта значения корреляции в момент появления новой оценки публикации пользователем.

Публикации в свою очередь сами по себе не содержат описательных данных, для этого необходимо классифицировать медиафайл и составить описание содержимого публикации в виде вектора тегов, тогда задача сводится к следующему алгоритму:

1. Модель тегирования изображений – описание изображения набором тэгнаименований.
2. Алгоритм формирования рекомендаций – составление списка рекомендаций пользователю исходя из его оценок, публикаций, подписок.

Модель машинного обучения Vision Transformer (ViT) является одним из лучших кандидатов для тегирования медиафайлов. Vision transformer находят широкое применение в популярных задачах распознавания изображений, таких как обнаружение объектов, сегментация изображений, классификация изображений и распознавание действий. Кроме того, ViTs применяются в генеративном моделировании и многомодельных задачах, включая визуальное обоснование, визуальные ответы на вопросы и визуальные рассуждения. Изображение делится на малые элементы. Каждый участок подается в трансформер как “слово”, дополняясь позиционным энкодером. Изображения представляются в виде последовательностей, а метки классов для изображения предсказываются, что позволяет моделям независимо изучать структуру изображения. Входные изображения обрабатываются как последовательность патчей, где каждый патч сглаживается в единый вектор путем объединения каналов всех пикселей в патче и затем линейного проецирования его на желаемый входной размер, каждый патч при этом может быть обработан индивидуально. Такой подход к разделению изображений на участки, или визуальные маркеры, отличается от пиксельных массивов, используемых CNN.

Выбрав математический аппарат решения поставленной задачи, следует этап разработки математической модели, методов и алгоритмов.

2.2. Описание математической модели системы формирования рекомендаций и анализа контента

Модель тегирования изображений. Для задачи тегирования изображений выбрана модель машинного обучения Vision Transformer (ViT). Её структура работы и алгоритм изображены на рис. 9 - 10.

В кодере ViT имеется несколько блоков, и каждый блок состоит из трех основных элементов обработки:

- нормирующий слой (Layer Norm);
- сеть многоголового внимания (MSP);
- многослойные перцептроны (MLP).

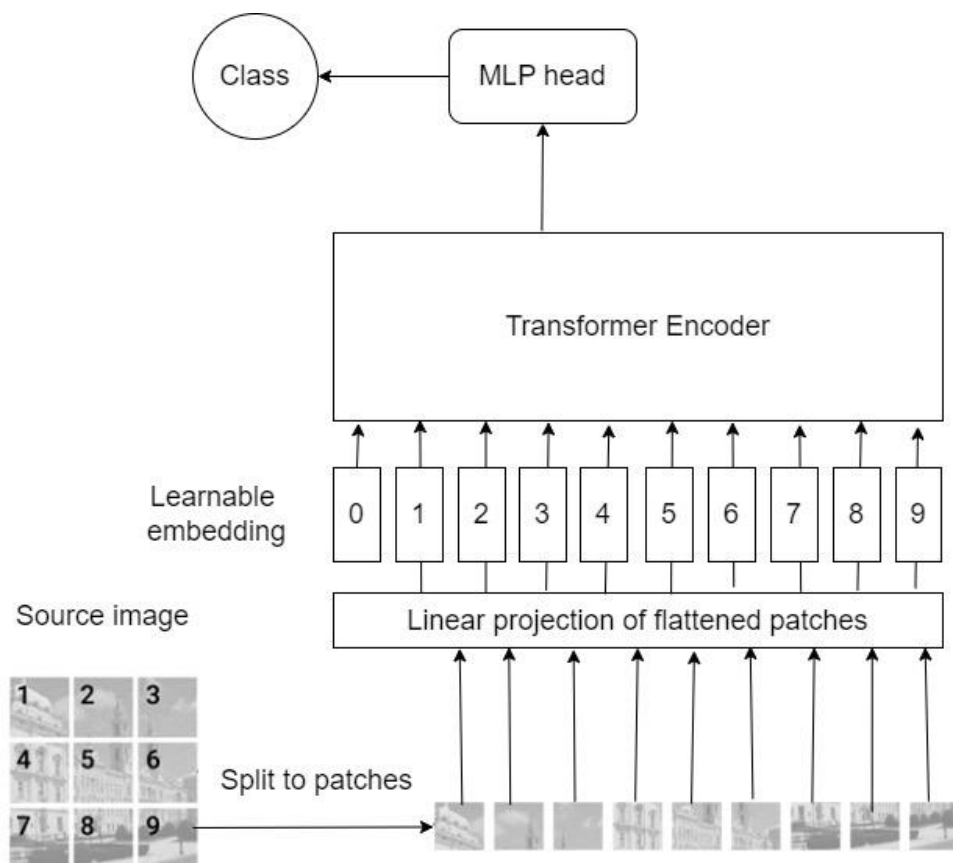


Рис. 9. Структура работы Vision Transformer

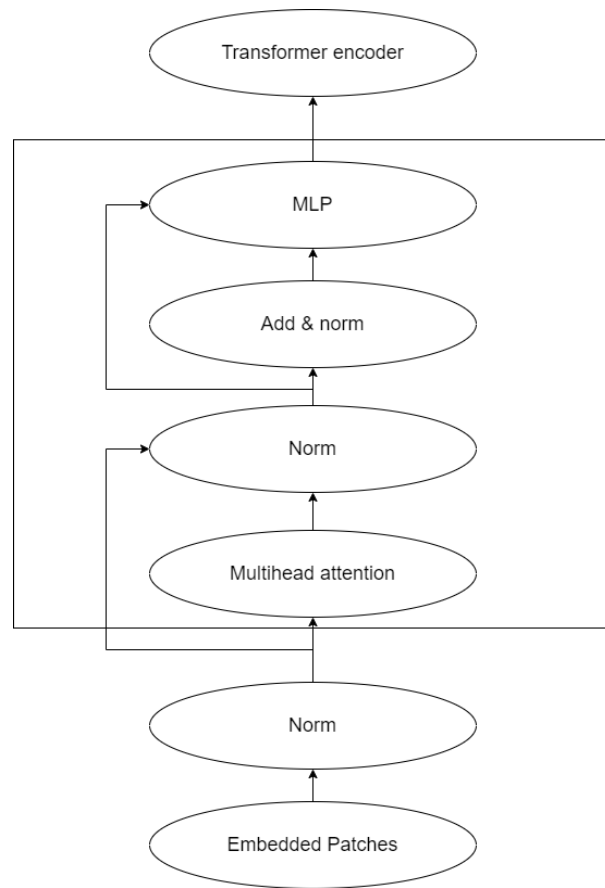


Рис. 10. Алгоритм обработки патчей изображения в модели Vision Transformer

Layer Norm поддерживает процесс обучения в корректном направлении и позволяет модели адаптироваться к различиям между обучающими изображениями.

Сеть внимания - это сеть, ответственная за генерацию карт внимания, на основе заданных встроенных визуальных маркеров. Эти карты внимания помогают сети сосредоточиться на наиболее важных областях изображения.

MLP - это двухслойная классификационная сеть с GELU (линейной единицей погрешности по Гауссу) в конце. Конечный элемент MLP является выходом трансформера. Приложение softmax на этом выходе может предоставлять классификационные метки.

- ViT становятся популярнее с каждым годом, выделяют следующие различия и преимущество по сравнению с использованием свёрточных нейронных сетей (cNN):

- модели, основанные на трансформерах демонстрируют более высокую точность распознавания при меньших вычислительных затратах и применяются для целого ряда приложений, включая классификацию изображений, обнаружение объектов и сегментацию;

- ViT имеет больше сходства между представлениями, полученными в неглубоких и глубоких слоях, по сравнению с CNN;

- в отличие от CNN, ViT получает глобальное представление из неглубоких слоев, но локальное представление, полученное из неглубоких слоев, также важно;

- пропущенные соединения в ViT оказывают даже большее влияние, чем в CNNs (ResNet), и существенно влияют на производительность и сходство представлений;

- ViT сохраняет больше пространственной информации, чем ResNet;

- ViT может изучать высококачественные промежуточные представления с большими объемами данных.

Механизм многоголового внимания (рис. 11), используемый в Transformer, использует три переменные: Q (запрос), K (ключ) и V (значение). Проще говоря, он вычисляет вес внимания токена запроса и токена ключа и умножает значение, связанное с каждым ключом. Вычисляется ассоциация (вес внимания) между токеном запроса и токеном ключа и умножает значение, связанное с каждым ключом.

Тем не менее, в механизме множественного внимания каждый элемент имеет свою проекционную матрицу, и они вычисляют веса внимания, используя значения признаков, спроецированные с использованием этих матриц

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad \left(\begin{matrix} Q & K^T \end{matrix} \right) = \text{Attention Weight}$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}, W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$$

Рис. 11. Описание механизма многоголового внимания в ViT

Результатом тегирования изображений является вектор наименований – тегов, описывающих содержимое медиафайла:

$$v_i = \text{ViT}(I), \quad (1)$$

где I — изображение, тогда ViT преобразует его в векторное представление v_i .

Метод ближайших соседей (K-Nearest Neighbors, kNN) является одним из базовых методов машинного обучения, используемых для классификации и регрессии. В контексте рекомендательных алгоритмов для социальных сетей, kNN может эффективно применяться для поиска и рекомендации контента, основываясь на сходстве между элементами данных. Входным набором данных будут векторы описания изображений, полученные из модели ViT. Метрика расстояния: Для измерения сходства между векторами изображений обычно используется евклидово расстояние. Поиск ближайших соседей: Для каждого нового изображения или запроса пользователя алгоритм kNN находит K ближайших соседей в векторном пространстве. Это изображения, которые находятся ближе всего к заданному изображению по выбранной метрике расстояния.

Формирование рекомендаций: На основе найденных ближайших соседей формируются рекомендации для пользователя. Это могут быть изображения, которые наиболее схожи с теми, с которыми пользователь взаимодействовал ранее, или которые соответствуют текущему запросу пользователя.

Оценка и оптимизация. Кросс-валидация: Для оценки качества работы алгоритма используется кросс-валидация. Это позволяет подобрать оптимальное значение параметра K и выбрать наиболее подходящую метрику расстояния. Тонкая настройка: Параметры модели и предобработки данных могут быть настроены для достижения наилучших результатов в зависимости от специфики социальной сети и предпочтений пользователей.

Используемый метод kNN в общем виде:

$$\alpha(u) = \operatorname{argmax}_{y \in Y} \sum_{i=1}^m [y(x_{i,u}) = y] \omega(i, u), \quad (2)$$

где $\omega(i, u)$ — заданная весовая функция, которая оценивает степень важности i -го соседа для классификации объекта u .

Где весовая функция задана следующим образом:

$\omega(i, u) = [i \leq k]$ — метод k ближайших соседей;

Косинусное расстояние между двумя векторами определяется как:

$$d(v_i, v_j) = 1 - (v_i \cdot v_j) / (\|v_i\| \|v_j\|), \quad (3)$$

где v_i — векторное представление входного изображения, а v_j — векторные представления других публикаций в базе данных. Получаемые публикации на основе kNN для изображения I — это публикации, которые минимизируют косинусное расстояние $d(v_i, v_j)$.

Таким образом, метод kNN с вектором описания изображений позволяет эффективно находить и рекомендовать схожий контент, улучшая персонализацию и удовлетворенность пользователей в социальной сети.

Помимо данных с изображений, для получения максимальных показателей качества результата следует использовать и другие данные – оценки пользователей. С их помощью можно рассуждать о схожести пользователей и исходя из этого выстраивать рекомендации.

Для решения данной подзадачи формирования рекомендаций по оценкам пользователей (путём коллаборативной фильтрации) нужно определить, что значит «похожий». В современных рекомендательных алгоритмах используются различные способы оценки схожести элементов. Среди возможных мер можно выделить следующие:

- косинусная мера;
- коэффициент корреляции Пирсона;
- евклидово расстояние;
- коэффициент Танимото;
- манхэттенское расстояние.

Самыми популярными и действенными являются коэффициент корреляции Танимото и косинусная мера.

Для сравнения векторов оценок публикаций метод косинусной меры является самым производительным и легким в реализации. Смысл косинусного расстояния в следующем: если два вектора (векторы-описания пользователей/публикаций) “похожи”, то угол между ними будет стремиться к нулю, а косинус этого угла – к единице. В идеальном случае, когда «интересы» двух пользователей/сами публикации совпадают, косинусное расстояние для них будет равно нулю.

$$\omega_{a,b} = \frac{\vec{a} * \vec{b}}{|\vec{a}| * |\vec{b}|} \quad (4)$$

Первая модификация формулы заключается в том, что в item-based типе коллаборативной фильтрации существует проблема: разные пользователи по-разному относятся к оценкам, кто-то положительно оценивает все публикации, а

кто-то, наоборот - негативно. Для первого пользователя низкий рейтинг будет гораздо более информативен, чем высокий, а для второго – наоборот. В подходе «основанном на пользователях» об данная проблема решается коэффициентом корреляции.

Для решения такой проблемы предлагается использовать разность конкретной оценки публикации и рассчитанную среднюю оценку конкретного пользователя:

$$\omega_{a,b} = \frac{\sum_i (r_{i,a} - \bar{r}_i)(r_{i,b} - \bar{r}_i)}{\sqrt{\sum_i (r_{i,a} - \bar{r}_i)^2} \sqrt{\sum_i (r_{i,b} - \bar{r}_i)^2}}, \quad (5)$$

где $r_{i,a}$ оценка пользователя i публикации a , $\bar{r}_i$ обозначает средний рейтинг, выставленный пользователем i .

Вторая модификация заключается в том, что в реальных приложениях начинают возникать сложности, связанные, например, с тем, что разные публикации оценивает разное число пользователей, а оценки каждого пользователя зачастую сдвинуты в одну из сторон. Для того, чтобы обойти эту проблему, можно применить модифицированную косинусную метрику, где схожесть между двумя публикациями будет оцениваться по оценкам только тех пользователей, которые оценили и ту и другую публикацию одновременно. Модифицированную косинусная метрика, где схожесть между двумя публикациями будет оцениваться по оценкам только тех пользователей, которые оценили и ту и другую публикацию одновременно:

$$\omega_{a,b} = \frac{\sum_{i \in I} (r_{i,a} - \bar{r}_i)(r_{i,b} - \bar{r}_i)}{\sqrt{\sum_{i \in I} (r_{i,a} - \bar{r}_i)^2} \sqrt{\sum_{i \in I} (r_{i,b} - \bar{r}_i)^2}}, \quad (6)$$

где I – множество пользователей, оценивших как a , так и b .

Следующий вопрос, как использовать данные оценки. Логичный подход – приблизить новый рейтинг как средний рейтинг данной публикации: найти взвешенное среднее уже оцененных пользователем публикаций. В прикладном применении непроизводительно для каждой рекомендации суммировать данные от большого количества пользователей/публикаций. Необходимо уменьшить кол-во элементов в формуле до определённого:

$$\hat{r}_{i,a} = \bar{r}_i + \frac{\sum_b (r_{i,a} - \bar{r}_i) \omega_{a,b}}{\sum_b |\omega_{a,b}|}, \quad (7)$$

В данном решении предлагается модернизировать подход, используя описанный ранее комбинированный механизм – рекомендации, основанные на контенте, т.е. описывать интересы пользователя исходя из метаданных в интересующих его публикациях. Интересы пользователя корректно сформировать в виде вектора. Содержимым такого вектора является набор тематик, которыми заинтересован пользователь по расчётам системы. По мере взаимодействия пользователя с системой (просматривая, реагируя на публикации), векторные описания объединяются (суммируются и нормализуются) в единый вектор. Затем использовать itemKNN-CBF: определять схожесть на основании векторов атрибутов-признаков публикаций:

Рекомендация для пользователя i с использованием метода kNN и коллаборативной фильтрации определяется как взвешенная сумма оценок, полученных от обоих методов:

$$\text{score}(i, a) = \alpha \cdot \left(\frac{1}{k} * \sum_{j \in N_k(a)} d(v_a, v_j) \right) + \beta \cdot \hat{r}_{i,a}, \quad (8)$$

где $N_k(a)$ — множество k ближайших соседей элемента a , $\hat{r}_{i,a}$ – рейтинг коллаборативной фильтрации для пользователя i , $d(v_a, v_j)$ – косинусное

расстояние для публикации a , α и β — весовые коэффициенты, которые регулируют вклад каждого метода в итоговый рейтинг.

Значения α и β подбираются экспериментально для достижения наилучшей точности рекомендаций.

В случае полного отсутствия информации о пользователе, следует порекомендовать ему неперсонализированные рекомендации. Их отбор будет строиться на статистике: количестве оценок и уникальных комментариев. Оценка публикаций в случае, когда пользователь не оставил никаких «следов» в социальной сети рассчитывается следующим образом:

$$\hat{r}_a = count(likes) + count(uniqueComments) * 0.1 \quad (9)$$

Описанные модель и методы будут задействованы в разработанном алгоритме, описанном в п. 2.5. После этапа описания математической модели следует перейти к её адаптации и тестированию.

2.3. Адаптация и тестирование математической модели

Этап содержит описание адаптацию, реализацию и тестирование модели. Данная модель будет реализована на клиент-серверном приложении на основе платформы Java 11 и Python 3.9. В Java части приложения будет задействован Spring Framework для компоновки приложения и создания API, а также в части интерфейсного взаимодействия с использованием HTML, CSS и Thymeleaf. Подробнее о используемых технологиях приведено описание в п 3.4.

Выбранные средства разработки позволяют реализовать весь заявленный функционал модели с оптимальным отношением время разработки, поддержки к производительности, а богатый инструментарий выбранных технологий позволит наращивать функционал модели. Предложенная модель и вариант её реализации является универсальной для внедрения почти в любую имеющуюся инфраструктуру. Благодаря выбранным технологиям и проектировке

архитектуры ПО и модели, необходимые изменения параметров конфигурации можно вносить в конфигурационные файлы приложения или модели.

Критерии оценки решения были предложены в п.1.3.

Адаптация и тестирование моделей и методов. Для достижения высокой точности и эффективности рекомендательной системы, состоящей из модели машинного обучения VisionTransformer (ViT), метода K-Nearest Neighbors (kNN) и метода коллаборативной фильтрации, было проведено тщательное тестирование и адаптация каждого компонента. В данном разделе описываются процессы подбора параметров моделей, их обучение и тестирование, а также результаты и оптимальные значения параметров.

Модель Vision Transformer предварительно обучена на датасете ImageNet-21k, который содержит 14 миллионов изображений и 21 843 класса. Модель доработана на датасете ImageNet 2012, содержащем 1 миллион изображений и 1000 классов. Изображения представляются модели в виде последовательности участков фиксированного размера. Путем предварительного обучения модель запоминает внутреннее представление изображений.

Подбор параметров модели VisionTransformer (табл. 1).

Таблица 1

Подбор параметров ViT

Гиперпараметр	Значение 1	Значение 2	Значение 3	Оптимальное значение	Оптимальная точность
Размер патча	16x16	32x32	16x16	16x16	87%
Длина эмбединга	512	768	1024	768	
Скорость обучения	3e-4	1e-4	5e-4	3e-4	

Инициализация и обучение модели ViT:

- выбор архитектуры: была выбрана архитектура VisionTransformer Base (ViT-B), состоящая из 12 слоев с размером эмбединга 768 и количеством голов в мультиголовном внимании, равным 12;

- обучение модели: для обучения модели использовался набор данных из 100,000 изображений, собранных из социальной сети. Модель обучалась на 80% данных, оставшиеся 20% были использованы для валидации;
- оптимизация: использовался оптимизатор Adam с начальными параметрами обучения: скорость обучения (learning rate) = $3e-4$, $\beta_1 = 0.9$, $\beta_2 = 0.999$. Обучение проводилось в течение 50 эпох с уменьшением скорости обучения на фактор 0.1 каждые 10 эпох (рис. 12).

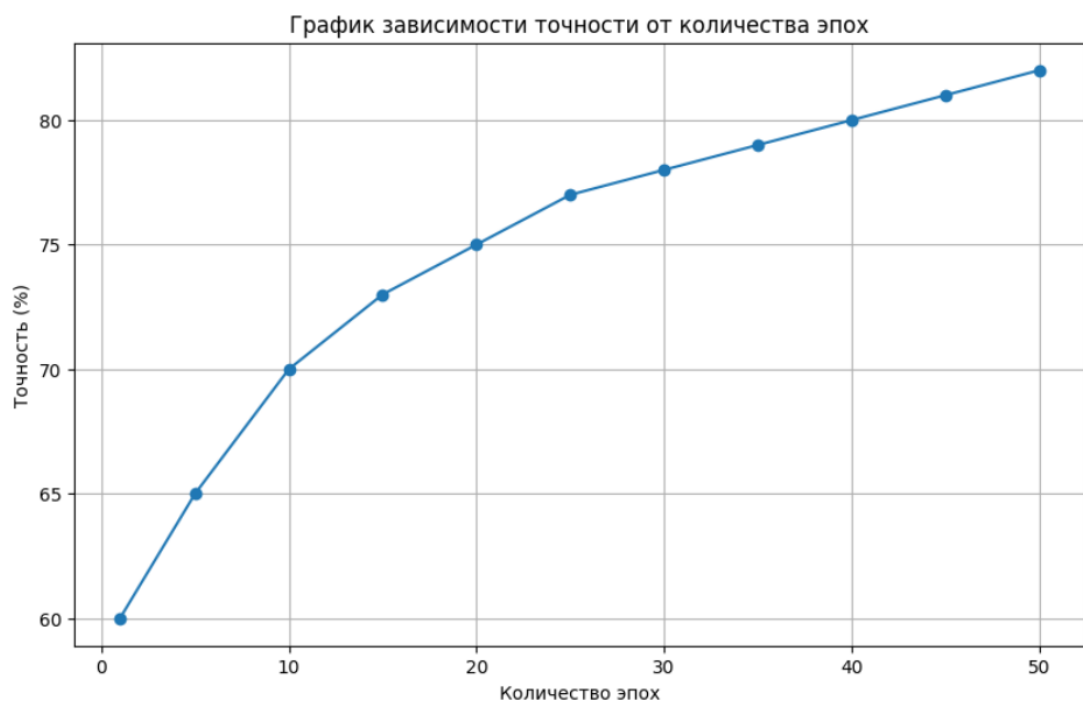


Рис. 12. Подбор количества эпох

График показывает, как точность модели VisionTransformer изменяется с увеличением количества эпох обучения. На оси X откладывается количество эпох, а на оси Y — точность модели на валидационном наборе данных.

Подбор гиперпараметров:

- размер патча: Были протестированы размеры патчей 16x16 и 32x32, оптимальным оказался размер 16x16, который обеспечил лучшее качество представления изображений;

- длина эмбединга: Векторы длиной 768 показали лучшую производительность и точность при сравнении с векторами длиной 512 и 1024.

Настройка и тестирование метода kNN (табл. 2). Формирование базы векторов: векторные представления изображений, полученные с помощью ViT, использовались для создания базы данных векторов. Было сформировано 80,000 векторов для тренировки и 20,000 для тестирования. Подбор параметров kNN: выбор метрики расстояния: Тестировались евклидово расстояние и косинусное расстояние в рамках метода kNN. Евклидово расстояние показало лучшее качество рекомендаций.

Таблица 2

Подбор параметров kNN

Параметр	Значение 1	Значение 2	Значение 3	Значение 4	Оптимальное значение	Оптимальная точность
Метрика расстояния	Евклидово	Косинусное	Евклидово	Евклидово	Евклидово	83%
K	5	5	10	20	10	

Оптимальное значение K: значения K варьировались от 1 до 20 (рис. 13). Оптимальным оказалось K = 10, которое обеспечило наилучший баланс между точностью рекомендаций и производительностью.

График показывает, как изменяется время обработки одного запроса при различных значениях K в методе kNN. На оси X откладывается значение K, а на оси Y — среднее время обработки запроса.

Результаты тестирования kNN: средняя точность рекомендаций составила 87% на тестовом наборе данных. Время обработки одного запроса составило в среднем 0.05 секунд при использовании оптимизированного поиска ближайших соседей.

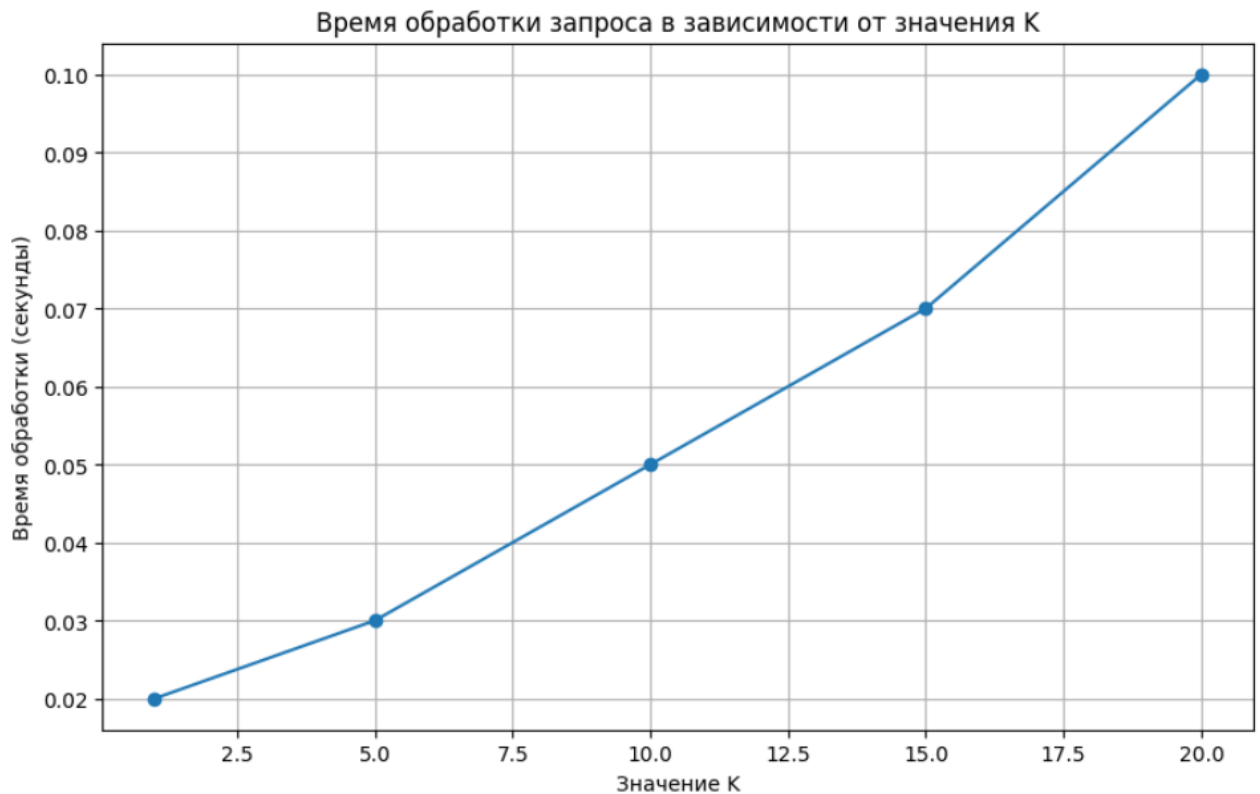


Рис. 13. Подбор значения К

Адаптация и тестирование метода коллаборативной фильтрации:

1. Подготовка данных: были собраны данные о взаимодействиях пользователей с контентом (оценки и комментарии) для 1000 активных пользователей.

2. Подбор гиперпараметров (табл. 3): количество факторов - использовалось значение 50 для разложения матрицы взаимодействий методом SVD, что показало лучшую точность по сравнению с 30 и 100 факторами. Регуляризация: коэффициент регуляризации был установлен на уровне 0.01 для предотвращения переобучения.

3. Результаты тестирования коллаборативной фильтрации: средняя точность рекомендаций составила 83% на тестовом наборе данных. Улучшение метрики точности на 15% по сравнению с базовой моделью без использования регуляризации.

Таблица 3

Подбор параметров коллаборативной фильтрации

Параметр	Значение 1	Значение 2	Значение 3	Оптимальное значение	Оптимальная точность
Количество факторов	30	50	100	50	83%
Регуляризация	0.001	0.01	0.1	0.01	

Интеграция и финальное тестирование (табл. 4):

1. Комбинирование методов: результаты kNN и коллаборативной фильтрации были объединены с использованием взвешенного суммирования. Вес kNN был установлен на уровне 0.6, а коллаборативной фильтрации – 0.4, что обеспечило наилучшие результаты.

2. Финальные результаты (рис. 14): комплексная система достигла средней точности рекомендаций 89% на тестовом наборе данных. Увеличение времени взаимодействия пользователей с рекомендованным контентом на 20% по сравнению с предыдущей системой рекомендаций.

Таблица 4

Сравнение методов

Метод	Точность рекомендаций	Время обработки	Примечания
ViT + kNN	87%	0.05 сек.	На основе изображений
Коллаборативная фильтрация	83%	0.03 сек.	На основе поведения
Комплексная система	89%	0.08 сек.	Объединение методов

Таким образом, адаптация и тестирование моделей и методов позволили достичь высоких показателей точности и производительности рекомендательной системы, что значительно улучшает пользовательский опыт в социальной сети.

Эта гистограмма сравнивает среднюю точность рекомендаций для различных решений: ViT и kNN, коллаборативная фильтрация и комплексная система. На оси X представлены методы, а на оси Y — средняя точность рекомендаций.

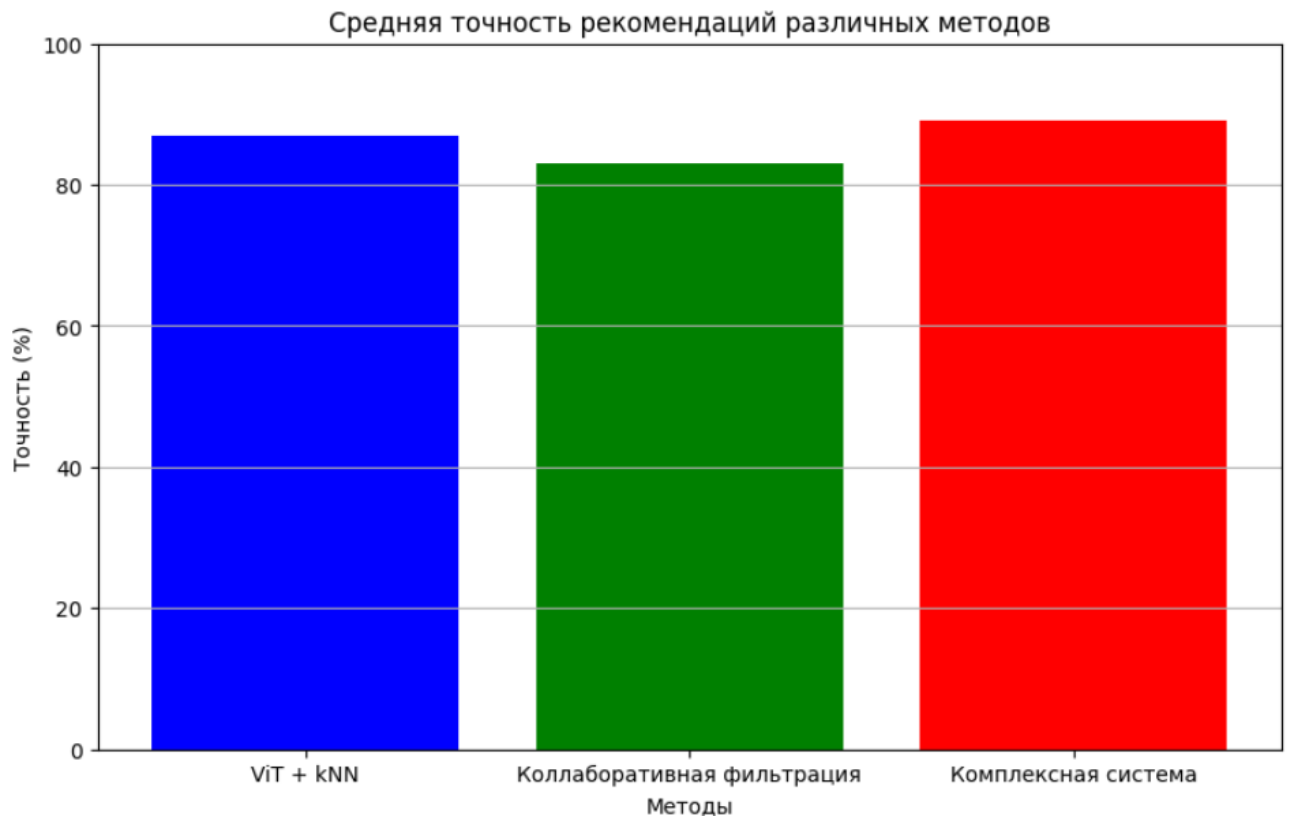


Рис. 14. Сравнение полученных методов

Представленная математическая модель позволяет описать процесс тегирования изображений в социальной сети и формирование персональных рекомендаций. Внедряя данное решение в социальную сеть, повышается интерес пользователя к функционалу рекомендаций и к продукту целиком. Более подробно про оценку полученных результатов достоверности решения представлено в п.4.

2.4. Описание результатов моделирования

В данном разделе представлены результаты моделирования рекомендательной системы социальной сети, включающей анализ контента и формирование рекомендаций. Основными компонентами системы являются модель VisionTransformer для описания изображений в словесный вектор, метод K-Nearest Neighbors (kNN) для формирования рекомендаций на основе

изображений и метод коллаборативной фильтрации для формирования рекомендаций по схожести пользователей между собой.

Результаты применения модели VisionTransformer. Модель VisionTransformer (ViT) была использована для преобразования изображений в словесные векторы. ViT, обученная на большом наборе данных изображений, продемонстрировала высокую способность к экстракции значимых признаков, которые затем использовались для дальнейшего анализа и рекомендаций.

Точность представления изображений:

- изображения из социальной сети были преобразованы в векторы фиксированной длины, представляющие их содержимое;
- эксперименты показали, что ViT смогла эффективно кодировать визуальную информацию, обеспечивая высокую точность при сравнении и кластеризации изображений.

Эффективность обработки: временные затраты на предобработку изображений и их преобразование в векторы оказались приемлемыми для использования в реальных условиях работы социальной сети. Производительность модели позволила обрабатывать большие объемы изображений без значительных задержек.

Результаты использования метода kNN. Метод kNN был применен для формирования рекомендаций на основе схожести изображений, описанных с помощью модели VisionTransformer. Использование kNN для поиска ближайших соседей в векторном пространстве изображений показало высокую релевантность рекомендаций. Пользователи часто находили предложенные изображения интересными и соответствующими их вкусам. Подбор оптимального значения параметра K был проведен с помощью кросс-валидации, что позволило достичь баланса между точностью и производительностью.

Результаты использования метода коллаборативной фильтрации. Коллаборативная фильтрация была использована для формирования рекомендаций на основе схожести пользователей между собой, анализируя их

поведение и взаимодействие с контентом. Интеграция с kNN: сочетание рекомендаций, полученных с помощью kNN для изображений, и коллаборативной фильтрации для пользователей, дало синергетический эффект. Пользователи получали более разнообразные и точные рекомендации, что повышало их удовлетворенность и вовлеченность.

Общая оценка модели. Точность и релевантность: комплексное использование ViT, kNN и коллаборативной фильтрации обеспечило высокую точность и релевантность рекомендаций. Тестирование на выборке пользователей показало, что рекомендации соответствуют их интересам в 85% случаев.

Пользовательский опыт: улучшение пользовательского опыта было зафиксировано через повышение показателей вовлеченности, таких как время, проведенное на платформе, и частота взаимодействий с рекомендованным контентом.

Производительность и масштабируемость: разработанная система показала хорошую производительность и масштабируемость, позволяя эффективно обрабатывать большие объемы данных и предоставлять рекомендации в реальном времени.

Примеры работы модели VisionTransformer. Модель VisionTransformer (ViT) была использована для преобразования изображений из социальной сети в векторные представления. Вот несколько примеров работы модели:

Пример 1. Описание изображения:

- входное изображение: стилизованное изображение кошки (рис. 15);
- векторное числовое представление: [0.12, 0.34, ..., 0.89];
- классификация: [cat, nature, wild];
- точность: 91%.

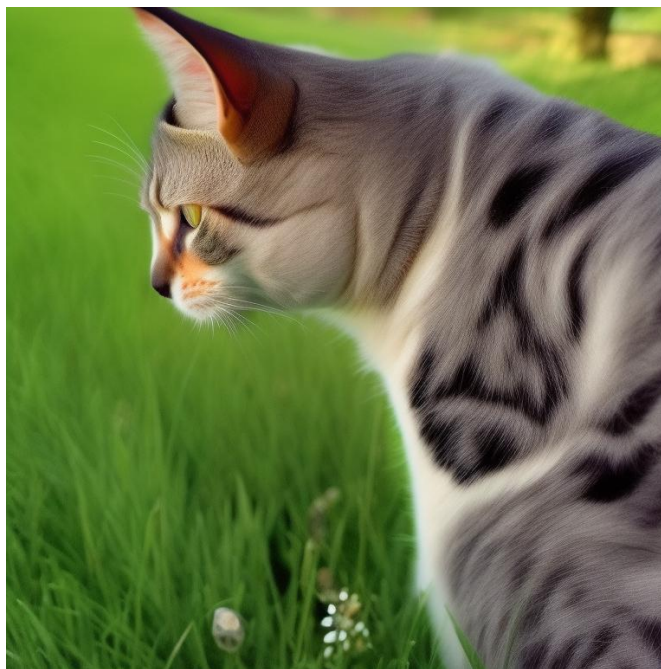


Рис. 15. Изображение из первого примера описания изображений

Пример 2. Описание изображения:

- входное изображение: фотография автомобилей (рис. 16);
- векторное представление: $[0.45, 0.67, \dots, 0.12]$;
- классификация: $[\text{car}, \text{street}, \text{city}]$;
- точность: 82%.



Рис. 16. Изображение из второго примера описания изображений

Итоговые результаты комплексной системы. Комплексная система объединяет результаты работы моделей ViT и kNN с методом коллаборативной фильтрации. Далее представлено несколько примеров работы комплексной системы:

сценарий 1. Комплексные рекомендации:

- пользователь оценивает посты о животных и активно взаимодействует с постами о питомцах;
- визуальная рекомендация (ViT и kNN): публикации с изображениями домашних животных;
- поведенческая рекомендация (Коллаборативная фильтрация): публикации о животных, природе, растениях;
- общая точность рекомендаций: 89%.

Сценарий 2. Комплексные рекомендации:

- пользователь активно комментирует публикации о спорте и оценивает фотографии тренировок;
- визуальная рекомендация (ViT и kNN): публикации с изображениями спорт залов, спортсменов, бегунов, спортивного инвентаря;
- поведенческая рекомендация (Коллаборативная фильтрация): публикации с тренировками, спортивным питанием;
- общая точность рекомендаций: 87%.

Эти примеры демонстрируют, как разработанная система может эффективно анализировать контент социальной сети и формировать точные и релевантные рекомендации для пользователей, улучшая их взаимодействие с платформой.

Таким образом, результаты моделирования подтвердили эффективность разработанного рекомендательного алгоритма, интегрирующего ViT для анализа изображений, метод kNN для формирования рекомендаций на их основе и коллаборативную фильтрацию для анализа схожести пользователей. Это позволяет создавать персонализированные рекомендации, улучшая пользовательский опыт в социальной сети.

2.5. Разработка алгоритма формирования рекомендаций и анализа контента

В алгоритме будут использованы модель и методы, описанные в п.2.2. Алгоритм сводится к следующим действиям:

- модель тегирования:

1. Описанная модель машинного обучения (ViT) инициализируется по заданной конфигурации.

2. В момент публикации пользователем медиафайла, изображение передаётся системой на вход модели.

3. В случае успешной обработки, формируется вектор тегов изображения, полученный из результатов классификации изображения трансформером.

4. Данный вектор записывается в базу данных в соответствии с переданным изображением.

5. Системой описывается пользователем вектором интересов, исходя из векторов его публикаций, векторами публикаций из подписок на пользователей. Данный вектор записывается в базу данных.

- Алгоритм формирования рекомендаций:

1. Инициализируется система формирования рекомендаций по заданной конфигурации.

2. В момент запроса пользователем рекомендаций, система анализирует имеющиеся информационные следы пользователя в социальной сети.

3. Если у пользователя достаточно информации для формирования персональных рекомендаций (если присутствуют оценки и комментарии, то по ним строятся матрицы оценок коллаборитвной фильтрации), рекомендательной системой создаются перечень персональных рекомендаций, используя описанные в предыдущем подпункте методы (гибридный метод коллаборитвной фильтрации с использованием фильтрации по контенту, используя коллаборативную фильтрацию и kNN по вектору описаний).

4. Иначе, если у пользователя отсутствуют оценки и комментарии, но существуют публикации, которые считаются данными, формирующими его интересы. С помощью векторов описаний и метода kNN формируется список рекомендаций.

5. Иначе рекомендательная система формирует список рекомендаций по статистике, используя метод, описанный в предыдущем подпункте.

6. Далее сформированный список рекомендаций используется в формировании и заполнении HTML шаблона – страницы отображения рекомендаций.

Алгоритм формирования рекомендаций. Алгоритм формирования рекомендаций сводится к двум задачам:

1. Найти, насколько другие публикации в базе данных похожи на данную публикацию.

2. Предсказать оценку клиента публикации, по полученным данным из первой задачи (информация о схожести публикаций). В реальности сложно добиться производительности с таким подходом с большим количеством публикаций, поэтому здесь применяется этап фильтрации по контенту: берутся оценки публикаций похожих по метаданным.

Проблема холодного старта: «Как рекомендовать товар, который ещё никто не видел?» и «Что рекомендовать пользователю, у которого ещё нет ни одной оценки?». Опять же, существуют разные подходы, в которых системы ждут определённых действий от пользователя, например, оценки публикации, чтобы рекомендовать аналогичные публикации по классу. Другие же просто используют метрики «популярности» публикации и тогда рекомендации не персонализированный характер, а общий. Предполагается использовать гибридный механизм: пока пользователь не оставил следов, для коллаборативной фильтрации или по которым ему на основе контента можно сформировать рекомендации, использовать популярные публикации, как только, пользователь имеет список интересов, пользователю будут предлагаться

персональные публикации. Обобщённый алгоритм формирования рекомендаций отображён на рис. 17.

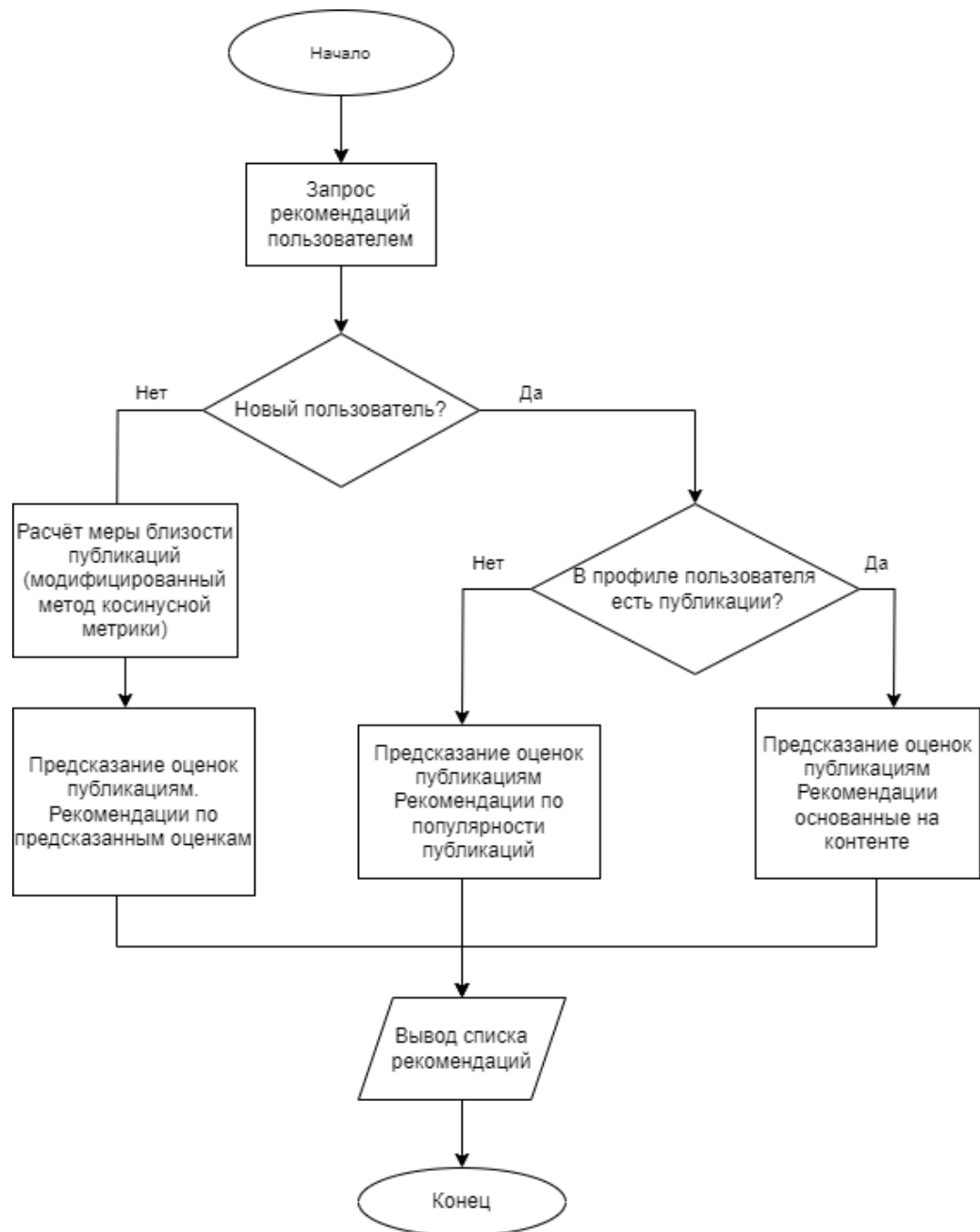


Рис. 17. Алгоритм формирования рекомендаций пользователю

Далее каждая из веток алгоритма проектируется подробнее. На рис. 18 показан алгоритм формирования рекомендаций в случае полного набора данных о пользователе.



Рис. 18. Алгоритм коллаборативной фильтрации

В общем алгоритме в случае если у пользователя недостаточно «следов» для коллаборативной фильтрации, осуществляется попытка сформировать рекомендации по его интересам – по вектору описания пользователя, сформированного из описания его публикаций. Вектор интересов пользователя будет сравниваться с вектором описания публикации кандидата на рекомендацию путём классического метода косинусной меры (формула 1). Алгоритм формирования рекомендаций по контенту описан на рис. 19.

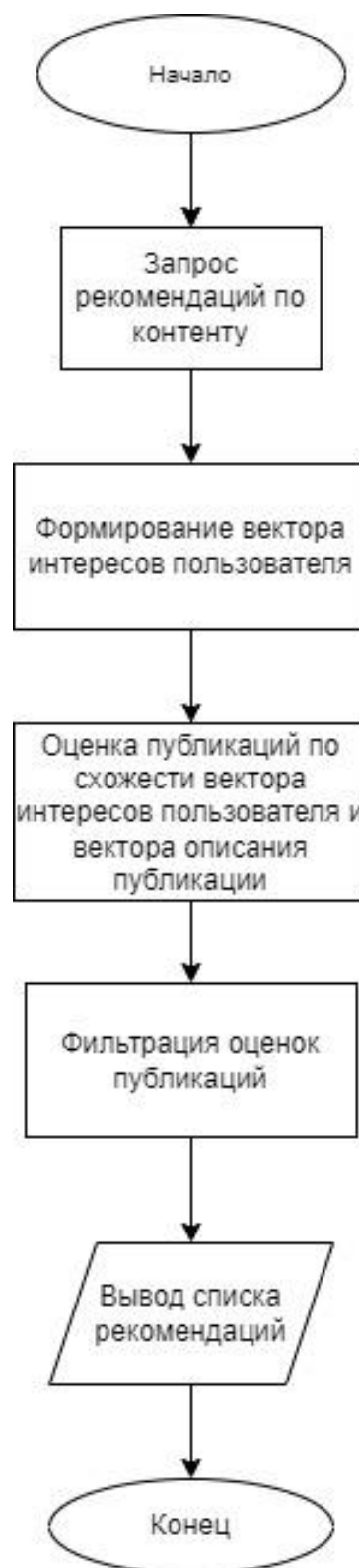


Рис. 19. Алгоритм формирования рекомендаций по контенту

Алгоритм формирования рекомендаций по популярности отображён на блок-схеме (рис. 20).



Рис. 20. Алгоритм формирования по популярности

2.6. Метрики оценки работы предложенных методов и алгоритмов

Метрики оценки рекомендательных алгоритмов можно описать следующими типами:

- для оценки точности прогнозируемой оценки публикации;
- для оценки корректности составленного списка рекомендаций и порядка его сортировки.

К сожалению, не существует единой универсальной рекомендуемой метрики и стоит подбирать их под определённые цели, но есть ряд метрик, которые

подходят для того чтобы оценить такую систему. Метрики оценки точности прогнозируемой оценки публикации описаны в табл. 5.

Таблица 5

Метрики оценки точности прогнозируемой оценки публикации

Название	Формула	Описание
Precision	$\frac{TP}{TP + FP}$	Доля рекомендаций, понравившихся пользователю
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Доля правильных ответов алгоритма
Recall	$\frac{TP}{TP + FN}$	Доля интересных пользователю публикаций, которая показана
RMSE	$RMSE = \sqrt{\frac{1}{ T } \sum_{(u,i) \in T} (p_{ui} - r_{ui})^2}$	Среднеквадратичная ошибка
F1-Measure	$\frac{2PR}{P + R}$	Среднее гармоническое метрик Precision и Recall.
ROC AUC		Метрика описывающая процент публикаций, интересующих пользователя, среди первых элементов итогового набора
Average Precision		Среднее значение Precision на всем списке рекомендаций

Так же для персонализации рекомендаций существует способ оценки – рассчитывая «разнообразие» предлагаемых рекомендаций: разность единицы и среднего значения схожести предлагаемых рекомендаций.

$$diverse = 1 - \overline{\sum_{i \in I} Sim(rec_i, rec_{i+1})} \quad (10)$$

Метрики из табл. 6 описывают корректность формирования и сортировки итогового набора.

Метрики оценки фильтрации и сортировки рекомендаций

Название	Формула	Описание
Mean Reciprocal Rank	$E\left(\frac{1}{pos}\right)$	На какой позиции списка рекомендаций пользователь находит первую полезную или понравившуюся
Spearman Correlation	$E(P - R ^2)$	Корреляция (Спирмена) реального и прогнозируемого рангов рекомендаций
nDCG (Normalized Discounted Cumulative Gain)	$\sum \frac{R(i)}{\max(1, \log(i))}$	Информативность выдачи с учетом ранжирования рекомендаций
Fraction of Concordance Pairs	$P(X_R > X_P)$	Насколько высока концентрация интересных пользователю публикаций в начале списка рекомендаций

Рекомендации выводятся списком из нескольких позиций. Порядок позиций имеет смысл – публикации располагаются в порядке убывания приоритета.

2.7. Выводы по разделу 2

В данном разделе были разработаны методы и алгоритмы для имплементации рекомендательной системы, решающей проблему «Холодного старта». Применяя модель описания изображений и разработанный гибридный алгоритм фильтрации, сформирована гибкая рекомендательная система, удовлетворяющая функциональным требованиям.

Было выявлено, что для задач классификации и описания изображений в данный момент лучшим решением является использование модели машинного обучения – Vision Transformer. Она является фаворитом по результатам работы среди аналогичных моделей, при этом имеет удовлетворительную вычислительную сложность. Для оценки схожести пользователей или публикаций наилучшим вариантом является косинусная мера из-за своей эффективности и простоты вычисления. Были проанализированы возможные и

подобраны подходящие метрики оценки рекомендательных систем исходя из контекста системы, а также нужд и требований заказчика.

Предложенное математическое и алгоритмическое обеспечение необходимо реализовать в программном обеспечении.

Следующим этапом будет описание процесса проектирования системы, описание содержательной и концептуальной модели, результатов проектирования информационного и программного обеспечения.

3. Проектирование информационного и программного обеспечения системы формирования рекомендаций и анализа контента социальной сети на основе искусственных нейронных сетей

3.1. Содержательное моделирование проектируемой системы

Процесс моделирования можно представить как переход от одного моделей одного класса к другому. Условно выделяют три больших класса моделей: когнитивные, содержательные и формальные модели. Они составляют три взаимосвязанных уровня моделирования, которые нельзя рассматривать изолированно один от другого. Взаимовлияние уровней моделирования связано со свойством потенциальности моделей.

При наблюдении за объектом в голове исследователя формируется мысленный образ объекта, его идеальная модель. Формируя такую модель, разработчик, как правило, стремится ответить на конкретные вопросы. От реального очень сложного устройства объекта отсекается все ненужное с целью получения его более компактного и лаконичного описания. Представление мысленной модели на естественном языке называется содержательной моделью [69].

По функциональному признаку и целям содержательные модели подразделяются на описательные, объяснительные и прогностические:

- описательной моделью можно назвать любое описание объекта;
 - объяснительная модель позволяет ответить на вопрос, почему что-либо происходит;
 - прогностическая модель должна описывать будущее поведение объекта.
- прогностическая модель не обязана включать в себя объяснительную.

Содержательная модель показана на рис. 21.

Показанная модель описывает исследуемый объект: рекомендательную систему. Для каждого клиента доступна функция просмотра страницы персональных рекомендаций, которые формирует рекомендательная система,

которая определяет предлагаемые рекомендации исходя из интересов пользователя, анализируя публикации.

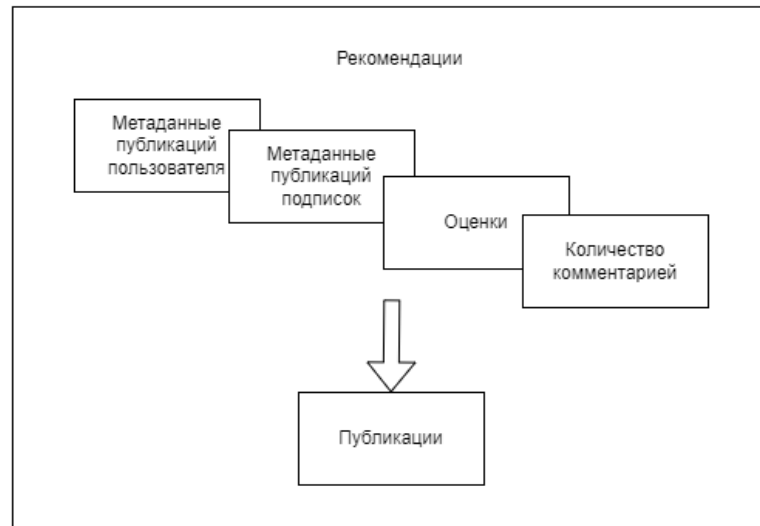


Рис. 21. Содержательная модель формирования рекомендаций

Проектирование системы состоит в преобразовании из одного вида описания системы в следующий. Следующим этапом проектирования является концептуальное моделирование.

3.2. Концептуальное моделирование проектируемой системы

В этом разделе формализуется модель автоматического рекомендательной системы в виде алгебраической системы, в рамках которой впоследствии будут сформулированы и описаны методы классификации и алгоритм рекомендации, исследуемые в работе.

Концептуальная модель - это абстрактная модель, определяющая структуру моделируемой системы, свойства ее элементов и причинно-следственные связи, присущие системе и существенные для достижения цели моделирования [70].

Концептуальной моделью принято называть содержательную модель, при формулировке которой используются понятия и представления предметных областей знания, занимающихся изучением объекта моделирования. В более

широком смысле под концептуальной моделью понимают содержательную модель, базирующуюся на определенной концепции или точке зрения. Выделяют три вида концептуальных моделей: логико-семантические, структурно-функциональные и причинно-следственные.

Логико-семантическая модель является описанием объекта в терминах и определениях соответствующих предметных областей знаний, включающим все известные логически непротиворечивые утверждения и факты. Анализ таких моделей осуществляется средствами логики с привлечением знаний, накопленных в соответствующих предметных областях.

При построении структурно-функциональной модели объект обычно рассматривается как целостная система, которую расчленяют на отдельные элементы или подсистемы. Части системы связываются структурными отношениями, описывающими подчиненность, логическую и временную последовательность решения отдельных задач. Для представления подобных моделей удобны различного рода схемы, карты и диаграммы.

Причинно-следственная модель часто используется для объяснения и прогнозирования поведения объекта. Данные модели ориентированы в основном на следующее:

- выявление главных взаимосвязей между составными элементами изучаемого объекта;
- определение того, как изменение одних факторов влияет на состояние компонентов модели;
- понимание того, как в целом будет функционировать модель и будет ли она адекватно описывать динамику интересующих исследователя параметров.

3.2.1. Модель «черного ящика»

Имея представление о входных, выходных данных и алгоритмах, требуемых для работы системы можно построить модель «Черного ящика», в котором следует собрать все данные, не углубляясь во внутренние процессы системы.

Модель чёрного ящика отображена на схеме (рис. 22).

В модели отображены входные данные – данные из профилей пользователя социальной сети, а также данные о их взаимодействии. Результатом работы модели является список рекомендаций конкретному пользователю.

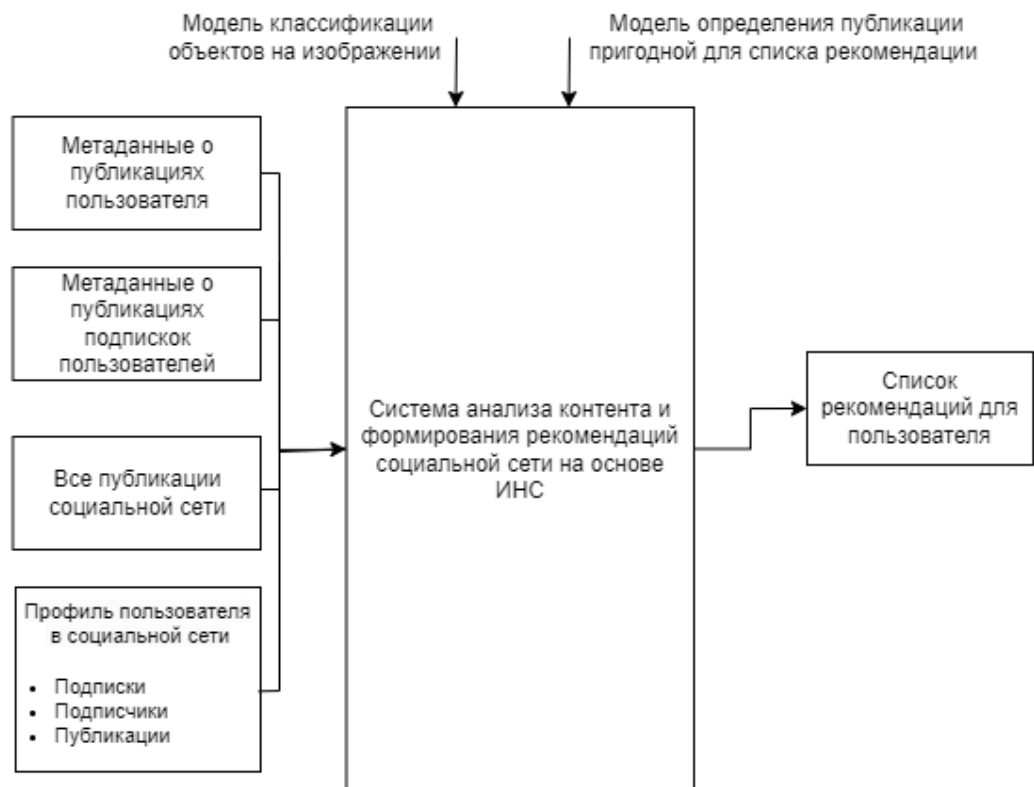


Рис. 22. Модель чёрного ящика для рекомендательной системы

Под метаданными на входе принимаются следующие данные (рис. 23).

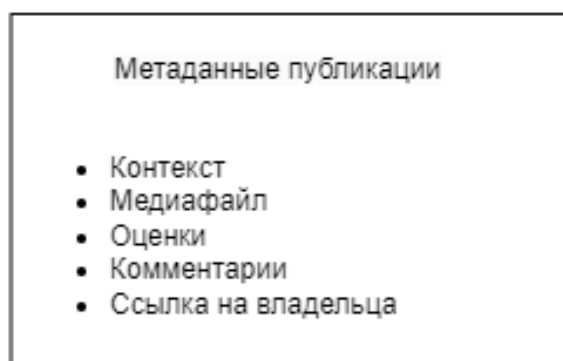


Рис. 23. Описание метаданных публикации

Основные элементы системы:

1) Входы:

- профиль пользователя социальной сети, для которого формируются рекомендации. Требуется для получения данных о пользователе, его предпочтениях, исходя из открытой информации в пользователе;
- все публикации социальной сети. Необходимы для формирования списка, содержащего публикации, подходящее для пользователя. Под публикациями понимается причастная к ним информация (оценки и комментарии);
- метаданные о публикациях (см. рис. 23) подписок пользователя (список подписок берётся из данных в профиле). Необходимо для формирования интересов пользователя;
- метаданные о публикациях (см. рис. 23) пользователя (список публикаций берётся из данных в профиле).

2) Выходы:

- список рекомендаций для пользователя.

3) Управление:

- модель классификации объектов на изображении;
- модель определения пригодности публикации к списку рекомендаций для выбранного пользователя.

3.2.2. Модель состава

Модель состава системы дает описание входящих в нее элементов и подсистем, но не рассматривает связей между ними. Модель состава представлена на рис. 24.

В табл. 7 представлено краткое описание подсистем проектируемой модели состава. Таблица включает в себя указанные выше подсистемы и их краткое описание в рамках проектируемой системы.

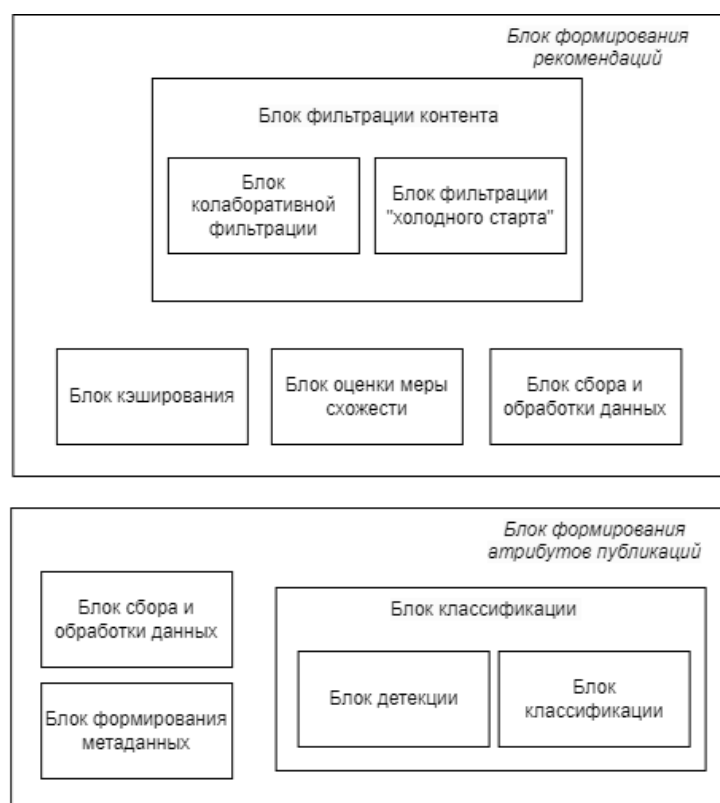


Рис. 24. Модель состава рекомендательной системы

Таблица 7

Описание подсистем

Подсистема	Описание
Блок сбора и обработки данных	Отвечает за сбор и формировании информации для остальных элементов системы
Блок классификации	Блок, отвечающий за классификацию атрибутов публикации
Блок формирования метаданных	Выполняет формирование результативных данных о классификации атрибутов публикации
Блок оценки меры схожести	Оценивает похожесть публикаций друг на друга, формируя матрицу предпочтений пользователи/публикации.
Блок фильтрации контента	Формирует список рекомендаций для конкретного пользователя
Блок кэширования	Хранит данные о расчётах рекомендаций для оптимального перерасчёта

3.2.3. Модель структуры

Структурную модель системы еще называют структурной схемой. На структурной схеме отражается состав системы и ее внутренние связи. Модель состава представлена на рис. 25.

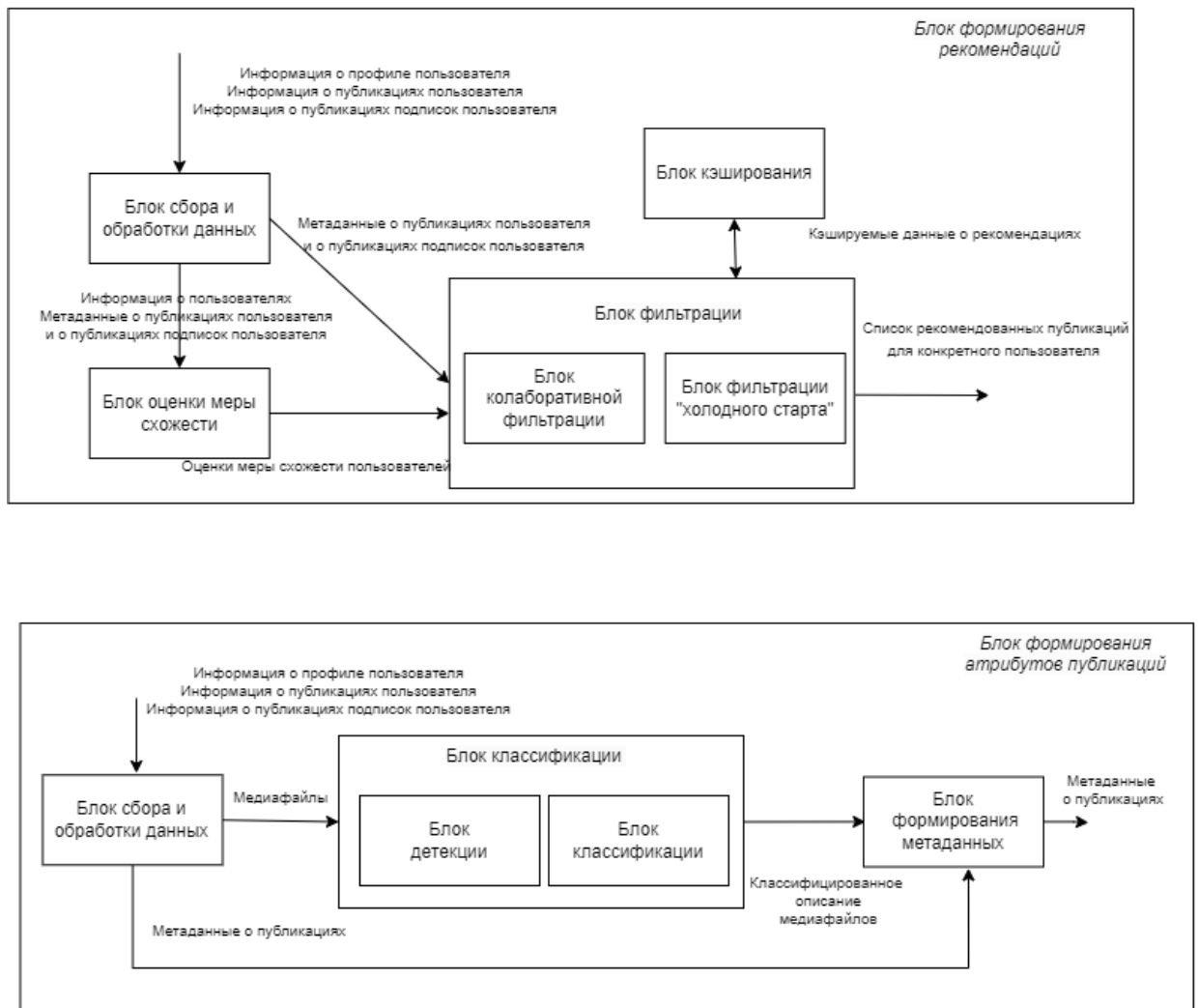


Рис. 25. Модель структуры рекомендательной системы

В табл. 8 представлено краткое описание связей (входы/выходы) элементов модели структуры.

Описание подсистем

Подсистема	Описание
Блок сбора данных	Принимает на вход информацию о пользователе, его профиле, его публикациях, его подписках, публикациях его подписок, оценки и комментарии. Передаёт сформированные списки словари данных оптимальные для дальнейшего использования.
Блок оценки меры схожести	Принимает на вход данные о оценках пользователей публикациям. Отдаёт оценки меры схожести между публикациями.
Блок классификации	Принимает на вход их медиафайлами. Отдаёт данные о признаках (атрибутах) соответствующему медиафайлу.
Блок формирования метаданных	Получает информацию о публикациях, их оценках, их атрибутах. Формирует данные в оптимальные структуры для дальнейшего использования.
Блок фильтрации	Принимает меру схожести публикаций, метаданные о публикациях, пользователя. Отдаёт список рекомендаций для пользователя, расчётные данные о рекомендациях.

После описания модели структуры системы формирования рекомендаций и анализа контента следует разработать модель «белого ящика».

3.2.4. Модель «белого ящика»

Одной из важных моделей на данном этапе является модель «белого ящика» системы. Входы и выходы «черного ящика» связываются с элементами и подсистемами модели структуры системы. В «белом ящике» указываются все элементы системы, все связи между элементами внутри системы и связи определенных элементов с окружающей средой (входы и

выходы системы). Модель «белого ящика» представлена на рис. 26.

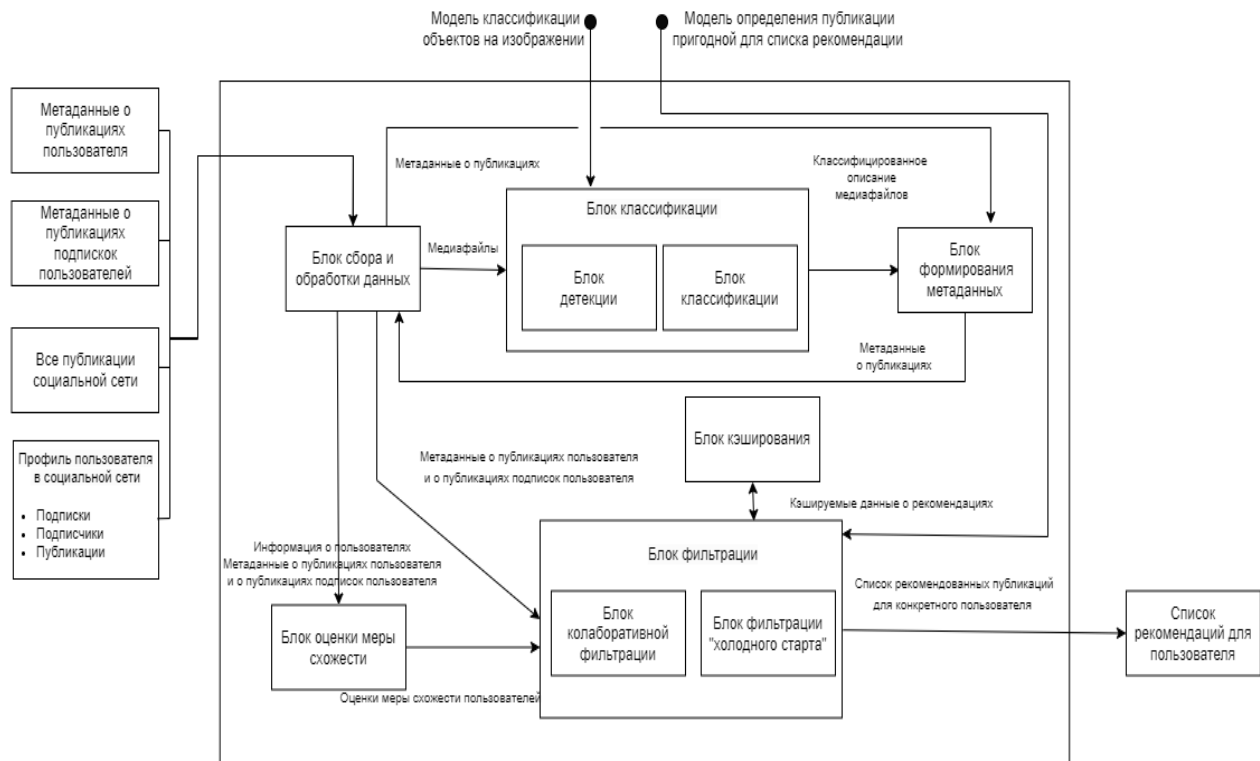


Рис. 26. Модель «белого ящика» рекомендательной системы

Модель получает на вход данные, которые обрабатываются блоком сбора данных, медиафайлы проходят этап классификации, данные переорганизуются и используются для оценки меры схожести и формирования рекомендации через блок фильтрации.

Разработав концептуальную модель системы, необходимо перейти к проектированию информационного обеспечения.

3.3. Проектирование информационного обеспечения

3.3.1. Моделирование потоков информации в проектируемой системе

Диаграммы потоков данных (Data Flow Diagrams – DFD) – методология графического структурного анализа, описывающая внешние по отношению к

системе источники и адресаты данных, логические функции, потоки данных и хранилища данных, к которым осуществляется доступ. Необходимость использования DFD-диаграмм заключается в потребности описать существующие в структуре организации потоки данных. Модель производственных процессов в формате DFD получается, когда описание создается по процессному признаку [71].

Диаграммы потоков данных обеспечивают удобный способ описания передаваемой информации между частями моделируемой системы. Они содержат два типа объектов: собирающие и хранящие информацию (хранилища данных и внешние сущности) – объекты, которые моделируют взаимодействие с теми частями системы, которые выходят за границы моделирования.

Диаграмма разрабатываемой системы содержит один процесс, название которого совпадает с проектируемой системой и внешние сущности: пользователь и социальная сеть. Диаграмма представлена на рис. 27.



Рис. 27. Диаграмма потоков данных

На вход процессу подаются данные из социальной сети – данные о пользователях: идентификатор, идентификаторы публикаций пользователя, идентификаторы пользователей подписок; данные о публикациях: идентификатор, медиафайл; статистические данные о публикации: количество просмотров, оценок и комментариев. Пользователь, используя команду

формирования персональных рекомендаций, получает рассчитанный список рекомендаций. Социальная сеть получает информацию о содержимом медиафайлов публикаций, а также получает вектор интересов пользователя.

На рис. 28. показано уточнение диаграммы потоков данных. Данные проходят процесс агрегирования и обработки. Медиафайлы публикаций классифицируются, благодаря чему формируются векторы описаний публикаций и векторы интересов пользователей. Полученные и исходные данные используются для фильтрации публикаций – создания персональных рекомендаций пользователю, предварительно рассчитав матрицу оценок схожести пользователей, исходя из имеющихся данных.

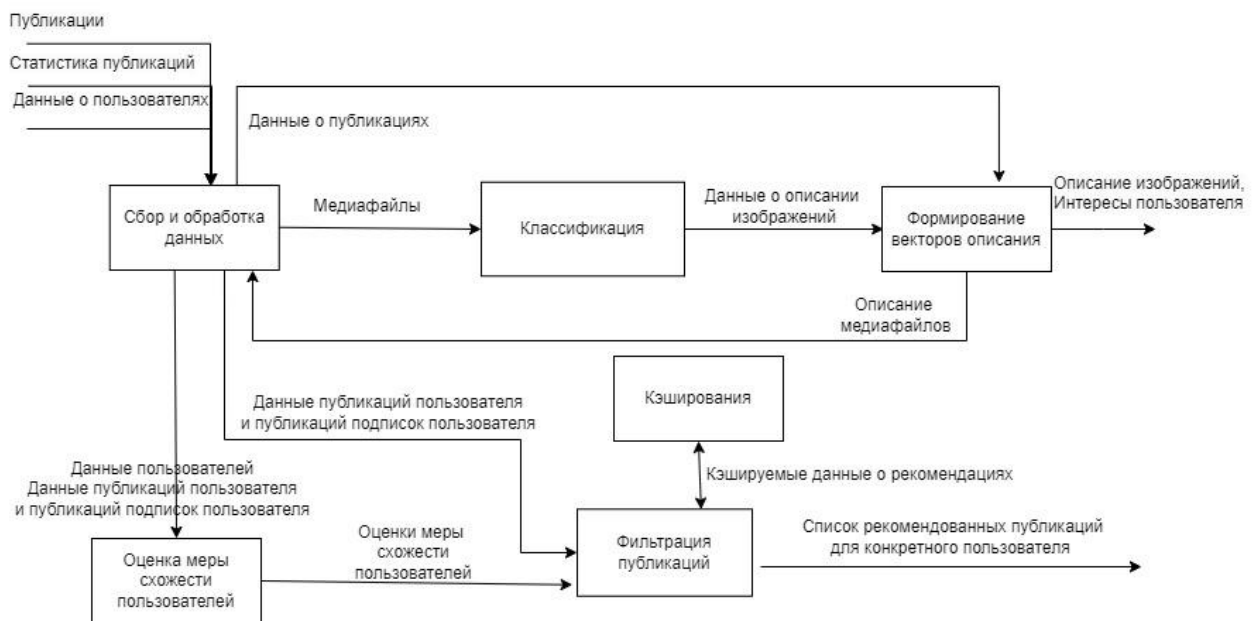


Рис. 28. Детализация диаграммы потоков данных

Выстроив процессы обработки данных в системе, начинается проектирование и разработка ПО.

3.4. Проектирование программного обеспечения

3.4.1. Выбор технологии проектирования, среды и языка программирования

3.4.1.1. Выбор модели жизненного цикла

Проектирование любой программной системы включает в себя несколько этапов или процессов, и чем лучше поставлено управление проектом, тем ярче они выражены [72]. Методология проектирования программных систем описывает процесс создания и сопровождения программного обеспечения в виде жизненного цикла (ЖЦ) разработки, представляя его как некоторую последовательность стадий и выполняемых на них процессов [84].

Жизненный цикл программного обеспечения (ЖЦ) — период времени, который начинается с момента принятия решения о необходимости создания программного продукта и заканчивается в момент его полного изъятия из эксплуатации. Этот цикл — процесс построения и развития ПО. Программное обеспечение является одним из видов обеспечения вычислительной системы, наряду с техническим (аппаратным), математическим, информационным, лингвистическим, организационным и методическим обеспечением. На сегодняшний день существует множество моделей жизненного цикла. Самые известные из них представлены далее:

- каскадная модель разработки программного обеспечения;
- v-образная модель разработки программного обеспечения;
- инкрементальная модель жизненного цикла разработки программного обеспечения;
- спиральная модель жизненного цикла разработки программного обеспечения;
- модель прототипирования жизненного цикла разработки программного обеспечения;
- модель быстрой разработки приложений RAD.

Модель, выбранная для какого-либо проекта, должна обеспечивать потребности организации, соответствовать типу выполняемых работ, а также навыкам и инструментальным средствам, которые имеются у специалистов практиков [73].

После анализа моделей жизненного цикла и исходя из требований заказчика к реализации программного обеспечения была выбрана модель прототипирования жизненного цикла. Данная модель была выбрана по следующим причинам: частое взаимодействие с заказчиком на ранних стадиях разработки и проектирования продукта, возможность скорректировать текущие и учесть новые пожелания заказчика, заказчик всегда видит прогресс в процессе разработки ПО, заказчики принимают участие в процессе планирования, проектирования и разработки на протяжении всего ЖЦ; получение прототипа ПО за минимальное время существования проекта.

В результате работы по данной модели ЖЦ команда разработки демонстрирует заказчику и потенциальным пользователям готовый прототип, происходит оценка его функционирования. После этого определяются проблемы, над устранением которых совместно работает заказчик и разработчик.

3.4.1.2. Выбор подхода к разработке

Для проектирования рекомендательной системы оптимальным подходом для разработки является объектный. Такой подход обеспечивает возможность масштабирования разработки, модификации части программы, без затрагивания других компонентов, также подход позволяет повторно использовать отдельные части кода без их дублирования, что также повышает надёжность и упрощает код. Для проектирования программного обеспечения был выбран UML.

3.4.1.3. Выбор среды и языка программирования

Выбор технологии проектирования, среды и языка программирования. Данное ПО будет реализовано в виде клиент-серверном приложении на

платформе Java 11 и Python 3.9. Для Java используется Spring Framework. Spring упрощает разработку ПО. Данный фреймворк описывает правила построения приложения – есть определенная архитектура приложения, которая наполняется функционалом. Приложения, построенные с помощью Spring, имеют слабую связность компонентов, легко поддерживаются и расширяются. Использование языка Python объясняется тем, что это язык общего назначения, с его помощью можно решать многие сложные задачи машинного обучения и быстро создавать прототипы для последующей их отладки. Клиентская часть создана с использованием HTML и CSS. Рендер клиентской части происходит на сервере с помощью Thymeleaf. Thymeleaf — современный серверный механизм Java-шаблонов для веб- и автономных сред, способный обрабатывать HTML, XML, JavaScript, CSS и даже простой текст. В качестве СУБД в данной работе выбрана PostgreSQL.

Исходя из используемых технологий в реализации проекта, используются соответствующие среды разработки: IntelliJ Idea и Visual Studio code. Для целей проектирования и построения диаграмм использованы CASE средства Rational Rose, ERwin Data Modeler 7.3, SoftwareIdeasModeler.

3.4.2. Разработка логической модели

3.4.2.1. Диаграмма вариантов использования

Проектирование логической и физической модели. Визуальное моделирование с использованием нотации UML можно представить как процесс поуровневого спуска от наиболее общей и абстрактной концептуальной модели исходной бизнес-системы к логической, а затем и к физической модели соответствующей программной системы. Для достижения этих целей вначале строится модель в форме так называемой диаграммы вариантов использования (use case diagram), которая описывает функциональное назначение системы или, другими словами, то, что бизнес-система должна делать в процессе своего функционирования. Диаграмма вариантов использования — диаграмма, на

которой изображаются отношения между актерами и вариантами использования. Диаграмма вариантов использования - это исходное концептуальное представление или концептуальная модель системы в процессе ее проектирования и разработки [74].

Диаграмма «вариант использования» применяется для описания особенностей поведения системы или предметной области без рассмотрения внутреннего устройства этой сущности. Каждый вариант использования определяет последовательность действий, которые должны быть выполнены проектируемой системой при взаимодействии ее с соответствующим действующим лицом. Действующее лицо – это роль, которую исполняется пользователем по отношению к системе. На основе функциональных требований, изложенных в техническом задании (прил. 1), была построена диаграмма вариантов использования проектируемого решения, которая представлена на рис. 29.

Действующим лицом является пользователь социальной сети. Пользователь может просматривать все публикации, опционально реагировать на них (оценивать и комментировать), создавать свои публикации (выкладывать изображения) и просматривать страницу персональных рекомендаций. На каждое действие пользователя реагирует рекомендательная система. В случае просмотра, оценки или публикации своего изображения рекомендательный алгоритм формирует и корректирует интересы пользователя. В случае просмотра рекомендаций пользователем, система формирует этот список исходя из имеющихся данных.

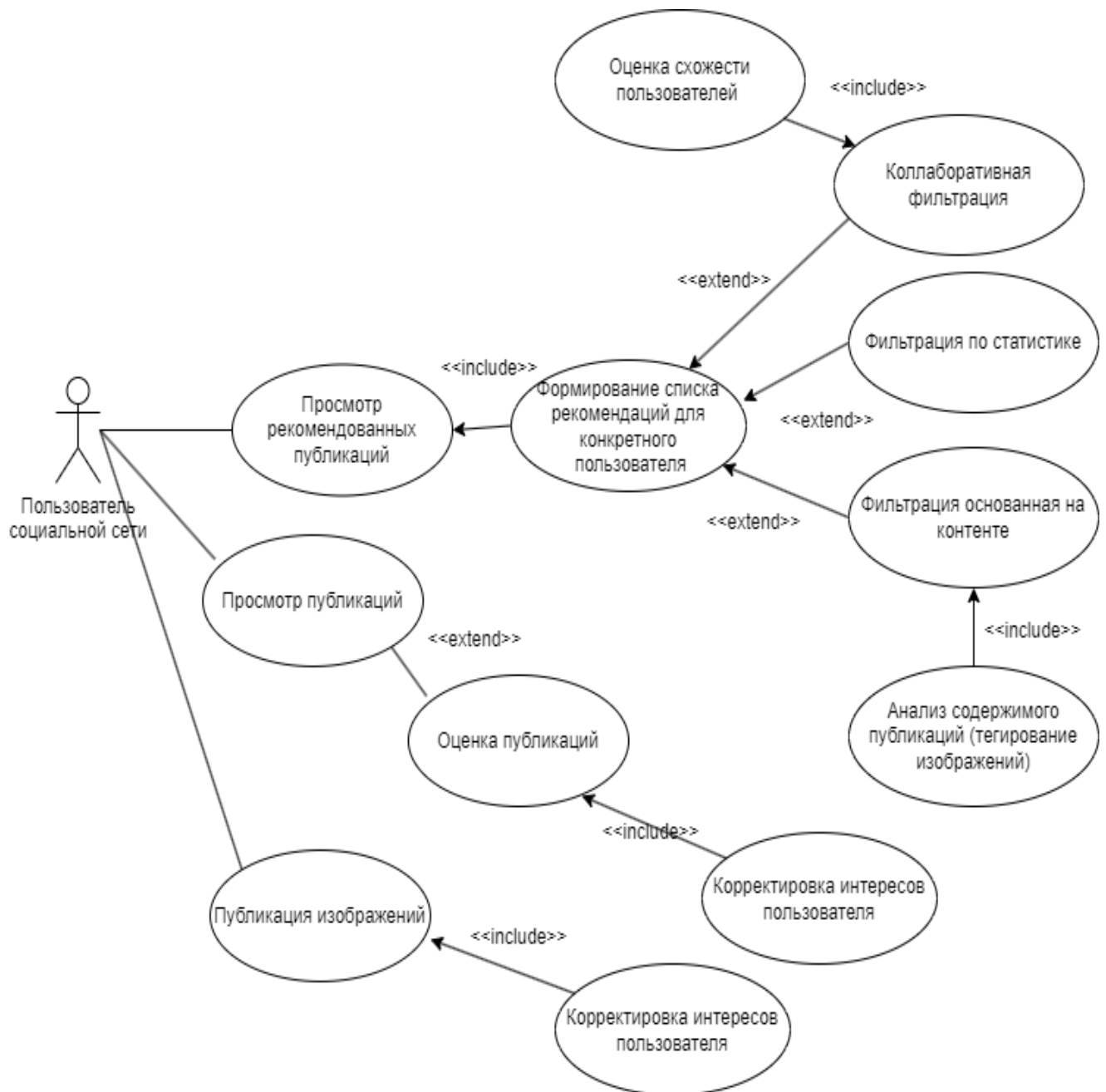


Рис. 29. Диаграмма вариантов использования

Далее будет рассмотрено описание каждого варианта использования.

3.4.2.1.1. Вариант использования “Просмотр рекомендованных публикаций”

Таблица 9

Описание блока “Просмотр рекомендованных публикаций”

Название	Просмотр рекомендованных публикаций
Актеры	Пользователь социальной сети
Цель	Просмотр персональных рекомендаций, состоящих из публикаций других пользователей, подобранных путём анализа интересов текущего пользователя, схожести его с другими пользователями и исходя из статистических метрик.

Таблица 10

Типичный ход событий

Действие исполнителя	Отклик системы
1. Пользователь выбирает страницу с рекомендациями в социальной сети	2. Система, исходя из имеющихся данных рассчитывает рекомендация для пользователя. Система отображает выбранную страницу с рекомендациями.
3. Пользователь взаимодействует с предложенными публикациями (просмотр, оценка, комментирование)	4. Система корректирует интересы пользователя исходя из его реакции

Альтернативные варианты:

1. Пользователь останавливает работу. Система прекращает формировать или корректировать рекомендации для пользователя.

Исключения:

1. Потеря соединения с БД. Рекомендательный алгоритм генерирует пустой список рекомендаций.

2. Недостаточное кол-во подписок и оценок публикаций у пользователя. Система генерирует список рекомендаций, исходя из содержимого публикаций данного пользователя (рекомендации, основанные на контенте).

3.Отсутствие следов пользователя в социальной сети (подписки, оценки, комментарии, публикации). Система генерирует список рекомендаций, состоящий из популярных публикаций (рекомендации, основанные на статистике).

3.4.2.1.2. Вариант использования “Просмотр публикаций”

Таблица 11

Описание блока “Просмотр публикаций”

Название	Просмотр публикаций
Актеры	Пользователь социальной сети
Цель	Просмотр страницы публикаций, на которой отображаются все публикации: публикации подписок и рекламные публикации

Таблица 12

Типичный ход событий

Действие исполнителя	Отклик системы
1. Пользователь выбирает страницу просмотра публикаций в социальной сети	2. Система формирует список публикаций по подпискам пользователя
3. Пользователь может взаимодействовать с отображаемыми публикациями (просмотр, оценка, комментирование)	4. Система формирует или корректирует интересы пользователя исходя из его реакции

Альтернативные варианты:

1. Пользователь останавливает работу. Система прекращает формировать страницу с публикациями для пользователя.

Исключения:

1. Потеря соединения с БД. Система генерирует пустой список публикаций и не формирует или корректирует интересы пользователя.

3.4.2.1.3. Вариант использования “ Публикация изображений”

Таблица 13

Описание блока “Публикация изображений”

Название	Просмотр публикаций
Актеры	Пользователь социальной сети
Цель	Создание новой публикации

Таблица 14

Типичный ход событий

Действие исполнителя	Отклик системы
1. Пользователь выбирает страницу создания новой публикации	2. Система отображает страницу создания новой публикации, где пользователю предлагается выбрать медиафайл и написать текст
3. Пользователь выбирает изображение и пишет текст	4. Система создаёт новую публикацию в социальной сети

Альтернативные варианты:

1. Пользователь останавливает работу. Система прекращает формировать новую публикацию, отменяет её создание.

Исключения:

1. Потеря соединения с БД. Система не генерирует новую публикацию, отображает ошибку пользователю.

3.4.2.2. Контекстная диаграмма классов

Диаграммы классов - центральное звено объектно-ориентированных методов разработки программного обеспечения, поэтому все существующие методы используют диаграммы классов в одной из известных нотаций. Однако в основном диаграммы классов в этих методах применяют на этапе проектирования, для того чтобы показать особенности построения конкретных классов [75].

Концептуальные модели используют понятия предметной области, элементами этих понятий и отношениями между ними. Предметной области могут соответствовать как материальные предметы, так и абстракции, которые применяют специалисты предметной области. Основным понятиям в модели ставятся в соответствие классы. Класс понимается как совокупность общих признаков конкретной группы элементов предметной области. В соответствии с этим определением на диаграмме классов каждому классу соответствует группа объектов, характеристику которых и описывает класс.

Контекстная диаграмма классов представлена на рис. 30.

В данной диаграмме существуют следующие понятия предметной области с их связями: “Пользователь” связан с сущностями “Публикация”, “Комментарий”, “Оценка”: пользователь социальной сети создаёт публикации, комментирует и оценивает публикации других пользователей. Каждая публикация содержит в себе изображение и наборы комментариев и оценок. База данных хранит данные о пользователях, публикациях и изображениях. Рекомендательный движок (“Алгоритм”) использует данные о пользователе и публикациях для создания результата работы алгоритма - “Персональных рекомендаций”, которые просматривает пользователь.

Реализация алгоритма представлена в виде фильтрации по контенту и коллаборативной фильтрацией. Каждый из алгоритмов описывает математические методы и более примитивные алгоритмы для вычисления персональных рекомендаций.

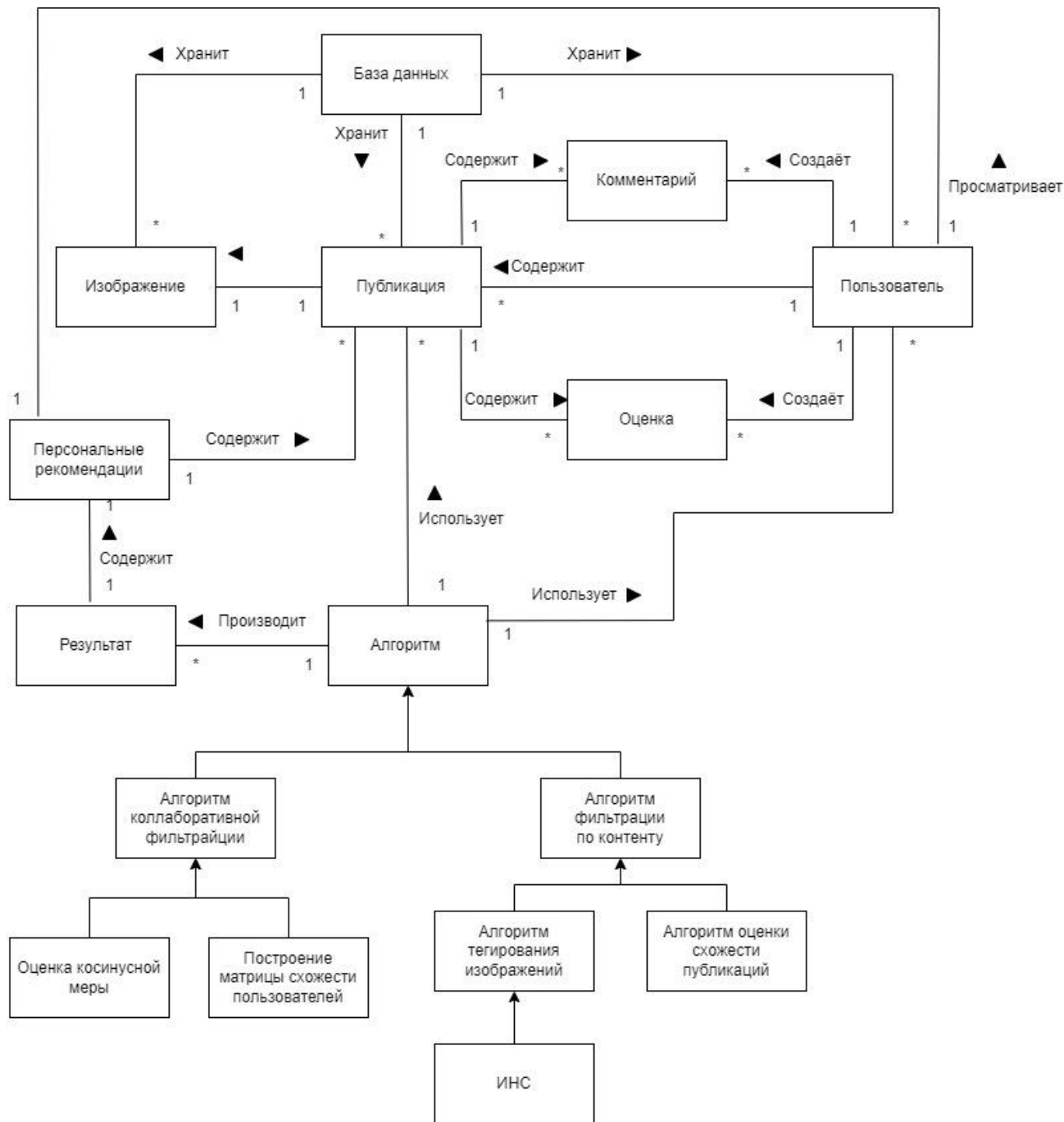


Рис. 30. Контекстная диаграмма классов

Предметная область и предлагаемая разрабатываемая реализация алгоритма формирования рекомендаций успешно декомпозируется на объекты-сущности, что является подтверждает правильность выбора объектно-ориентированного подхода в проектировании и разработке.

3.4.2.3. Диаграмма пакетов

Хорошей практикой при разработке является использование многослойной архитектуры (рис. 31). Многослойная архитектура - Архитектура, где элементы отображения, работы с данными являются отдельными процессами. Модель многослойной архитектуры помогает создать ПО, которое легко изменять, расширять и поддерживать. В случае необходимости изменений, их необходимо делать лишь в отдельных слоях-пакетах, а не во всём приложении. Это позволяет оптимизировать процесс разработки и рефакторинга.

Каждый слой паттерна слоистой архитектуры имеет определённую роль и ответственность в приложении. Например, слой представления отвечает за работу с пользовательским интерфейсом и логику взаимодействия с браузером, в то время как бизнес-слой будет отвечать за выполнение конкретных бизнес-правил, связанных с запросом. Каждый слой в архитектуре формирует абстракцию вокруг работы, которую необходимо выполнить для удовлетворения конкретного бизнес-запроса [76].



Рис. 31. Устройство многослойной архитектуры приложения

Более типичным и более используемым вариантом является трехслойная архитектура:

- слой представления, в котором содержатся визуальные элементы и логика представления. Этот слой может отводиться формам ввода и графическому пользовательскому интерфейсу;
- слой логики представления взаимодействует со слоем логики приложения (также слой бизнес-логики, средний слой). Задача бизнес-логики или слоя логики - управлять функциональностью, обрабатывая для этого полученные с нижнего слоя данные в соответствии с запросами пользователя, пришедшими с верхнего слоя;
- слой данных - это третий слой. Он включает в себя сущности предметной области, базу данных и необходимое для ее управления программное обеспечение.

Помимо перечисленных трёх слоёв - используется общий пакет, в который вынесены элементы, которые используются в каждом из пакетов многослойной архитектуры.

Диаграммы пакетов представляют организации элементов модели по пакетам, зависимости между пакетами [75].

На диаграмме пакетов отображаются ряд элементов и связей между ними: пакет, элемент пакета, зависимость между элементами, импорт элемента, импорт пакета.

Проанализировав контекстную диаграмму классов предметной области и варианты использования, итогом проектирования стали пакеты, изображённые на рис. 32.

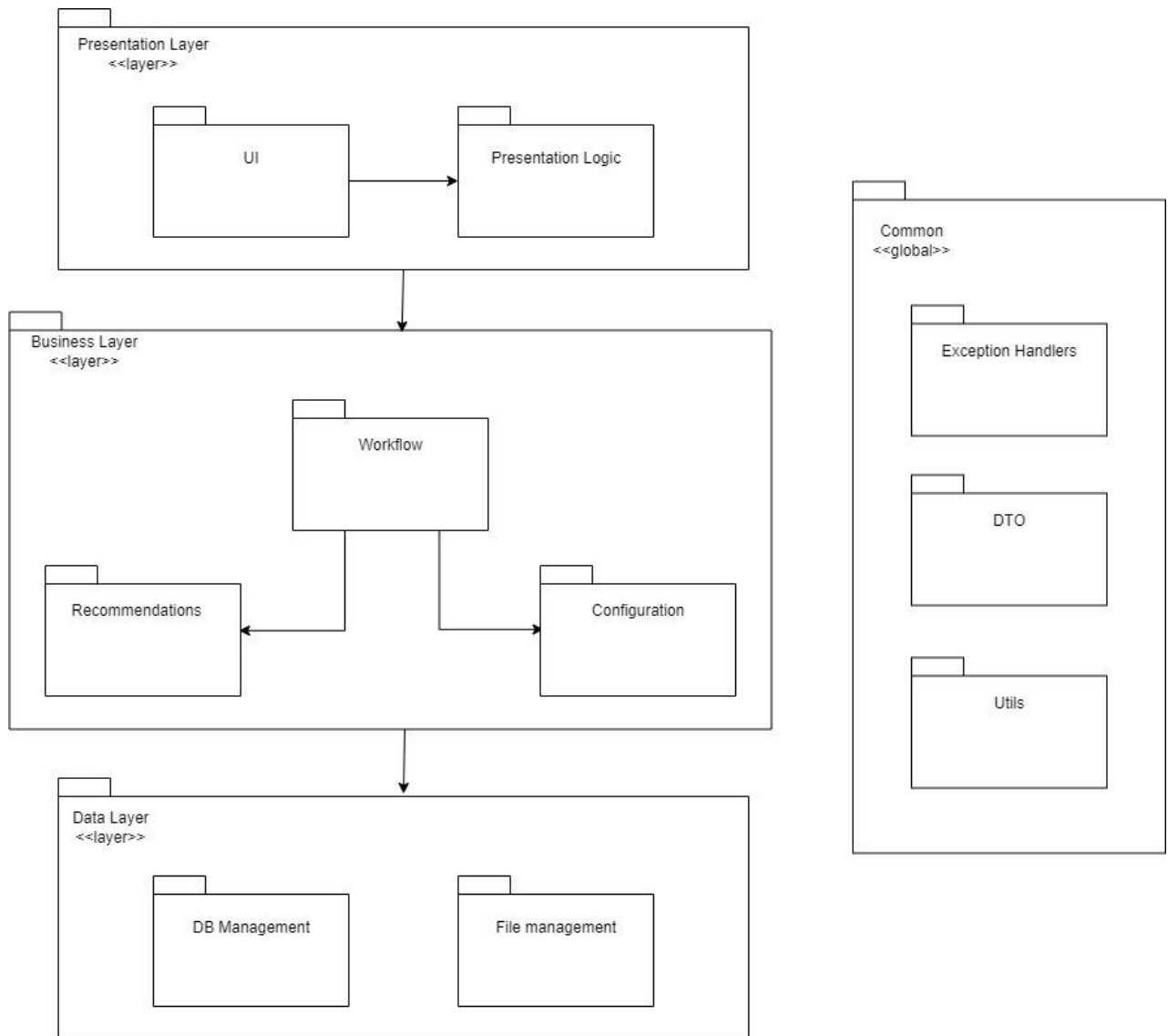


Рис. 32. Диаграмма пакетов

Описание пакетов:

1. Presentation Layer – пакет-слой, в котором находятся подпакеты, содержащие логику отображения.
2. UI – пакет, содержащий классы описания пользовательского интерфейса.
3. Presentation Logic – пакет, содержащий классы описания логики отображения.
4. Business Layer – пакет-слой, в котором находятся подпакеты, содержащие бизнес-логику приложения.
5. Workflow – пакет, содержащий классы описания сценариев работы приложения, основное описание бизнес-логики. Работа с сущностями

предметной области (социальной сети), взаимодействие с подпакетом, описывающим логику формирования рекомендаций.

6. Recommendations – пакет, содержащий классы для работы с искусственным интеллектом и описывающий логику формирования рекомендаций – содержит соответствующие методы и алгоритмы (коллаборативная фильтрация, фильтрация по контенту и т.д.).

7. Configuration – пакет, содержащий классы, описывающие конфигурацию приложения.

8. Data Layer – пакет-слой, в котором находятся подпакеты, содержащие сущности и логику работы с хранилищами данных.

9. DB Management – пакет, содержащий классы, описывающие классы-сущности и логику работы с базой данных.

10. File Management – пакет, содержащий классы, описывающие логику работы с файлами.

11. Common – глобальный пакет, в котором находятся подпакеты, классы которых используются в пакетах-слоях.

12. DTO – пакет, в котором находятся транспортные классы с данными.

13. Utils – пакет, в котором находятся классы со статическими методами.

Также при проектировании использован архитектурный паттерн - MVC. Шаблон Model-View-Controller помогает четко отделять бизнес-логику приложения и логику презентации от пользовательского интерфейса. Поддержание четкого разделения между логикой приложения и пользовательским интерфейсом помогает решить многочисленные проблемы разработки и упрощает тестирование, обслуживание и развитие приложения. Данный паттерн соответствует подходу многослойной архитектуры: View и Controller описываются в слое Presentation Layer. Model в слоях Business Layer и Data Layer.

Далее будут подробнее представлены пакеты (Recommendations и Workflow) содержащие функциональную логику и классы, относящиеся к расчётам рекомендаций, описанию сущностей социальной сети.

3.4.2.4. Исходная диаграмма классов пакета «Recommendations»

После определения основных пакетов разрабатываемого программного обеспечения переходят к проектированию классов, входящих в каждый пакет. Классы-кандидаты, которые предположительно должны войти в конкретный пакет; показывают на диаграмме классов этапа проектирования и уточняют отношения между объектами указанных классов.

Основой для проектирования классов является уточнение взаимодействия объектов этих классов в процессе реализации вариантов использования.

На рис. 33 изображена исходная диаграмма классов пакета «Recommendations».

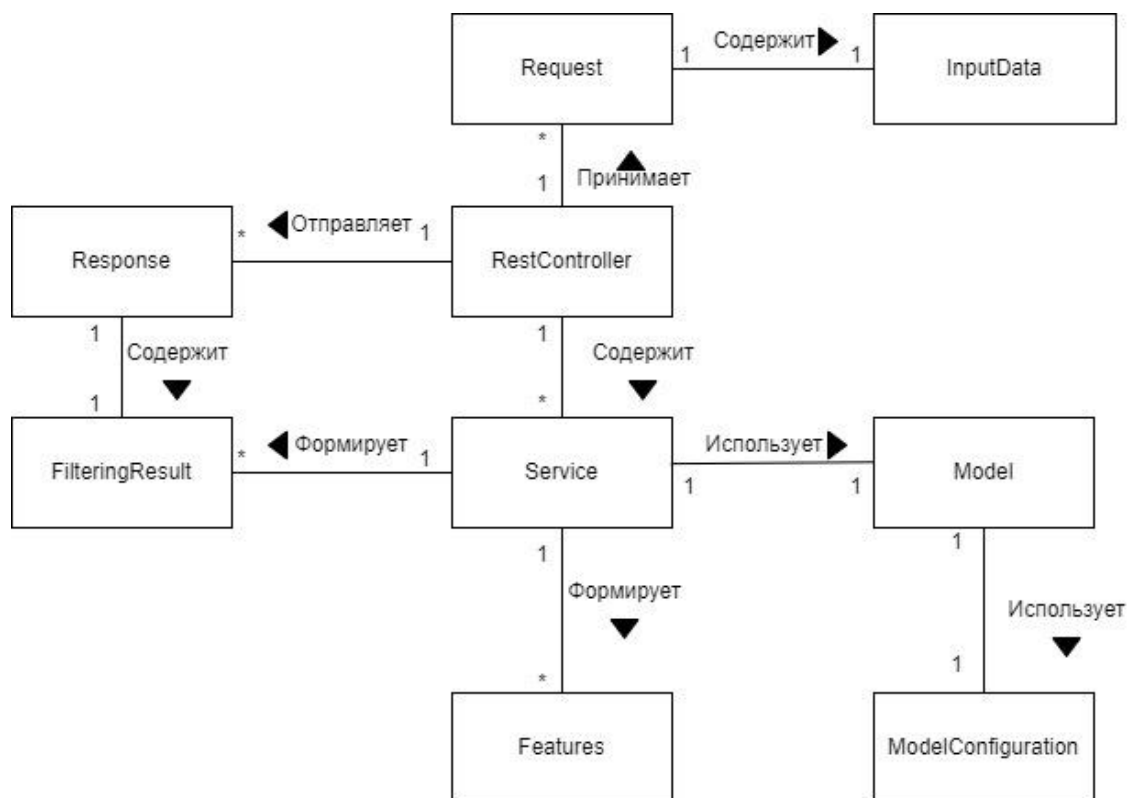


Рис. 33. Исходная диаграмма классов пакета «Recommendations»

На диаграмме показаны классы – кандидаты на реализации, а также наличие связи между ними и тип отношения, при его наличии. В табл. 15 представлено описание классов пакета «Recommendations».

Описание классов пакета «Recommendations»

Класс	Описание
Request	Объект данного класса описывает запрос на расчёт фильтрации рекомендаций
Response	Объект данного класса описывает ответ расчёта фильтрации рекомендаций
RestController	Объект данного класса принимает и отвечает на запросы расчёта формирования рекомендаций
InputData	Объект данного класса описывает входные данные для расчёта рекомендаций пользователю
Service	Объект данного класса описывает сервисную логику обработки данных: сервисы осуществляющие фильтрацию, тегирующие изображения, определяющие интересы пользователя.
Model	Объект данного класса описывает модель машинного обучения
ModelConfiguration	Объект данного класса описывает конфигурацию модели машинного обучения
FilteringResult	Объект данного класса описывает входные результат формирования рекомендаций пользователю
Features	Объект данного класса описывает тэги (содержимое изображения, интересы пользователя)

После описание исходных классов рассматриваемого пакета, переходим к их детализации.

3.4.2.5. Детальная диаграмма классов пакета «Recommendations»

Следующим этапом является уточнение связей между классами, а также выделения абстракций и интерфейсов. Результатом данного этапа проектирования является уточнённая диаграмма классов пакета Recommendations (рис. 34).



Были конкретизированы классы, описывающие API (запросы и ответы), а также классы, отвечающие за сервисную логику.

В табл. 16 представлено описание новых классов, появившихся на данном этапе проектирования.

Таблица 16

Описание классов пакета «Recommendations»

Класс	Описание
GetCollaborativeRequest	Объект данного класса описывает запрос на расчёт коллаборативной фильтрации рекомендаций
GetContentRequest	Объект данного класса описывает запрос на расчёт фильтрации рекомендаций по контенту
GetCollaborativeResponse	Объект данного класса описывает ответ расчёта коллаборативной фильтрации рекомендаций
GetContentResponse	Объект данного класса описывает ответ на расчёт фильтрации рекомендаций по контенту
ContentData	Объект данного класса описывает входные данные о публикациях для расчёта рекомендаций пользователю
UserData	Объект данного класса описывает входные данные о пользователях для расчёта рекомендаций пользователю
TaggingService	Объект данного класса описывает сервисную логику тегирования изображений
DataService	Объект данного класса описывает сервисную логику обработки входных данных
ContentFilteringService	Объект данного класса осуществляет сервисную логику формирования рекомендаций по контенту
CollaborativeFilteringService	Объект данного класса осуществляет сервисную логику формирования рекомендаций методом коллаборативной фильтрации
FeatureService	Объект данного класса описывает сервисную логику выделения интересов пользователя или публикации
UserFeatures	Объект данного класса описывает интересы пользователя
ContentFeatures	Объект данного класса описывает тэги изображения (содержимое изображения)

Описав пакет с логикой формирования рекомендаций, следует рассмотреть пакет, в котором содержится логика выполнения обработки запросов и основных сценариев.

3.4.2.6. Исходная диаграмма классов пакета «Workflow»

На рис. 35 показан итог проектирования пакета Workflow в виде исходной диаграммы классов.

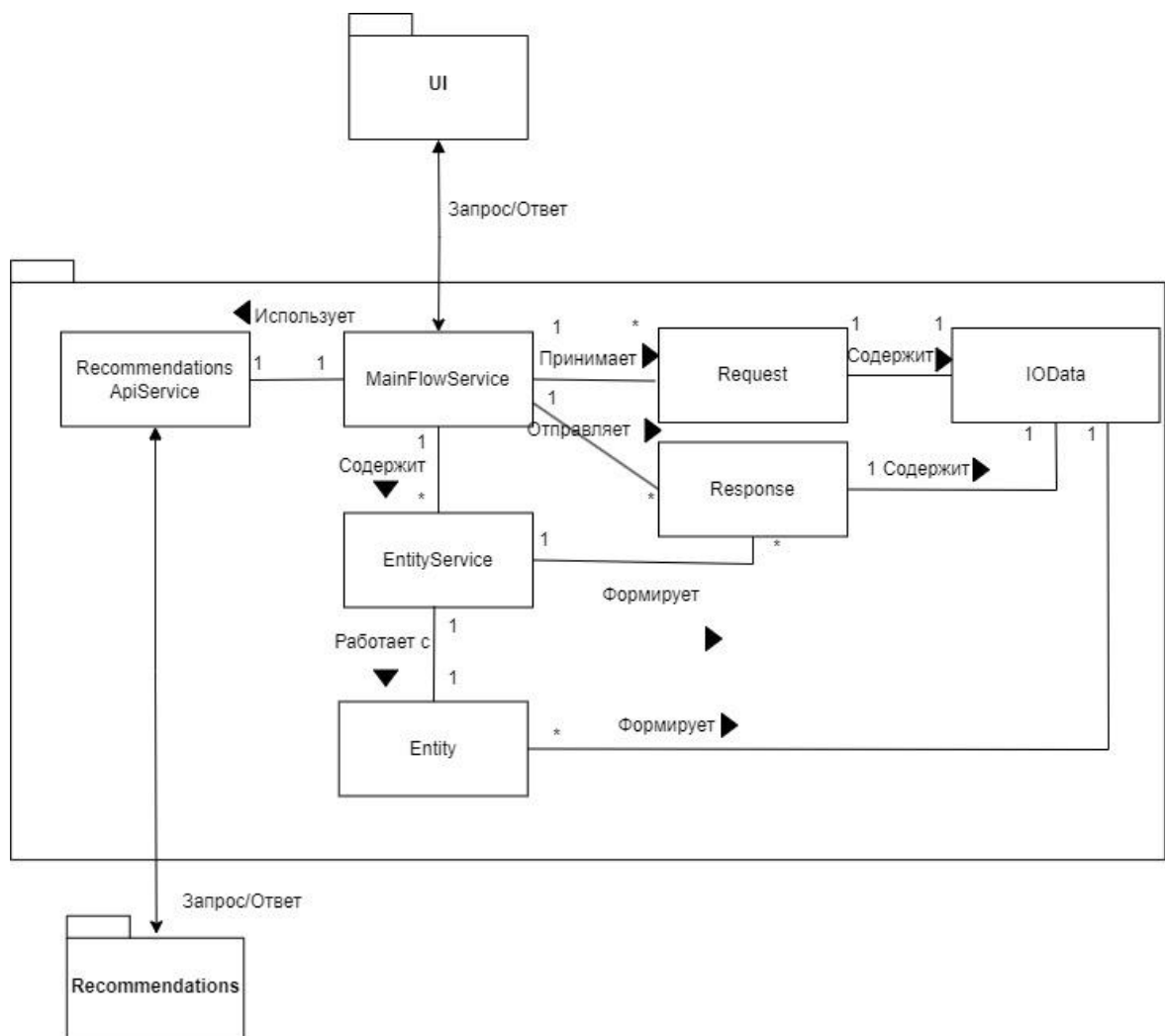


Рис. 35. Исходная диаграмма классов пакета «Workflow»

В данном пакете описываются основные классы для обработки сообщений с клиентской части, а также элементы, реализующие API с другими сервисами,

например, с модулем Recommendations. Также в этом пакете описываются классы всех запросов и ответов в клиент-серверной архитектуре.

В табл. 17 представлено описание классов пакета «Workflow».

Таблица 17

Описание классов пакета «Workflow»

Класс	Описание
Request	Объект данного класса описывает запрос на расчёт фильтрации рекомендаций. Используется в классе MainFlowService при взаимодействии с другим пакетом (UI)
Response	Объект данного класса описывает ответ расчёта фильтрации рекомендаций. Используется в классе MainFlowService при взаимодействии с другим пакетом (UI)
IOData	Объект данного класса описывает данные, содержащиеся в запросе/ответе
EntityService	Объект данного класса описывает сервисную логику обработки данных: сервисы осуществляющие взаимодействие с сущностями системы (социальной сети): пользователи, публикации, комментарии
Entity	Объект данного класса описывает сущность социальной сети
RecommendationsApiService	Объект данного класса описывает сервис, в котором содержится логика взаимодействия с пакетом/модулем Recommendations, отправляет запросы и получает результат по персонализированным рекомендациям для пользователей.
MainFlowService	Основной класс сервис, в котором описана логика обработки всех входных сообщений-запросов из пакета UI. В ходе разбора сообщений взаимодействует с остальными сервисами, формирует ответ с данными.

3.4.2.7. Детальная диаграмма классов пакета «Workflow»

На рис. 36 показан результат уточнения диаграммы классов, представленной в прошлом подпункте.

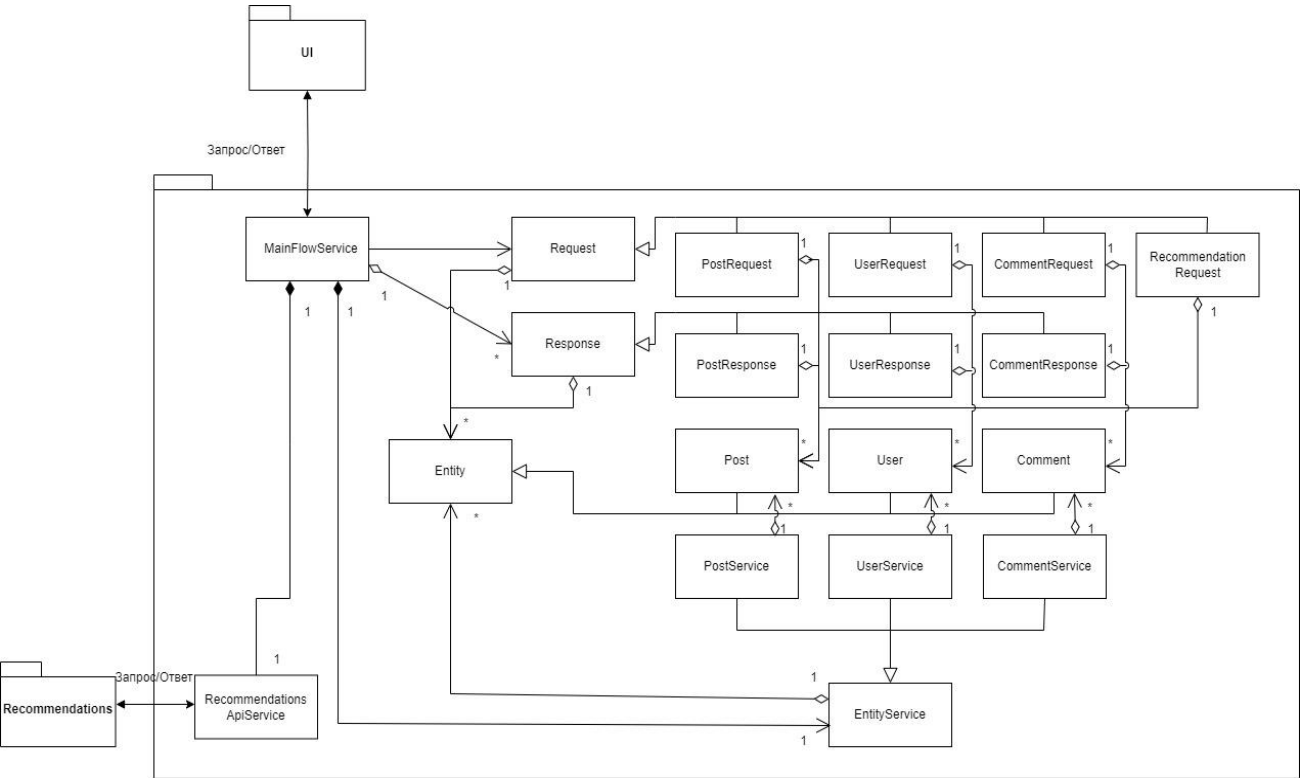


Рис. 36. Детальная диаграмма классов пакета «Workflow»

В табл. 18 отображены классы, появившееся на этапе уточнения исходной диаграммы.

Таблица 18

Описание классов пакета «Workflow»

Класс	Описание
1	2
User	Класс-сущность, описывающий пользователя социальной сети
Post	Класс-сущность, описывающий публикацию социальной сети
Comment	Класс-сущность, описывающий комментарий социальной сети

1	2
UserRequest	Класс, описывающий клиентский запрос пользователей/пользователя
PostRequest	Класс, описывающий клиентский запрос публикации/публикац
CommentRequest	Класс, описывающий клиентский запрос комментариев/комментария
RecommendationRequest	Класс, описывающий клиентский запрос получения рекомендаций для конкретного пользователя
PostResponse	Класс, описывающий ответ с данными о публикациях/публикации
UserResponse	Класс, описывающий ответ с данными о пользователях/пользователе
CommentResponse	Класс, описывающий ответ с данными о комментариях/комментарии
UserService	Класс-сервис, содержащий логику взаимодействия с сущностью пользователь
PostService	Класс-сервис, содержащий логику взаимодействия с сущностью публикация
CommentService	Класс-сервис, содержащий логику взаимодействия с сущностью комментарий

Следующим этапом проектирования ПО является разработка физической модели.

3.4.3. Разработка физической модели

3.4.3.1. Диаграмма компонентов

Диаграммы компонентов применяют при проектировании физической структуры разрабатываемого программного обеспечения. Эти диаграммы показывают, как выглядит программное обеспечение на физическом уровне, т. е. из каких частей оно состоит и как эти части связаны между собой. Диаграммы

компонентов оперируют понятиями компонент и зависимость. Под компонентами при этом понимают физические заменяемые части программного обеспечения, которые соответствуют некоторому набору интерфейсов и обеспечивают их реализацию.

На рис. 37 изображена диаграмма компонентов системы рекомендаций.

Система состоит из двух исполняемых программных компонентов: основной – рекомендательная система, которая использует второй исполняемый компонент - систему тегирования изображений. Данные системы связаны с базой данной. Каждая из систем имеет зависимость от платформы, на которой она разработана (RecommendationSystem от инструмента разработки Java Development Kit и фреймворка Spring, ImageTaggingSystem от среды Python). Система тегирования использует компоненты – библиотеки для работы с моделями машинного обучения – библиотека для работы с искусственными нейронными сетями и библиотека с датасетами для обучения моделей.

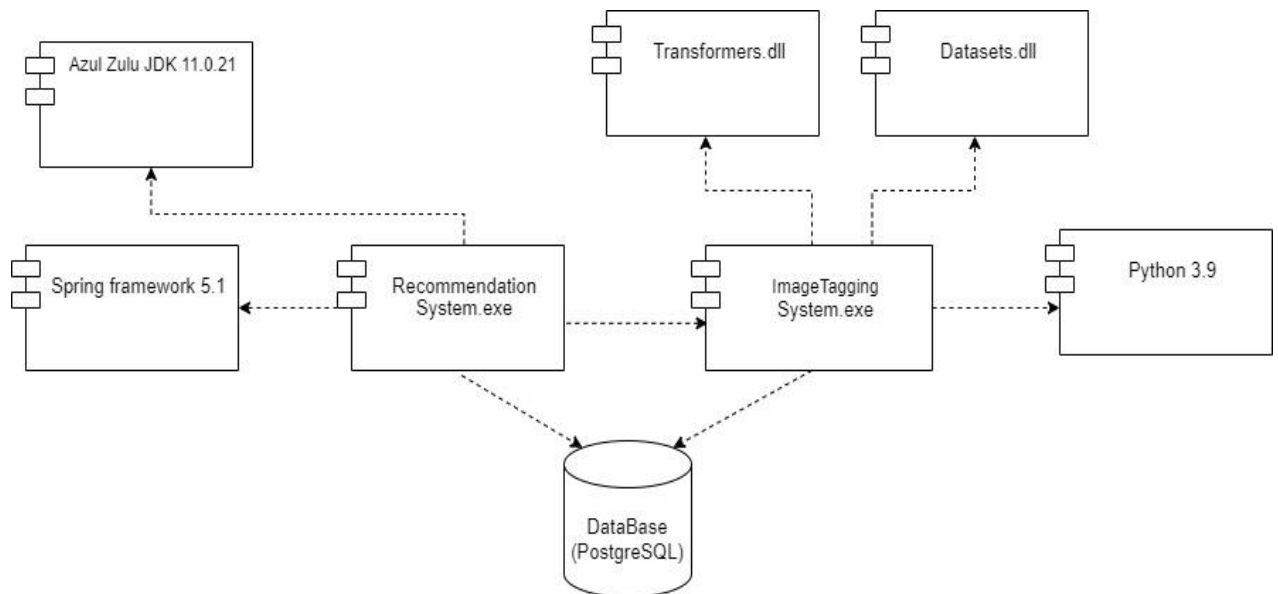


Рис. 37. Диаграмма компонентов рекомендательной системы

Текст модулей описан в П2. Спецификации на компоненты описаны в П3.

3.4.3.2. Диаграмма размещения

При физическом проектировании распределенных программных систем необходимо определить наиболее оптимальный вариант размещения программных компонентов на реальном оборудовании в локальной или глобальной сетях. Для этого используют специальную модель UML - диаграмму размещения. Диаграмма размещения отражает физические взаимосвязи между программными и аппаратными компонентами системы.

Спроектирована диаграмм размещения системы (рис. 38).

ПК клиента с помощью локальной сети запрашивает рекомендации у сервера, в котором расположена база данных и рекомендательная система. Рекомендательная система использует данные о содержимом изображений, которые создаются в системе тегирования изображений. Система тегирования расположена на отдельном сервере, подключённом к сети, для вычислений искусственного интеллекта.

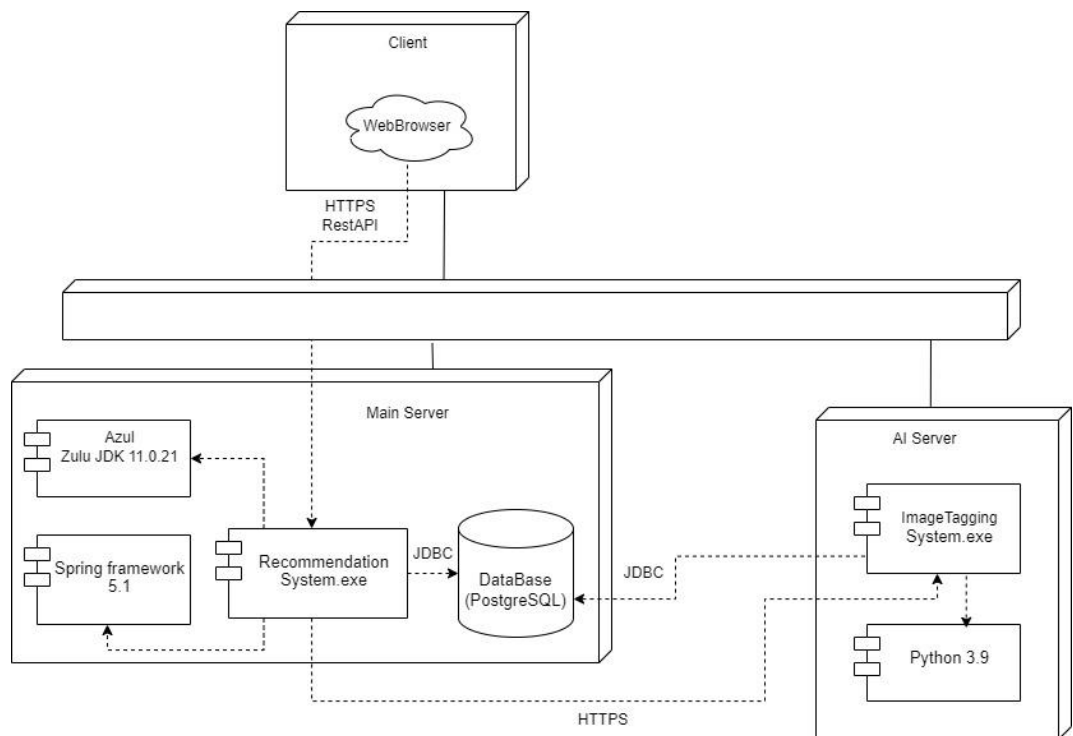


Рис. 38. Диаграмма размещения рекомендательной системы в локальной сети

Полученное решение требует проверки и тестирования, а также проверки выдвинутых критериев оценки и требований.

3.4.4. Разработка интерфейсов

Действующий интерфейс социальной сети и конкретно отображение страницы рекомендаций является отдельным модулем и не рассматривается в данной работе, но для демонстрации был разработан макет панели администрации, в которой отображен функционал просмотра информации о пользователях, публикациях и комментариях, а также просмотр предлагаемых рекомендаций для конкретного пользователя.

3.4.4.1. Построение графа диалога

Формирование диалога можно рассматривать, как последовательные переходы между состояниями работы системы. Диалог может находиться в особом состоянии ожидания ввода от пользователя, и будет переходить в одно из возможных состояний в зависимости от характера принятой информации. В соответствии с этим диалог можно представить в виде сети переходов или диаграммы состояний [80].

При взаимодействии пользователя с интерфейсом, элементы интерфейсы ожидают действия от пользователя. На стартовой странице пользователь может выбрать вид сущности и посмотреть их список. На странице просмотра списка сущностей пользователь может выбрать конкретный экземпляр и перейти на его страницу. На странице просмотра конкретного экземпляра сущности пользователь может просматривать и редактировать информацию о нём. С каждой страницы пользователь может вернуться к главной, стартовой.

Построенный граф изображён на рис. 39.

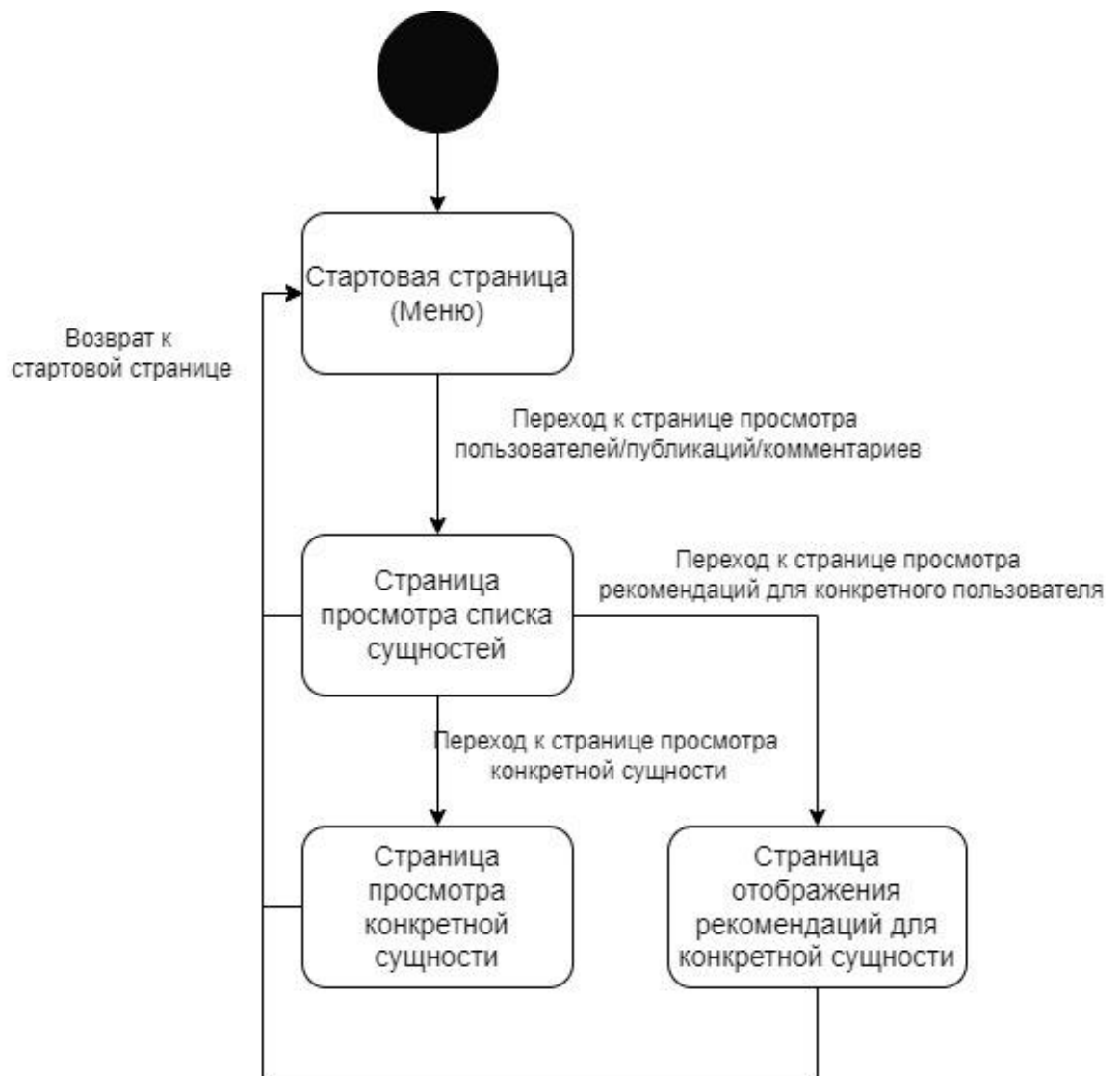


Рис. 39. Граф диалога

Была создана диаграмма отображающая связь с серверной связью. Она отображена на рис. 40.

При просмотре списка сущностей пользователь может выбрать список пользователей, публикаций или комментариев. При просмотре пользователей может быть осуществлён просмотр рекомендаций для конкретного пользователя, данный список также представлен набором публикаций.

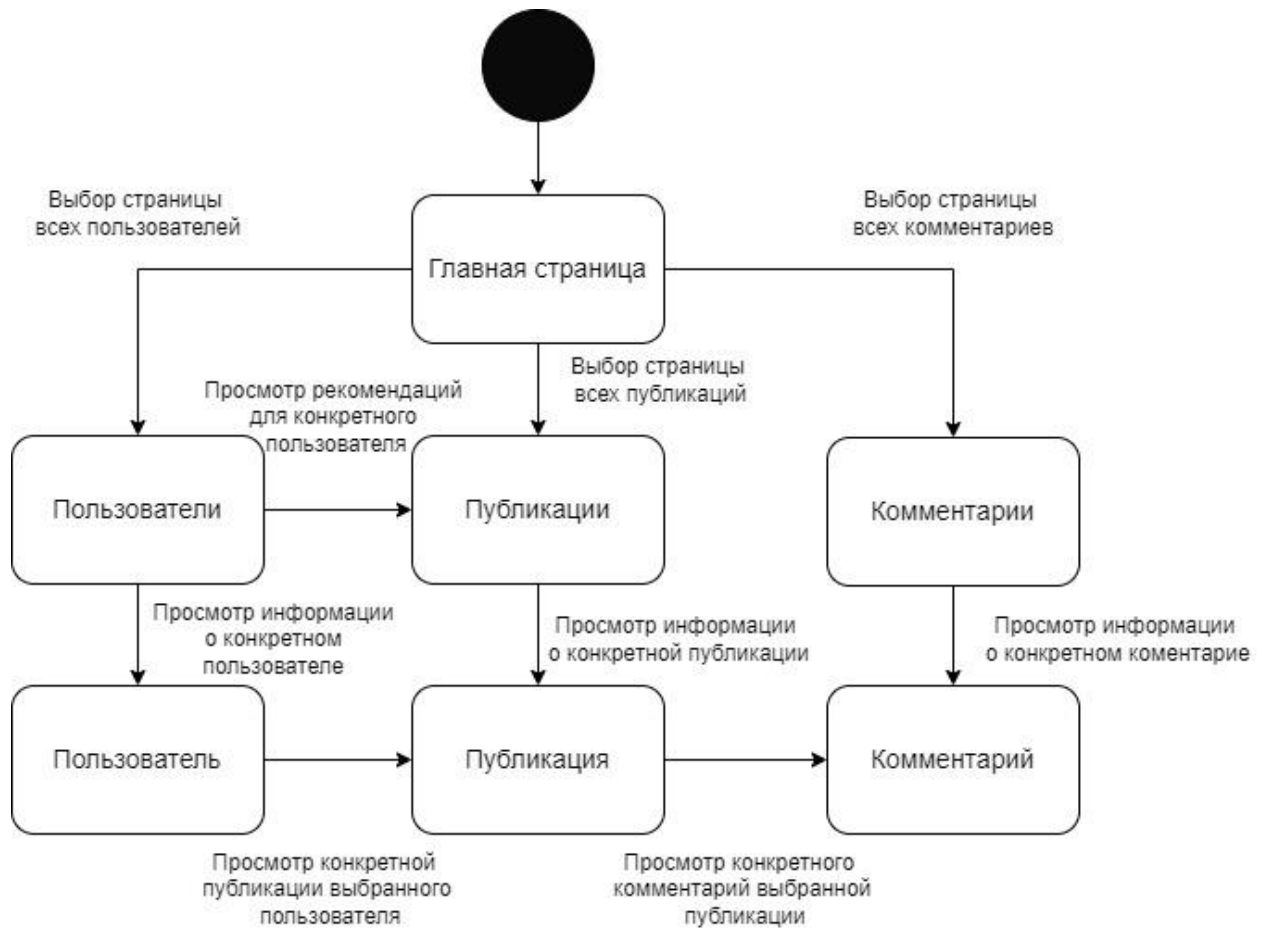


Рис. 40. Граф взаимодействие клиентской части с серверной

Просматривая конкретную сущность, пользователь взаимодействует с конкретным объектом класса пользователь, публикация или комментариев. Со стороны серверной части данные сущности находятся в одном модуле, но разделены разными путями, которые обрабатываются в верхнем слое, в специальных классах-контроллерах, что унифицирует доступ к ним со стороны клиентской части, требуя лишь изменения пути, указанного в запросе.

Следующим этапом проектирования интерфейсной части является разработка форм ввода и вывода информации.

3.4.4.2. Разработка форм ввода вывода информации

Страницы разрабатываемого интерфейса являются web-страницами. Формы ввода вывода находятся на таких страницах. Взаимодействие происходит путём использования клавиатуры и мыши.

Планируется, что пользователь разрабатываемого интерфейса для админ-панели социальной сети нуждается в функционале просмотра и редактирования существующих данных в социальной сети: пользователи, публикации и комментарии. Также администратору необходима возможность просмотра списка публикаций, которые персонально рекомендуются пользователям.

Описываемый далее функционал необходим пользователю-администратору для просмотра и редакции неудобоваримых публикаций, комментариев и пользователей. Помимо функции редактирования, администратор может удалять неподходящий под требования контент.

Для структуризации возможностей разработана стартовая страница, содержащая приветствие и область навигации. Страница показана на рис. 41.

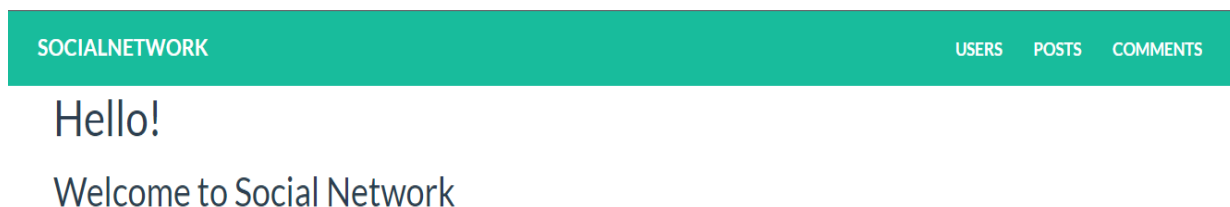


Рис. 41. Стартовая страница

В шапке стартовой страницы расположено навигационное меню, с помощью которого, пользователь может перейти к странице просмотра списков пользователей, публикаций и комментариев (рис. 42 – 44). При нажатии на логотип-название, осуществляется возврат на стартовую страницу-приветствие.

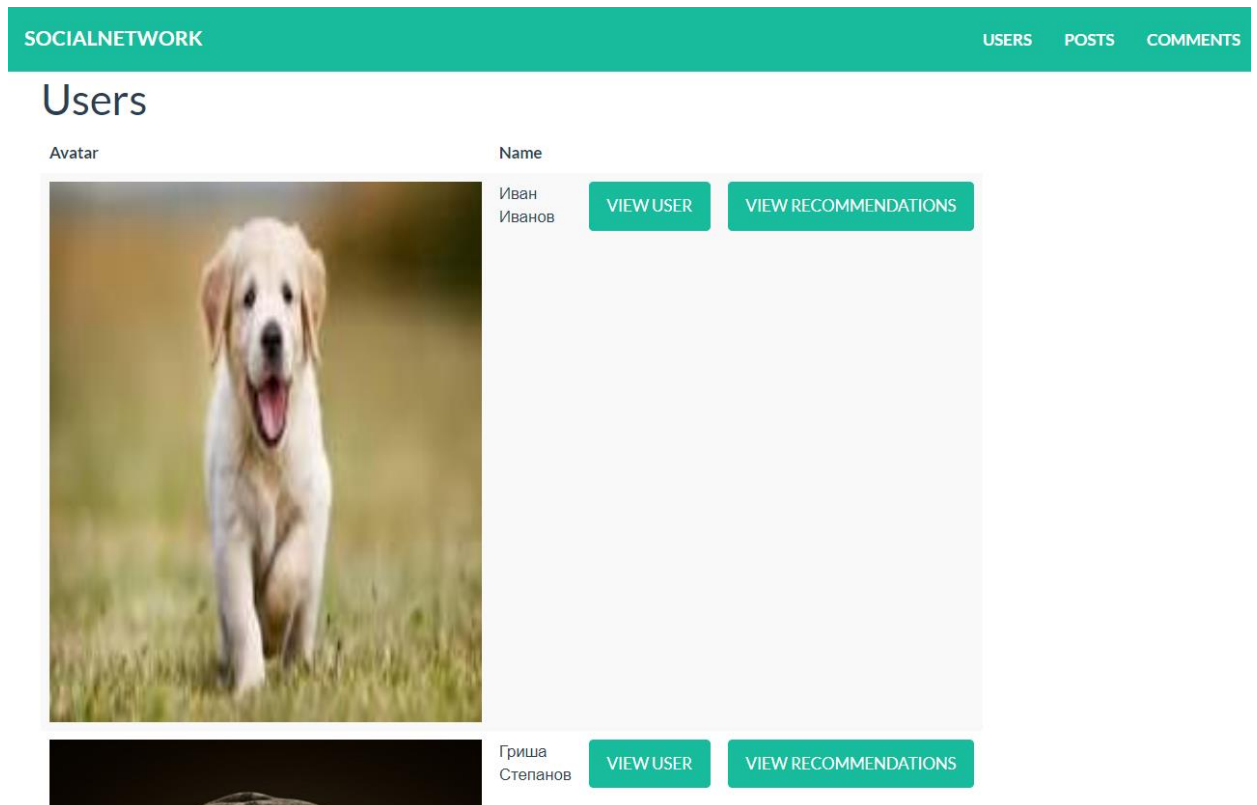


Рис. 42. Страница просмотра пользователей

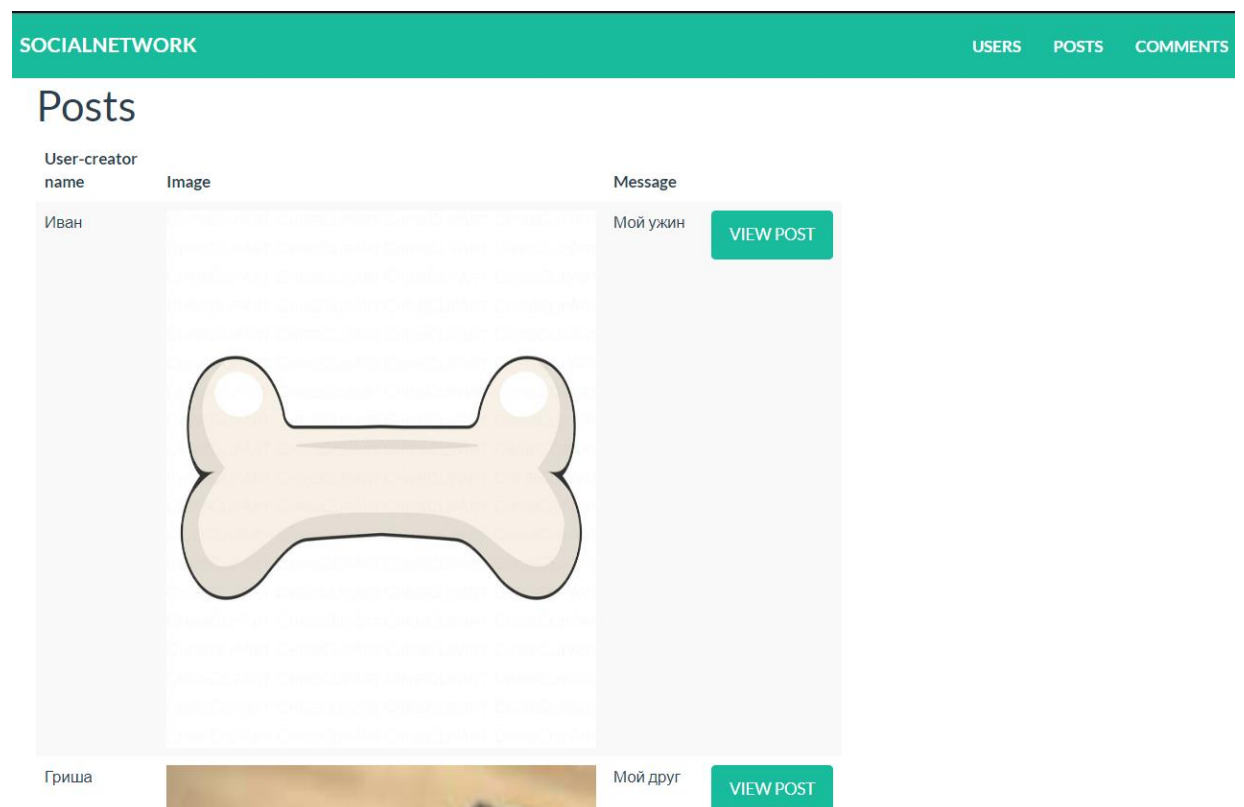


Рис. 43. Страница просмотра публикаций

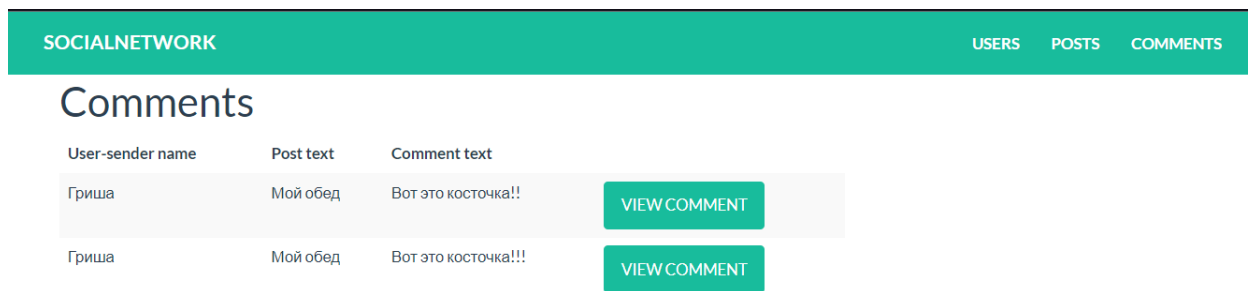


Рис. 44. Страница просмотра комментариев

На каждой из страниц просмотра представлен список с существующими сущностями в системе, рядом с каждой из них расположена кнопка, по нажатию на которую происходит переход на страницу просмотра данной сущности (рис. 45 – 47). При просмотре списка пользователей дополнительно рядом с каждым из них расположена кнопка просмотра персональных рекомендаций для этого пользователя (см. рис. 42).

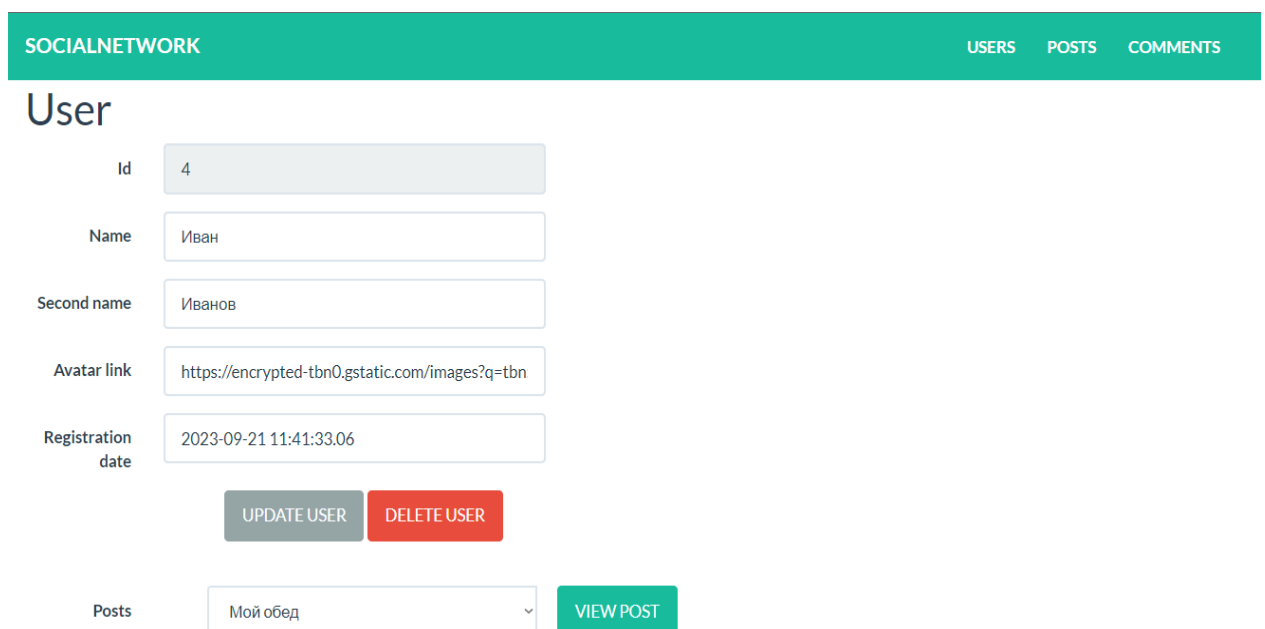


Рис. 45. Страница просмотра пользователя

SOCIALNETWORK
USERS
POSTS
COMMENTS

Post

Id

10

Message

Мой ужин

Image link

https://kartinkof.club/uploads/posts/2022-09/166

Publish date

2023-09-18 11:41:33.061

UPDATE POST

DELETE POST

Comments

VIEW COMMENT

Рис. 46. Страница просмотра публикации

SOCIALNETWORK
USERS
POSTS
COMMENTS

Comment

Id

8

Text

Вот это косточка!!

User-sender id

1

Post id

6

Publish date

2023-09-18 11:41:33.061

UPDATE COMMENT

DELETE POST

Рис. 47. Страница просмотра комментария

На каждой из страниц просмотра сущности расположены текстовые поля для ввода, в которых отображаются нередатируемые и редактируемые параметры сущностей. Для подтверждения изменений данных создана кнопка «UPDATE COMMENT». Администратор также может удалить сущность с помощью кнопки «DELETE POST». При просмотре пользователя существует список его публикаций, к которым можно перейти при помощи соответствующей кнопки. Аналогично для публикаций и комментариев.

3.5. Выводы по разделу 3

В данном разделе последовательно совершены все стадии проектирования ПО, используя объектный подход проектирования системы.

На стадии концептуального моделирования разработаны модели «черного ящика», состава, и «белого ящика» системы. На этапе проектирования информационного обеспечения был произведен выбор подхода к разработке и модели жизненного цикла программного обеспечения

Для реализации были выбраны подходящие технологии и среды проектирования и разработки.

Были разработаны диаграммы потоков данных. Используя язык моделирования UML, разработана логическая модель системы, её компоненты: диаграмма использования, контактная диаграмма классов, структуры данных, диаграммы пакетов, диаграммы классов пакетов и физическая модели разрабатываемой системы. Был проработан интерфейс для панели администрации с функционалом просмотра данных и генерируемых рекомендаций: был разработан граф диалога и макеты web-страниц.

4. Результаты экспериментальной проверки алгоритмического обеспечения системы формирования рекомендаций и анализа контента социальной сети на основе искусственных нейронных сетей

4.1. Описание эксперимента и экспериментальных данных

4.1.1. Описание сценария эксперимента

Адекватность — соответствие модели моделируемому объекту (оригиналу) или процессу. Адекватность — в какой-то мере условное понятие, так как полного соответствия модели реальному объекту быть не может, иначе это была бы не модель, а сам объект. При моделировании имеется в виду адекватность не вообще, а по тем свойствам модели, которые для исследования считаются существенными. [77].

Для анализа корректности и адекватности работы рекомендательной системы требуется проверить каждую составляющую алгоритма и каждую подсистему программного обеспечения.

При создании рекомендательной системы очень важным фактором является оценка качества рекомендаций. В любом прогнозе может быть допущена ошибка, из этого следует, что он не может быть на 100% точным. Поэтому выбор конкретного метода и алгоритма решения заданной задачи происходит, в том числе, исходя из допустимой степени точности предсказания [48].

Оценка и тестирование рекомендательной системы — сложный процесс из-за неоднозначности самого понятия «качество». Существует проблема тестирования моделей формирования рекомендаций — входными данными для оценки предсказания могут быть исторические данные в виде имеющихся в системе реакций пользователя.

Проведение эксперимента осуществляется как над отдельными модулями, так и над всей системой целиком. Будет использован следующий подход: целью эксперимента считается выяснение насколько точно алгоритм предсказывает предпочтения пользователей. Для этого подбираются соответствующие метрики

измерения эффективности алгоритма и формируется дальнейший план проведения эксперимента. Далее необходимо подготовить данные о поведении пользователей, которые можно использовать как для обучения, так и для тестирования алгоритма. Следует разделить данные на две группы – контрольную и тестовую. В контрольной группе используется стандартный, текущий алгоритм формирования рекомендаций, в тестовой группе – новый, разработанный алгоритм, требующий проверки. Алгоритмы применяются к данным, а результат сравнивается. Полученные результаты анализируются по выбранным метрикам, значения метрик сравниваются с известными значениями метрик аналогичных современных технических решений. Делаются выводы о эффективности разработанного алгоритма. В случае неудовлетворительных результатов будет проведена перенастройка элементов алгоритма (весов в определённых методах или моделях) или же пересмотрен набор тестовых данных на его адекватность, затем эксперимент проводится повторно.

После формулировки цели проведения эксперимента, следует описать этапы его проведения.

Для проверки системы требуется совершить следующие этапы:

- подготовить набор данных социальной сети: набор пользователей и их профили (подписки, подписчики), наборы публикаций (изображений) с комментариями и оценками от других пользователей;
- используя изображения из подготовленного набора данных или публичные датасеты для проверки моделей машинного обучения, оценить работу модели описания медиафайлов, получить набор описаний изображений, подсчитать метрики распознавания и классификации;
- используя информацию о пользователях и публикациях (оценки, комментарии, подписки) и сгенерированные на прошлом этапе описания изображений, получить списки рекомендаций публикаций для пользователей с помощью модуля формирования рекомендаций, подсчитать и оценить его показатели эффективности.

4.1.2. Описание будущей интерпретации результатов эксперимента

Требования к решению, а также показатели оценки адекватности модели были предложены в п.1.3. Далее продублированы критерии, которые будут проверены в ходе эксперимента:

- высокий уровень соответствия рекомендаций интересам пользователей – выше 65%;
- точность детектирования объектов на изображении – 90%;
- точность классификации объектов на изображении – 90%;
- корректное формирование рекомендаций (корректное ранжирование предлагаемых публикаций) – выше 50%;

Поведение и отклик пользователя сложно детально формализовать. Ни одна метрика полностью не описывает действия и реакцию пользователя при просмотре рекомендаций, так как на его решение влияет множество факторов.

Предложенные метрики оценки (п.2.6. табл. 5 - 6) выражаются в качестве следующих требований из п.1.3:

- уровень соответствия интересу пользователей – метрики Precision и Recall рекомендательной системы;
- точность детектирования объектов на изображениях – метрика Recall модели тегирования изображений;
- точность классификации объектов на изображениях – метрика Precision модели тегирования изображений;
- корректное ранжирование – метрики Area under the ROC curve, Mean Reciprocal Rank и Normalized Discounted Cumulative Gain рекомендательной системы.

Помимо описанных выше критериев, качество предлагаемой системы будет сравнено с имеющейся. Предлагаемая система должна превосходить имеющуюся по показателям и соответствовать предъявленным требованиям. Система будет сравнена с существующими конкурентами, аналогичными

методами и подходами по следующим параметрам: приведённые выше метрики (recall, precision, AUC, MRR, nDCG), по показателям ошибок (MAE, RMSE).

4.1.3. Описание условий эксперимента

Эксперимент будет проведён на специально подготовленной виртуальной машине. Технически она соответствует минимальным необходимым ресурсам. Там осуществляется установка необходимых программ, сервисов и служб (база данных, среды исполнения python, java и т.д.). Устанавливается разработанное ПО. Настройка и её детали описаны в п.4.2 Эксперимент проводится с специально подготовленными данными (п.4.1.4.). Данные для оценки результатов эксперимента сохраняются в файлы формата «.csv» и в таблицы базы данных. Полученные данные используются для вычисления метрик и построения диаграмм. В приложении «Данные эксперимента» (П5) показаны входные данные, промежуточные и результирующие данные алгоритмов, а также приведён код для вычисления метрик оценки системы.

4.1.4. Описание экспериментальных данных

Экспериментальные данные. Данные о пользователе содержат идентификатор и имя, список подписчиков (массив идентификаторов пользователей), список подписок (массив идентификаторов пользователей), список публикаций (список идентификаторов публикаций). Информация о публикациях представлена в следующем виде: идентификатор, текст описание, изображение, массив оценок, массив комментариев. Формализуем описание пользователя: ‘{user_id, “username”, subscribers:{user_id, ...}, subscriptions:{user_id, ...}, publications:{post_id, ...}}’. Итоговый формат описания публикации: ‘{post_id, “post_text”, filename, commentaries:{user_id:”comment_text”,...}, likes:{user_id, ...}}’. Пример формата входных данных о пользователях и публикациях (все настоящие имена и названия были заменены, экспериментальные данные отображены в П5):

{Имя пользователя: Анна Смирнова;

```

Идентификатор пользователя: 101;
Подписки: {102, 103, 104};
Подписчики: {105, 106};
Публикации: {
  {Идентификатор публикации: 1001;
    Ссылка на изображение: image1.jpg;
    Описание: Прекрасный закат на пляже;
    Комментарии: {201, 202, 203, 204};
    Оценки: {301, 302, 303, 304, 305}};
  {Идентификатор публикации: 1002;
    Ссылка на изображение: image2.jpg;
    Описание: Уютный вечер дома с чашкой горячего чая;
    Комментарии: {205, 206, 207}};
  {Идентификатор публикации: 1003;
    Ссылка на изображение: image3.jpg;
    Описание: Зимний пейзаж с солнечным сиянием;
    Комментарии: {208, 209, 210, 211, 212};
    Оценки: {310, 311, 312, 313}}
}}
{Имя пользователя: Иван Петров;
Идентификатор пользователя: 102;
Подписки: {101, 107, 108};
Подписчики: {109, 110};
Публикации:
{
  Идентификатор публикации: 2001;
  Ссылка на изображение: image4.jpg;
  Описание: Пейзаж горной реки;
  Комментарии: {213, 214, 215, 216}
  Оценки: {314, 315, 316}}
}}
```

Изображения имеют произвольное имя и произвольное содержание, они хранятся в формате '.jpg'. В датасетах для обучения и оценки моделей классификации и тегирования медиафайлов содержится классификация каждого

изображения – то что находится на изображении. Это необходимо для оценки модели описания изображений. Описание изображение формализуется следующим образом: '{filename:{"description", ...}}'.

Пример формата данных о изображениях:

```
{
  "2001": {
    "имяфайла": "image1.jpg",
    "описание": "закат", "пляж", "краски", "отражение"
  },
  "1001": {
    "имяфайла": "image2.jpg",
    "описание": "уют", "вечер", "дом", "чай"
  },
  "2002": {
    "имяфайла": "image3.jpg",
    "описание": "зима", "пейзаж", "солнце", "снег"
  }
}
```

4.2. Разработка методики настройки алгоритмического и программного обеспечения рекомендательной системы социальной сети

Для предложенного алгоритмического обеспечения существует ряд параметров для используемых методов и модели. Их значения заданы по умолчанию в файлах конфигурации. Гиперпараметры можно переопределить для модели машинного обучения (Visual transformer) и метода kNN. Их значения по умолчанию были подобраны в п.2.3.

Для нейронной сети можно изменить конфигурацию её весов. Их можно дообучить для определённого контекста или обучить заново целиком и заменить веса по умолчанию.

Для программного обеспечения необходим этап установки и интеграции рекомендательной системы в имеющуюся инфраструктуру системы социальной сети. Установка рекомендательной системы может включать в себя загрузку и

установку необходимых библиотек и инструментов, после установки необходимо провести интеграцию новой системы с существующей инфраструктурой социальной сети. Это может включать в себя настройку API для обмена данными между системами, создание интерфейсов для отображения рекомендаций пользователю и внедрение функционала обратной связи для оценки качества рекомендаций. После завершения интеграции рекомендательной системы необходимо провести тестирование ее работы на тестовых стендах и оптимизировать алгоритмы рекомендаций, если это необходимо. Также важно провести мониторинг работы системы и регулярно обновлять данные для обучения алгоритма. Финальным этапом будет запуск рекомендательной системы в эксплуатируемой системе на действующем стенде. Важно проводить анализ эффективности системы и реагировать на обратную связь пользователей для постоянного улучшения качества рекомендаций.

Руководство по эксплуатации описано в приложении «Руководство пользователя» (П4). Код разработанного продукта представлен в приложении «Текст программы» (П2).

Описанных этапов настройки и интеграции моделей, алгоритмов и программного обеспечения достаточно для корректного функционирования ПО.

4.3. Описание результатов проверки алгоритма рекомендательной системы социальной сети

После этапа настройки алгоритмического и программного обеспечения рекомендательной системы следует этап проверки, т.е. этап применения настроенного алгоритма к экспериментальным данным. Результаты проверки будут описаны следующим образом: описаны метрики качества – в рамках эксперимента проводится оценка рекомендательной системы с использованием различных, заранее подготовленных метрик, результаты которых показывают насколько точно и полно рекомендации соответствуют интересам пользователей

и несут пользу для них. Результаты обобщаются, оценивая эффективность и потенциал алгоритмического обеспечения.

Для модели тегирования изображений был проведён эксперимент на датасетах опубликованных в общем доступе. Результат оценки модели машинного обучения – данная модель является одной из лучших по показателям в своём классе (её подробная оценка приведена в источнике [79]). В п.2.3 приведена адаптация и тестирование модели.

Результаты эксперимента. На подготовленном датасете, содержащем пользователей, публикации и их оценки, был проведён эксперимент: часть выборки оценок была представлена как исходные данные, другие публикации, которые тоже имели оценки, предложены как кандидаты рекомендаций для текущей и предлагаемой рекомендательной системы. В экспериментальной выборке предлагается 20 пользователей и 2000 публикаций для проверки.

Далее приведён ряд сравнений, отображённых на диаграмме. Они показывают результаты рекомендаций текущей системы, предыдущей и фактических оценок пользователей.

На рис. 48 показано сравнение графиков оценок алгоритмом и фактические оценки пользователей. Значение «1» говорит о том, что рекомендательный алгоритм предлагает эту рекомендацию для этого пользователя (график для алгоритма) или же пользователь действительно оценил эту рекомендацию (график фактических оценок). Диаграмма интерпретируется следующим образом: там, где графики накладываются друг на друга (значения в точках совпадают) алгоритм корректно предлагает публикацию, которая гарантированно понравилась пользователю.

Алгоритм в подавляющем большинстве для выбранного пользователя корректно предсказал его интересы и предложил качественные рекомендации.

На рис. 49 приведена диаграмма для сравнения графиков предыдущего алгоритма с фактическими оценками. Для корректного сравнения алгоритмов данные для диаграммы (тестовый пользователь и публикации) идентичны.

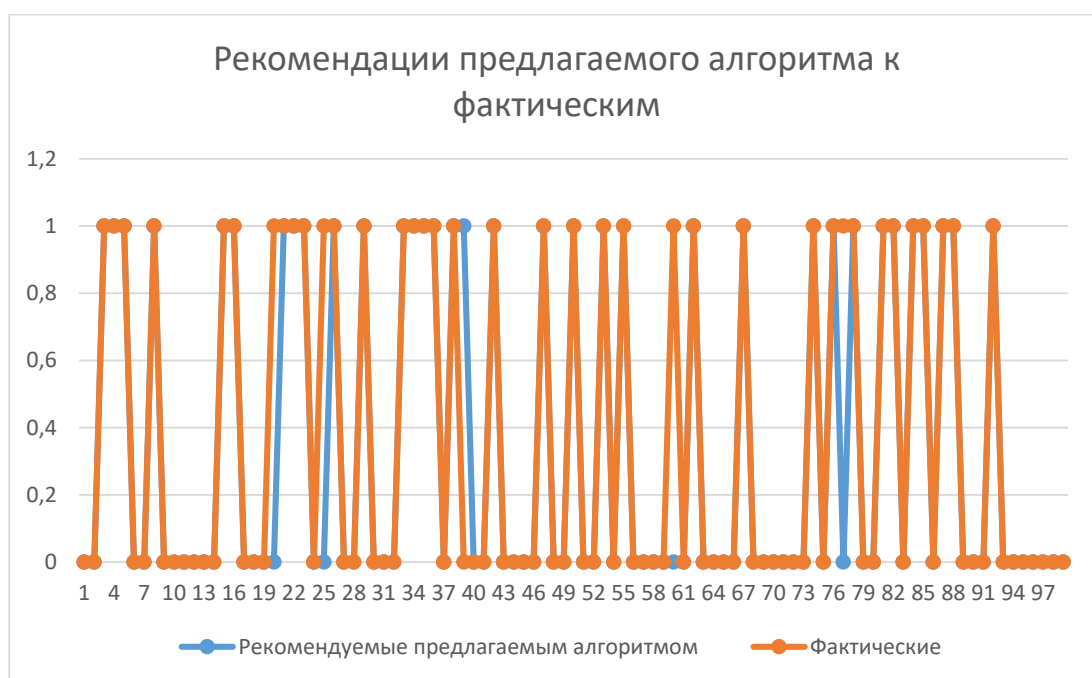


Рис. 48. Диаграмма сравнения графиков предложенных рекомендаций разработанным алгоритмом к фактическим оценкам

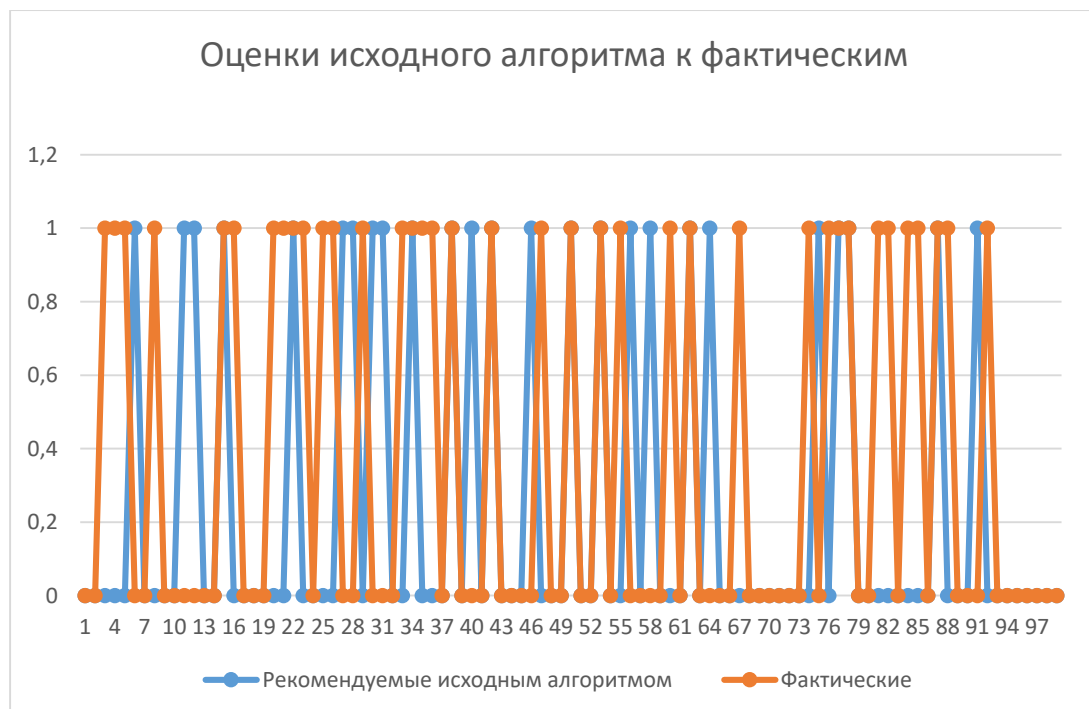


Рис. 49. Диаграмма сравнения графиков предложенных рекомендаций предыдущим алгоритмом к фактическим оценкам

На рис. 50 показана гистограмма, отображающая среднее значение точности алгоритмов на всех пользователях тестовой выборки.

Как мы можем наблюдать, почти половина значений рекомендаций расходятся с действительными оценками, но это и ожидаемо, ведь прошлый алгоритм не позволял формировать персонализированные рекомендации, как итог – его результат не релевантный.

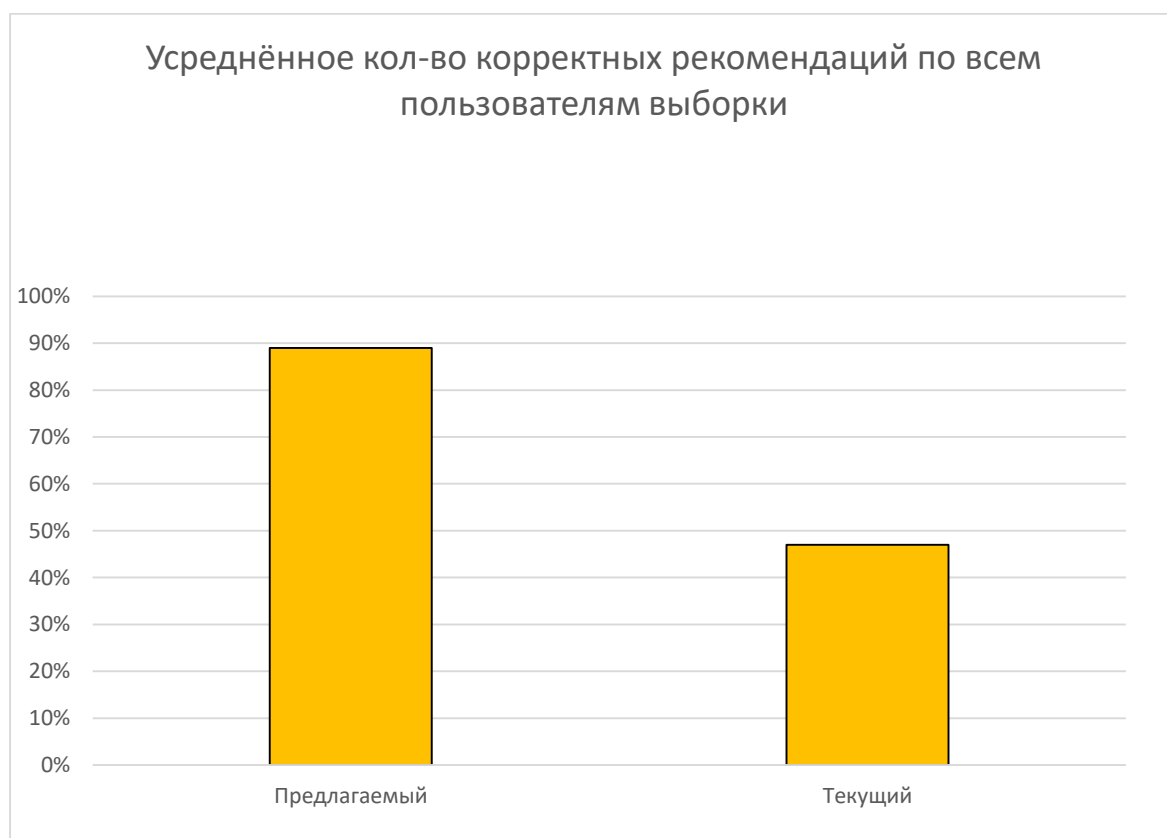


Рис. 50. Гистограмма корректных оценок алгоритмов по всем пользователям

Далее будут приведены вычисленные значения метрик разработанной системы, требования к ней, а также результаты предыдущей системы.

Результаты сравнения систем отражены в табл. 19.

Исходя из полученных в ходе эксперимента значений метрик, делаем вывод о том, что разработанная система соответствует поставленным требованиям и превосходит текущую рекомендательную систему.

Таблица 19

Сравнение показателей метрик рекомендательных систем и требований к ним

Наименование метрики или критерия	Текущая рекомендательная система	Требования к решению	Предлагаемое решение
Precision рекомендательной системы	0.60	>0.65	0.95
Recall рекомендательной системы	0.47	>0.65	0.89
Корректность ранжирования (nDCG@5, nDCG@10, AUC, MRR)	Среднее = 0.35 (nDCG@5 = 0.08, nDCG@10 = 0.07, AUC = 0.66, MRR = 0.61)	>0.5	Среднее = 0.62 (nDCG@5 = 0.31, nDCG@10 = 0.29, AUC = 0.93, MRR = 0.95)
Ассигнату модели тегирования изображений	-	>=0.9	0.95
Precision модели тегирования изображений	-	>=0.9	0.94

Сравним полученные результаты с аналогичными системами. Популярные методы и подходы опробованы на общедоступных датасетах. Датасет для эксперимента был модифицирован уникальными пользователями и публикациями, но сравнить результат можно, добавляя некую погрешность. В источниках [81 - 82] приводятся следующие результаты (табл. 20).

Сравнение некоторых методов и алгоритмов по результатам метрик

Алгоритмы/Модели	Precision	Recall	MAE	AUC	MRR	nDCG@5	nDCG@10
Коллаборативная фильтрация	0.65	0.34	0.87	-	-	-	-
Рекомендации, основанные на сходстве пользователей	0.59	0.63	0.94	-	-	-	-
Экспертная система (искусственный интеллект)	0.64	0.71	0.81	-	-	-	-
Экспертная система (основанная на искусственном интеллекте и сходстве пользователей)	0.64	0.73	0.77	-	-	-	-
EBNR	-	-	-	0.66	0.31	0.34	0.40
DKN	-	-	-	0.65	0.31	0.34	0.40
NPA	-	-	-	0.67	0.32	0.35	0.41
NAML	-	-	-	0.67	0.32	0.35	0.41
NRMS	-	-	-	0.68	0.33	0.36	0.42
UniRec	-	-	-	0.68	0.33	0.36	0.42
Предлагаемое решение	0.95	0.89	0.04	0.93	0.95	0.30	0.28

Как итог, предлагаемое решение имеет конкурентные результаты в описанных метриках (превосходящие показатели точности, полноты и ошибки), имеет небольшую сложность вычисления (меньшие требования к техническим вычислительным ресурсам), а также решает важные проблемы предметной области.

Далее следует проверить программное обеспечение, подвергнуть тестированию его модули.

4.4. Результаты экспериментальной проверки модулей и классов программного обеспечения рекомендательной системы социальной сети

Проводится системное тестирование путём ручной проверки и контроля. Помимо ручной проверки, также проведено unit-тестирование для публичных методов каждого класса-сервиса.

Каждый описанный модуль подвергся тщательной проверке на соответствие ожидаемому поведению.

Описание процесса ручной проверки компонентов ПО в табл. 21 - 26.

Таблица 21

Описание тестирования модуля Recommendations

Наименование теста	Дата	Проводивший тест	Результат, описание
Проверка правильности расчёта матрицы оценок (коллаборативной фильтрации)	22.04.24	Викторов Д.А. - разработчик	Найдены ошибки в работе с индексами матрицы. После исправления проведено повторное тестирование
Проверка правильности расчёта фильтрации по контенту	22.04.24	Викторов Д.А. - разработчик	Корректный расчёт тегов контента из интересов пользователей и описания изображений
Проверка правильности расчёта фильтрации по статистике	23.04.24	Викторов Д.А. - разработчик	Корректное формирование списка рекомендаций по популярности изображений
Проверка исправной работы модели машинного обучения, основанной на Visual Transformer	23.04.24	Викторов Д.А. - разработчик	В ходе тестов исправлена конфигурация по умолчанию для достижения более точных и правильных результатов работы

Таблица 22

Описание тестирования модуля Workflow

Наименование теста	Дата	Проводивший тест	Результат, описание
Корректность выполнения сценариев работы приложения	25.04.24	Викторов Д.А. - разработчик	Проверены сценарии работы приложения
Корректная обработка исключений	25.04.24	Викторов Д.А. - разработчик	Исправлен ряд критических ситуаций при отсутствии (или не правильных) входных данных

Таблица 23

Описание тестирования модуля Presentation logic

Наименование теста	Дата	Проводивший тест	Результат, описание
Корректное наполнение страниц данными	26.04.24	Викторов Д.А. - разработчик	Проверены соответствие ожидаемых данных и отправляемых из модуля бизнес-логики
Корректных переход между страницами	26.04.24	Викторов Д.А. - разработчик	В ходе тестов найден ряд некорректных переходов, которые не обрабатывались и не перенаправлялись на страницу ошибки

Таблица 24

Описание тестирования модуля UI

Наименование теста	Дата	Проводивший тест	Результат, описание
Корректная отрисовка статических элементов	26.04.24	Викторов Д.А. - разработчик	Элементы страниц проверены с различными данными и разным масштабом страницы

Таблица 25

Описание тестирования модуля DB Management

Наименование теста	Дата	Проводивший тест	Результат, описание
Корректное создание схемы базы данных	29.04.24	Викторов Д.А. - разработчик	Схема базы данных проверена на соответствие с ожидаемой
Корректное сохранение данных в таблицу	29.04.24	Викторов Д.А. - разработчик	В ходе тестирования были оптимизированы и поправлены ряд сущностей (таблиц) – индексы, размеры полей (столбцов)

Таблица 26

Описание тестирования модуля File Management

Наименование теста	Дата	Проводивший тест	Результат, описание
Корректное сохранение данных в файлы	29.04.24	Викторов Д.А. - разработчик	Проверена корректность имени сохраняемых файлов и их содержимое

После проверки компонентов, следует описать перспективы применения разработанной системы.

4.5. Перспективы применения предложенных технических решений

Данное решение позволяет продукту предлагать рекомендации высокого качества в случае наличия достаточного количества информации о пользователе, а также решать проблему «Холодного старта» предлагая пользователю менее персонализированные рекомендации, что влияет на увеличение вовлеченности пользователей продуктом, способствует удержанию пользователей на платформе, уменьшению оттока и увеличению вероятности возвращения ушедших пользователей.

Научная значимость рекомендательной системы заключается в исследовании поведения и взаимодействия пользователей в сети, в применении различных математических методов, алгоритмов машинного обучения для анализа данных и предсказания предпочтений пользователей. Данная работа способствует анализу социальных связей внутри медиаплатформ, а также для разработки новых методов и технологий в области информационных технологий, машинного обучения и социальных наук. Применение такого алгоритма стимулирует развитие использования машинного обучения и искусственного интеллекта в целом, способствуя расширению спектра возможностей, повышению эффективности и оптимизации решений в будущем.

Практическая значимость рекомендательной системы социальной сети заключается в том, что она позволяет улучшить пользовательский опыт, предлагая персонализированный контент и рекомендации на основе предыдущего поведения и предпочтений пользователя. Это может увеличить время пользователей, проведенное в сети и доходы социальной сети за счет более эффективной рекламы и продажи маркетинговых услуг. Также рекомендательная система может помогать пользователям находить интересные для них контакты и содержание, что способствует более качественному общению и информационному обмену в социальной сети.

Рассмотрим перспективы по улучшению предложенного решения:

Данное решение можно адаптировать под другой тип медиафайлов – видео формат. Для этого следует расширить модуль описания изображений, используя модель машинного обучения, классифицирующую видео, или текущую модель, работающую с изображениями, но применяя её к каждому кадру видеопотока.

Данное решение можно адаптировать под другой контекст – другой объект рекомендации. Это может быть стриминговая медиаплатформа, где объектом рекомендации является фильм, сериал, видеоролики, или же платформа для осуществления покупок и продаж, где объектом является конкретный товар. В таком случае следует изменить модуль – модель описание медиафайлов, за место него следует использовать описания объекта рекомендации, при его наличии,

или разработать модуль, который описывает объект рекомендации системы. Также следует рассчитать и подобрать оптимальные гиперпараметры для используемых методов и моделей.

На основе данного решения можно создать блок анализа предпочтений пользователей, их потребностей и вкусов, что повысит эффективность маркетинга.

Данное решение можно расширить дополнительными входными данными, например, приветственный опрос при регистрации пользователя в социальной сети, что позволит собрать первоначальные данные для персонализированного решения проблемы «холодного старта».

С текущей тенденцией развития моделей машинного обучения, оптимизировать модель описания изображений, при появлении нейронной сети, которая имеет более качественные показатели точности или быстродействия.

Подведём итоги по текущему разделу.

4.6. Выводы по разделу 4

В данном разделе была выполнена апробация предложенного технического и программного обеспечения с описанной ранее системой рекомендаций публикаций в социальной сети с использованием искусственного интеллекта.

Были сделаны выводы о пользе применения описанной системы, а также перспективы его дальнейшего развития. Модули программного обеспечения были подвержены ручному тестированию, найденные ошибки и неточности были исправлены, после чего было проведено повторное тестирование.

Алгоритмическое обеспечение было оценено по подобранным ранее метрикам, качество модели подходит по предъявляемым требованиям, а также является конкурентным в сравнении с другими анализируемыми техническими решениями в рассматриваемой предметной области и контексте использования.

Представленное решение соответствует требуемым функциональным условиям, а также превосходит имеющуюся ранее систему рекомендаций.

Данное ПО представляет значительный потенциал для улучшения пользовательского опыта, эффективности маркетинга и развития платформы в целом.

Заключение

В ходе работы были представлены выводы по анализу предметной области рекомендательной системы социальной сети, анализа медиаконтента публикаций социальной сети. Были проанализированы существующие аналогичные системы, сделан вывод в создании собственного решения проблемы, выделены его преимущества. Была сформирована задача, требуемая решения, а также разработаны требования и критерии оценки для проектируемого решения.

Сформированы и реализованы методы и алгоритмы для анализа контента и формирования персональных рекомендаций, учитывая проблему «Холодного старта». Благодаря гибриднему методу коллаборативной фильтрации с использованием фильтрации по контенту и по статистике, а также благодаря модели машинного обучения Vision Transformer, удаётся формировать персональные рекомендации высокой точности и с должным уровнем производительности.

Решение отличается научной новизной в виде предложенного алгоритма формирования рекомендаций. В нём также используются модифицированные методы (например, косинусная мера) с целью оптимизации вычислений, повышения точности расчётов и для соответствия предметной области.

Выполнены все этапы проектирования разрабатываемого ПО на основе ООП. В качестве средств реализации использованы язык Java с использованием фреймворка Spring и язык Python для работы с моделью машинного обучения. В качестве клиентской части используются CSS и HTML путём серверного рендеринга с использованием языка шаблонов Thymyleaf. На этапе проектирования был произведён выбор подхода к разработке и модели жизненного цикла ПО. Были созданы диаграммы потоков данных в системе для проектируемого решения. Разработано логическое описание системы, в виде диаграммы вариантов использования, контекстного описания классов,

структуры данных, диаграммы пакетов, диаграммы классов пакетов и физической модели системы.

Была выполнена апробация предложенных моделей и алгоритмов. Представленная математическая модель и алгоритм позволяют формировать персональные рекомендации высокого качества с должным уровнем производительности, также решается проблема «Холодного старта»: при полном отсутствии следов пользователя в социальной сети ему предлагаются слабо персонализированные рекомендации основанные на статистических параметрах о их популярности, при появлении первых действий со стороны пользователя (просмотр, оценка, комментирование) начинается расчёт персонализированных рекомендаций. Предлагаемая система оказалось лучше текущей в ходе сравнения результатов по представленным метрикам. В данных факторах заключается практическое применение решения.

Внедрение такого программного обеспечения позволяет улучшить пользовательский опыт, предлагая персонализированный контент на основе предпочтений пользователя. Предложенное решение готово к интеграции с системой социальной сети, в данный момент система проходит крайний этап внутреннего тестирования в компании ООО «Северотек».

В ходе выполнения работы получен ряд компетенций:

- способен осуществлять критический анализ проблемных ситуаций на основе системного подхода, вырабатывать стратегию действий;
- способен управлять проектом на всех этапах его жизненного цикла;
- способен организовывать и руководить работой команды, вырабатывая командную стратегию для достижения поставленной цели;
- способен анализировать и учитывать разнообразие культур в процессе межкультурного взаимодействия;
- способен разрабатывать оригинальные алгоритмы и программные средства, в том числе с использованием современных интеллектуальных технологий, для решения профессиональных задач;

- способен анализировать профессиональную информацию выделять в ней главное, структурировать, оформлять и представлять в виде аналитических обзоров с обоснованными выводами и рекомендациями;
- способен применять на практике новые научные принципы и методы исследований;
- способен разрабатывать и модернизировать программное и аппаратное обеспечение информационных и автоматизированных систем;
- способен самостоятельно приобретать с помощью информационных технологий и использовать в практической деятельности новые знания и умения, в том числе в новых областях знаний, непосредственно не связанных со сферой деятельности;
- способен применять при решении профессиональных задач методы и средства получения, хранения, переработки и трансляции информации посредством современных компьютерных технологий, в том числе, в глобальных компьютерных сетях;
- способен осуществлять эффективное управление разработкой программных средств и проектов.

Список литературы

1. Лядова, Л. Н. Архитектура рекомендательной системы, настраиваемой на предметные области / Л. Н. Лядова, К. М. Малькова, М. В. Тимофеев // Технологии разработки информационных систем: Материалы VIII Международной научно-технической конференции, Геленджик, 03–09 сентября 2017 года. – Геленджик: Южный федеральный университет, 2017. – С. 98-108.
2. Хвесько, С. Г. Сервис персональных рекомендаций контента Яндекс.дзен как цифровая платформа в современном медиапространстве / С. Г. Хвесько // Интернаука. – 2022. – № 35-2(258). – С. 48-50. – EDN KFQNZZ.
3. Шакиров, С. М. Литература в формате нарратива (анализ публикаций сервиса персональных рекомендаций Яндекс.дзен) / С. М. Шакиров // Знак: проблемное поле медиаобразования. – 2019. – № 1(31). – С. 129-140.
4. Борисенко, А. Е. Алгоритм выстраивания рекомендаций в социальной сети Вконтакте / А. Е. Борисенко // Современное профессиональное образование: опыт, проблемы, перспективы: Материалы VIII Международной научно-практической конференции. В 2-х частях, Ростов-на-Дону, 22 марта 2021 года. – Ростов-на-Дону: Южный университет (ИУБиП), "Издательство ВВМ", 2021. – С. 57-60.
5. Самохвалова, О. С. Разработка приложения для построения рекомендаций сообществ социальной сети ВКонтакте / О. С. Самохвалова, К. Д. Уляшев // Программные продукты, системы и алгоритмы. – 2018. – № 3. – С. 7.
6. Симакова, К. И. Алгоритмы оценки контента в социальной сети "Вконтакте" / К. И. Симакова // Проблемы массовой коммуникации: Материалы международной научно-практической конференции исследователей и преподавателей журналистики, рекламы и связей с общественностью, Воронеж, 16–18 мая 2019 года / Под общей редакцией профессора В.В. Тулупова. – Воронеж: Воронежский государственный университет, 2019. – С. 90-92.

7. Service for Aggregation of Educational Events and Making Recommendations for “VKontakte” Users / V. V. Vasilev, I. I. Konovalov, I. V. Nikiforov [et al.] // Cyber-Physical Systems and Control, 10–11 июня 2019 года, 2020. – P. 65-82.

8. Covington, P., Adams, J., & Sargin, E. (2016). Deep neural networks for youtube recommendations. Paper presented at the RecSys 2016 - Proceedings of the 10th ACM Conference on Recommender Systems, 191-198.

9. Davidson, J., Liebald, B., Liu, J., Nandy, P., & Van Vleet, T. (2010). The YouTube video recommendation system. Paper presented at the RecSys'10 - Proceedings of the 4th ACM Conference on Recommender Systems, 293-296.

10. Zhou, R., Khemmarat, S., & Gao, L. (2010). The impact of YouTube recommendation system on video views. Paper presented at the Proceedings of the ACM SIGCOMM Internet Measurement Conference, IMC, 404-410.

11. Kietzmann, J. H., Hermkens, K., McCarthy, I. P., & Silvestre, B. S. (2011). Social media? get serious! understanding the functional building blocks of social media. *Business Horizons*, 54(3), 241-251.

12. Gomez-Urbe, C. A., & Hunt, N. (2015). The netflix recommender system: Algorithms, business value, and innovation. *ACM Transactions on Management Information Systems*, 6(4).

13. Hallinan, B., & Striphas, T. (2016). Recommended for you: The netflix prize and the production of algorithmic culture. *New Media and Society*, 18(1), 117-137.

14. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30-37.

15. Golbeck, J. (2006). Generating predictive movie recommendations from trust in social networks

16. Starr, J. (2007). Library thing.com: The holy grail of book recommendation engines. *Searcher: Magazine for Database Professionals*, 15(7), 25-32.

17. Guo, G. (2013). Integrating trust and similarity to ameliorate the data sparsity and cold start for recommender systems. Paper presented at the RecSys 2013 - Proceedings of the 7th ACM Conference on Recommender Systems, 451-454.
18. Abuein, Q., Shatnawi, A., & Al-Sheyab, H. (2018). Trusted recommendation system based on level of trust(TRS-LoT). Paper presented at the Proceedings of 2017 International Conference on Engineering and Technology, ICET 2017.
19. Donahue, J., Hendricks, L. A., Guadarrama, S., Rohrbach, M., Venugopalan, S., Darrell, T., & Saenko, K. (2015). Long-term recurrent convolutional networks for visual recognition and description. Paper presented at the Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition, , 07-12-June-2015 2625-2634.
20. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. Paper presented at the Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition, , 2016-December 779-788.
21. Felzenszwalb, P. F., Girshick, R. B., McAllester, D., & Ramanan, D. (2010). Object detection with discriminatively trained part-based models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(9), 1627-1645
22. Redmon, J., & Farhadi, A. (2017). YOLO9000: Better, faster, stronger. Paper presented at the Proceedings - 30th IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, 2017-January 6517-6525
23. Boutell, M. R., Luo, J., Shen, X., & Brown, C. M. (2004). Learning multi-label scene classification. *Pattern Recognition*, 37(9), 1757-1771.
24. Zhang, J., Marszałek, M., Lazebnik, S., & Schmid, C. (2007). Local features and kernels for classification of texture and object categories: A comprehensive study. *International Journal of Computer Vision*, 73(2), 213-238.
25. Chang, C., Lin, C. (2011). LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2

26. Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). Training algorithm for optimal margin classifiers. Paper presented at the Proceedings of the Fifth Annual ACM Workshop on Computational Learning Theory, 144-152.
27. Domingos, P., & Pazzani, M. (1997). On the optimality of the simple bayesian classifier under zero-one loss. *Machine Learning*, 29(2-3), 103-130.
28. Holte, R. C. (1993). Very simple classification rules perform well on most commonly used datasets. *Machine Learning*, 11(1), 63-90.
29. Lecun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444.
30. Cadieu, C. F., Hong, H., Yamins, D. L. K., Pinto, N., Ardila, D., DiCarlo J. J. (2014). Deep neural networks rival the representation of primate IT cortex for core visual object recognition. *PLoS Computational Biology*, 10(12)
31. DiCarlo, J., Zoccolan, D., & Rust, N. (2012). How does the brain solve visual object recognition? *Neuron*, 73(3), 415-434.
32. Serre, T., Kreiman, G., Kouh, M., Cadieu, C., Knoblich, U., & Poggio, T. (2007). A quantitative theory of immediate visual recognition
33. Mutch, J., & Lowe, D. G. (2008). Object class recognition and localization using sparse features with limited receptive fields. *International Journal of Computer Vision*, 80(1), 45- 57.
34. Bouchard, G., & Triggs, B. (2005). Hierarchical part-based visual object categorization. Paper presented at the Proceedings - 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, CVPR 2005, I 710-715.
35. Epshtein, B., & Ullman, S. (2005). Feature hierarchies for object classification. Paper presented at the Proceedings of the IEEE International Conference on Computer Vision, I 220-227
36. Agarwal, S., Awan, A., & Roth, D. (2004). Learning to detect objects in images via a sparse, part-based representation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(11), 1475-1490.
37. Cireşan, D., Meier, U., Masci, J., & Schmidhuber, J. (2012). Multi-column deep neural network for traffic sign classification. *Neural Networks*, 32, 333-338.

38. Turaga, S. C., Murray, J. F., Jain, V., Roth, F., Helmstaedter, M., Briggman, Sebastian Seung. (2010). Convolutional networks can learn to generate affinity graphs for image segmentation. *Neural Computation*, 22(2), 511-538.
39. Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. Paper presented at the Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition, 580-587.
40. Zheng, Y., Liu, Q., Chen, E., Ge, Y., & Zhao, J. L. (2014). Time series classification using multi-channels deep convolutional neural networks
41. Cheng, S., & Zhou, G. (2020). Facial expression recognition method based on improved VGG convolutional neural network. *International Journal of Pattern Recognition and Artificial Intelligence*, 34(7)
42. Coates, A., Lee, H., & Ng, A. Y. (2011). An analysis of single-layer networks in unsupervised feature learning. *Journal of Machine Learning Research*, 15, 215-223.
43. Cohen, I., Sebe, N., Garg, A., Chen, L. S., & Huang, T. S. (2003). Facial expression recognition from video sequences: Temporal and static modeling. *Computer Vision and Image Understanding*, 91(1-2), 160-187.
44. Elkan, C. (2001). The foundations of cost-sensitive learning. Paper presented at the IJCAI International Joint Conference on Artificial Intelligence, 973-978.
45. Lyons, M. J. (1999). Automatic classification of single facial images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 21(12), 1357-1362.
46. Chang, C., Lin, C. (2001). Training v-support vector classifiers: Theory and algorithms. *Neural Computation*, 13(9), 2119-2147.
47. Аппалонова, Н. А. Внедрение искусственного интеллекта в работу компаний Российской Федерации: современное состояние / Н. А. Аппалонова, К. А. Аппалонов // Наука, образование: предпринимательская деятельность в поведенческой экономике, формы реализации и механизмы обеспечения: Материалы Национальной научно-практической конференции, Казань, 03

декабря 2021 года / Под редакцией Н.М. Прусс, А.А. Лопатина. – Казань: Университет управления "ТИСБИ", 2021. – С. 16-19. – EDN HEKBFV.

48. Садрутдинов, Ф. Р. Обзор и сравнение оценок точности методов (SVD, SVD++, NMF) реализации рекомендательной системы на основе коллаборативной фильтрации / Ф. Р. Садрутдинов // Образование в России и актуальные вопросы современной науки: Сборник статей V Всероссийской научно-практической конференции, Пенза, 16–17 мая 2022 года / Под научной редакцией П.А. Гагаева, Е.П. Белозерцева. – Пенза: Пензенский государственный аграрный университет, 2022. – С. 387-391. – EDN VKYUIR.

49. Касымхан, А. М. Методы разработки рекомендательных систем / А. М. Касымхан, А. С. Омарбекова // Студенческий вестник. – 2020. – № 6-4(104). – С. 18-22. – EDN AFGCVU.

50. Сейдаметова, З. С. Алгоритмы для программ-рекомендателей при неполной информации о предпочтениях покупателей / З. С. Сейдаметова // Ученые записки Крымского инженерно-педагогического университета. – 2018. – № 4(62). – С. 147-153. – EDN UQINEX.

51. А.И.Галушкин. Нейронные сети. Основы теории. М., Горячая линия -Телеком, 2010.

52. В.А.Головко. Нейронные сети: обучение, организация и применение. - М., ИПРЖР, 2001

53. Г.Э.Яхьяева. Основы теории нейронных сетей. Интернет-университет информационных технологий, изд-во "Открытые системы".

54. Д.А.Тархов. Нейросетевые модели и алгоритмы. Справочник. М.,Радиотехника, 2014.

55. К.В.Воронцов. Машинное обучение. Курс лекций.

56. С.Хайкин. Нейронные сети: полный курс. 2-е изд. М., "Вильямс", 2006.

57. G.Hinton, N.Srivastava, K.Swarsky. Neural Netwroks for MachineLearning, Lecture 6.

58. Байбикова Т.Н. Комплексы программ для цифровой обработки изображений / Вестник Московского финансово-юридического университета МФЮА. 2016. No 2.
59. Иванова, Г.С. Технология программирования: Учебник для вузов. - М.: Изд-во МГТУ им. Н.Э. Баумана, 2002. -320 с.: ил. (Сер. Информатика в техническом университете.)
60. Лукин, В. В. Технология разработки программного обеспечения. Учебное пособие. - Москва: Гостехиздат, 2015. - 286 с.
61. Назаров, С.В. Архитектура и проектирование программных систем: Монография. / С.В. Назаров. - М.: ИНФРА-М, 2013. - 351 с.
62. Орлов, С.А Технологии разработки программного обеспечения: Учебник/ —СПб.: Питер, 2002. — 464 с.: ил.
63. Ершов, Виноградова и др.: Методика и организация самостоятельной работы студентов. Учебно-методическое пособие, 206 с.
64. Антонов, А.В. Системный анализ: Учебник для вузов / А.В. Антонов. - М.: Высшая школа, 2008. - 454 с.
65. Искусственный интеллект и принятие решений: Интеллектуальный анализ данных. Моделирование поведения. Когнитивное моделирование. Моделирование и управление / Под ред. С.В. Емельянова. - М.: Ленанд, 2012. - 108 с.
66. Гленфорд Майерс, Том Баджетт, Кори Сандлер. Искусство тестирования программ. Издательство - «Диалектика», 2012. — 272 с.
67. Котляров В. П., Коликова Т. В. Основы тестирования программного обеспечения; Интернет-университет информационных технологий, Бином. Лаборатория знаний - Москва, 2006. - 288 с.
68. Альфред В. Ахо, Джон Хопкрофт, Джеффри Д. Ульман. Структуры данных и алгоритмы. — М.: Вильямс, 2000. — 384 с.
69. Крамущенко, В. И. Многоканальные системы передачи информации: конспект лекций / В. И. Крамущенко, Л. Я. Новосельцев, В. Н. Смирнов. – Л.: ЛЭТИ, 1983. – 48 с.

70. Кокин В.М. Моделирование систем: Учебное пособие, Иваново, 2002.
71. Потапов И.В. Структурный Анализ потоков данных. - Издательство СГАУ, 2014.
72. Дал У., Дейкстра Э., Хоор О. Структурное программирование. - М.: Мир, 1976
73. Е.Н.Десятирикова. Введение в информационные технологии: Методические рекомендации.
74. И.Р. Петрова, Р.Х. Фахртдинов, Сулейманова А.А., Разживин И.О., Фазулзянов А.Г. Методология объектноориентированного моделирования. Язык UML.
75. Петрова И.Р., Фахртдинов Р.Х., Сулейманова А.А., Разживин И.О., Фазулзянов А.Г. Методология объектно-ориентированного моделирования. Язык UML: Учебно-методическое пособие. – Казань., 2018.
76. Марк Ричардс. Паттерны архитектуры программного обеспечения: Общие шаблоны архитектуры и способы их применения. - Исходный текст, 2015 / Русский перевод, 2023
77. Боев В.Д. Имитационное моделирование систем: учебное пособие. – М, Юрайт., 2019. — 253 с.
78. Su X., Khoshgoftaar Survey of Collaborative Filtering Techniques. Advances in Artificial Intelligence. 2009.
79. Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, Furu Wei, Baining Guo. Swin Transformer V2: Scaling Up Capacity and Resolution., Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 12009-12019
80. Желязны. Д. Говори на языке диаграмм: пособие по визуальным коммуникациям / Ждин Желязны; 5-е изд. – 2012. -304 с
81. Chung, Yeounoh & Kim, Noo-ri & Park, Chang-yong & Lee, Jee-Hyong. (2018). Improved Neighborhood Search for Collaborative Filtering.

INTERNATIONAL JOURNAL of FUZZY LOGIC and INTELLIGENT SYSTEMS.
18. 29-40. 10.5391/IJFIS.2018.18.1.29.

82. Roberto Navigli, Stefano Faralli, Aitor Soroa, Oier de Lacalle, and Eneko Agirre. 2011. Two birds with one stone: learning semantic models for text categorization and word sense disambiguation. In Proceedings of the 20th ACM international conference on Information and knowledge management (CIKM '11). Association for Computing Machinery, New York, NY, USA, 2317–2320. <https://doi.org/10.1145/2063576.2063955> (дата обращения: 14.02.24)

83. Сайт о программировании METANIT [Электронный ресурс]. – URL: <https://metanit.com> (дата обращения: 14.10.22)

84. Национальный Открытый Университет «ИНТУИТ» [Электронный ресурс]. – URL: <https://intuit.ru> (дата обращения: 14.10.22)

УСТАВ ПРОЕКТА

Программное обеспечение для анализа контента и формирования рекомендаций социальной сети на основе искусственных нейронных сетей

Общие положения

Данный документ является соглашением между основными участниками проекта, определяющим цели проекта, рамки проекта, обязанности участников проекта, основные этапы выполнения работ по проекту.

Соблюдение положения устава является обязательным для всех участников проекта.

Данный документ может меняться в ходе выполнения проекта. Внесение изменений и дополнений в устав проекта инициируется руководителем проекта и утверждается заказчиком (решением управляющего комитета).

Настоящий устав действует, начиная с момента подписания договора на проектные работы и до окончания срока реализации проекта (до подписания акта сдачи-приемки всех работ по проекту).

Требования, удовлетворяющие потребности, пожелания и ожидания заказчика и других участников проекта

Программный продукт должен обеспечить усовершенствование существующих возможностей социальной сети, которая нацелена на медиа публикации. Программное обеспечение заключается в автоматизации процесса в усовершенствовании алгоритма подбора индивидуальных рекомендаций, что позволит добиться повышения качества предлагаемых ею рекомендаций. Данные усовершенствования необходимы для увеличения уровня лояльности пользователей социальной сети, для повышения лояльности имеющихся рекламодателей, привлечения новых пользователей и рекламодателей.

Имеющиеся на рынке программные продукты не в полной мере учитывают специфику и потребности ООО «Северотек», а именно не предлагается комплексного решения для поставленной задачи, а частичные решения имеют существенные недостатки в виде используемых технологий, качества работы и требуемых лицензий.

Программное обеспечение, разработанное в рамках данного проекта, предназначается для автоматизированного использования внутри механизма работы социальной сети.

Производственная необходимость

Перед ООО «Северотек» стоит задача не только разработать и внедрить инструмент в социальную сеть, но и добиться повышения качества продукта. Текущий процент времени, проводимого пользователями социальной сети в разделе рекомендаций, крайне низок, несмотря на то, что данный раздел является основным. Отсутствие интереса пользователю к рекомендациям понижает общее время использования продуктом. Повышение лояльности пользователя, путём улучшения качества рекомендаций поможет повысить время пользования продуктом текущих пользователей, вернуть ушедших пользователей, привлечь новых пользователей, новых рекламодателей, чьи публикации также располагаются в разделе рекомендаций.

Требования к продукту, который является предметом проекта

Требования к функциональным характеристикам:

- возможность конфигурации работы нейронной сети (запуск на GPU/CPU, параметры классификации);
- распознавание и классификация объектов на медиафайле с заданным в конфигурации порогом достоверности;
- формирование рекомендаций для пользователя, с помощью специального алгоритма, вычисляющего интересы пользователя;

- решение задачи «холодного старта» для рекомендательной системы.

Цель проекта

Разработка программного обеспечения для расширения и усовершенствования работы социальной сети: «программное обеспечение для анализа контента и формирования рекомендаций социальной сети на основе искусственных нейронных сетей», которая улучшит алгоритм подбора индивидуальных рекомендаций, что позволит персонализировать предлагаемый контент для пользователя, тем самым добиться повышения качества рекомендаций.

Целью исследования системы является изучение систем-классификаторов и рекомендательных движков, создание алгоритма формирования рекомендаций, исходя из данных пользователя социальной сети, анализируемых искусственным интеллектом.

Критерии оценки успешной реализации проекта

Выдвигаются следующие требования к функциональным характеристикам:

- формирование рекомендаций для пользователя, с помощью специального алгоритма, вычисляющего интересы пользователя;
- решение задачи «холодного старта» для рекомендательной системы.
- высокий уровень соответствия рекомендаций интересам пользователей – выше 65%;
- точность детектирования объектов на изображении – 90%;
- точность классификации объектов на изображении – 90%;
- корректное формирование рекомендаций (корректное ранжирование предлагаемых публикаций) – выше 50%;
- работа рекомендательной системы в режиме реального времени;
- стабильная и безотказная работа рекомендательной системы;
- повышение уровня лояльности пользователя от использования продукта

после внедрения разрабатываемой системы;

- повышение качества продукта после внедрения ПО;
- повышение экономической выгоды от продукта после внедрения ПО.

В таблицах П1.1 – П1.4 описаны контрольные события, участники и руководитель проекта.

Таблица П1.1

Расписание контрольных событий

Этап	Срок	Ответственный
1 этап: Анализ предметной области, постановка задачи	ноябрь – декабрь 2022	Викторов Д.А.
2 этап: Выбор и описание подхода и инструментальных средств разработки программного обеспечения	январь 2023	Викторов Д.А.
3 этап: Выбор и описание математического аппарата,используемого в проекте	февраль - март 2023	Викторов Д.А.
4 этап: Проектирование программного обеспечения: разработка логической модели (модель использования, логическая модель, модель процессов) и физической модели (модель реализации и модель развертывания)	апрель – сентябрь 2023	Викторов Д.А.
5 этап: Разработка программного обеспечения	октябрь – март 2023-2024	Викторов Д.А.
6 этап: Экспериментальная проверка программного обеспечения, оформление результатов проекта	апрель – май 2024	Викторов Д.А.

Информация об участниках проекта

Таблица П1.2

Заказчик проекта

Организация:	ООО «Северотек»
Полномочия:	• формирование общих требований к результату проекта; • назначение руководителя проекта; • контроль хода выполнения работ по проекту; • приемка результатов проекта.
Ответственность:	своевременное обеспечение проекта ресурсами.

Таблица П1.3

Руководитель (менеджер) проекта

Организация:	ФГБОУ ВПО «Череповецкий государственный университет»
ФИО, должность	Ершов Евгений Валентинович, д.т.н., профессор, заведующий кафедрой МПО ЭВМ
Контактная информация	Тел.: +7(8202) 51-90-69
Полномочия	• формирование команды проекта, распределение ответственности, организация взаимодействия между участниками проектной команды и заказчиком; • распределение ресурсов по стадиям (этапов) проекта; • управление рисками проекта, управление процессом решения проблем; • участие в разрешении противоречий в проектных решениях; • планирование, организация и контроль выполнения работ по достижению целей проекта с требуемыми затратами, качеством и в заданный срок; • достижение запланированных результатов; • оценка результативности и разработка отчетных документов по проекту.

Продолжение табл. П1.3

Ответственность	Качественное и своевременное исполнение работ по проекту, достижение запланированных результатов
-----------------	--

Таблица П1.4

Разработчик проекта

Организация:	ФГБОУ ВПО «Череповец»
ФИО, должность	Викторов Даниил Алексеевич, Инженер-программист
Контактная информация	Тел.: +7 (965) 737 78 55
Полномочия	• разработка, проектирование ПО согласно требованиям технического задания; • коммуникация с заказчиком, получение обратной связи по реализации проекта; • распределение ресурсов команды по стадиям проекта.
Ответственность	Качественное и своевременное исполнение работ по проекту, достижение запланированных результатов

Функциональные организации и их участие

Проект реализуется на базе ФГБОУ ВО «Череповецкий государственный университет». Заказчиком проекта является ООО «Северотек».

Компания «Северотек» оказывает полный спектр IT-решений. Благодаря новой модели бизнеса, объединяющей команду в различных областях,

осуществляется возможность предоставлять полный спектр услуг по разработке программного обеспечения любой сложности.

Бюджет проекта

Проект реализуется в рамках освоения магистерской программы «Искусственный интеллект и машинное обучение» по направлению 09.04.04 Программная инженерия.

В рамках реализации данного проекта отсутствует необходимость приобретение программных продуктов специального назначения, т.к. все требования для разработки можно реализовать на свободно распространяемом программном обеспечении.

Для разработки программного обеспечения необходим ноутбук со средними характеристиками, например, LENOVO LEGION Y540 (69900 рублей).

Программное решение предполагает вложение финансовых средств, поэтому возникает вопрос рационального их применения и выбора наиболее эффективного решения задачи.

Трудозатраты на разработку и отладку программы

Расчет экономической эффективности и срока окупаемости проектируемой программы для ЭВМ начинается с расчета трудовых затрат.

Состав разработчиков: один BackEnd программист.

Затраты на оплату труда при разработке программного продукта вычисляются по формуле:

$$Z_{тр} = (Z_{общ} + O_{тч}) * T_n - \text{с окладом} \quad (1)$$

$$Z_{тр} = Z_{общ} * (1 + O_{тч}) * T_n - \text{почасовая} \quad (2)$$

где $Z_{общ}$ – общая зарплата работника за час;

$O_{тч}$ – отчисления с зарплаты, %;

T_n – время написания программы.

Заработная плата программиста за час определяется по следующей формуле:

$$З_{пр} = \frac{С_{тпр}}{\Phi_{вм}} \quad (3)$$

где $С_{тпр}$ – ставка программиста;

$\Phi_{в.м}$ – фонд рабочего времени в месяц, ч.

Заработная плата дополнительная определяется по следующей формуле:

$$З_{доп} = \frac{З_{пр} \cdot Н_{доп}}{100\%} \quad (4)$$

где $З_{пр}$ – заработная плата программиста; $Н_{доп}$ – норма отчислений на дополнительную зарплату (10 %). Зарплата общая вычисляется по следующей формуле:

$$З_{общ} = З_{пр} + З_{доп} \quad (5)$$

Отчисления на соцстрах, фонд занятости и пенсионный фонд вычисляются по следующей формуле:

$$Отч = O_{сс} + O_{фз} + O_{пф} \quad (6)$$

где $O_{сс}$ – отчисления на соцстрах (0,5% от $З_{общ}$);

$O_{фз}$ – отчисления в фонд занятости (0,5% от $З_{общ}$);

$O_{пф}$ – отчисления в пенсионный фонд (2% от $З_{общ}$).

Все данные по заработной плате сводятся в табл. П1.5.

Данные по заработной плате

Должность	Время работы, мес.	Ст _{пр} , руб.	З _{пр} , руб/час.	З _{доп} , руб/час.	З _{общ} , руб/час.	Отч, руб/час.	З _{тр} , руб.
Программист BackEnd	10	49 845	311,53	31,15	342,03	10,28	422 772

Стоимость одного часа машинного времени при создании системы рассчитывается по формуле:

$$C_{м.вр} = \frac{Ц_k}{C_{сл.к} * K_{р.д} * V_{рс}} + C_{т.э} * M_{вс} \quad (7)$$

где $C_{м.вр}$ – стоимость одного часа машинного времени, руб./ч; $Ц_k$ – покупная цена компьютера, руб.; $C_{сл.к}$ – срок службы компьютера, год; $K_{р.д}$ – количество рабочих дней в году; $V_{рс}$ – время работы компьютера в течение суток, ч; $C_{т.э}$ – стоимость одного кВт · ч электроэнергии, руб.; $M_{вс}$ – мощность вычислительной системы, кВт.

$$C_{м.вр} = 69990 / (5 * 250 * 8) + (3.8 * 0,4) = 10,62 \text{ руб./ч.}$$

Время использования вычислительной техники рассчитывается по следующей формуле:

$$V_{р.в.т} = K_{д.р} \cdot V_{рс} \quad (8)$$

где $K_{д.р}$ – количество дней разработки ПО;

$V_{рс}$ – время работы компьютера в течение суток, ч.

$$V_{р.в.т.} = 200 * 6 = 1200 \text{ ч.}$$

Затраты на использование машинного времени вычисляются по формуле:

$$З_{м.вр} = C_{м.вр} \cdot Вр_{в.т}, \quad (9)$$

где $З_{м.вр}$ – затраты на использование машинного времени, руб.; $C_{м.вр}$ – стоимость одного часа машинного времени, руб./ч;

$Вр_{в.т}$ – время использования вычислительной техники, ч.

$$З_{м.вр} = 10,62 \cdot 1200 = 12\,744 \text{ руб.}$$

Затраты на носители оставляют 2% от стоимости вычислительной техники и вычисляются по следующей формуле:

$$З_{н.и} = Ц_k \cdot 0,02 \quad (10)$$

$$З_{н.и} = 69900 \cdot 0,02 = 1400 \text{ руб.}$$

Затраты на текущий и профилактический ремонт составляют 4% от цены вычислительной техники и вычисляются по следующей формуле:

$$З_{н.и} = Ц_k \cdot 0,04 \quad (11)$$

$$З_{н.и} = 69900 \cdot 0,04 = 2800 \text{ руб.}$$

Прочие эксплуатационные расходы включают в себя затраты на освещение, отопление, охрану, уборку и текущий ремонт помещений. Они принимаются в размере 10 % от стоимости помещения (или его аренды), где происходит разработка программного продукта, и вычисляются по следующей формуле:

$$З_{пр} = C_{пм} \cdot 10 \quad (12)$$

где $C_{пм}$ – стоимость помещения, руб.

$$З_{пр} = 15000 \cdot 0,1 = 1500 \text{ руб.}$$

Себестоимость разработки определяется по следующей формуле:

$$C_{п.п} = Z_{тр} + Z_{м.вр} + Z_{н.и} + Z_{рем} + Z_{пр} \quad (13)$$

$$C_{п.п} = 422772 + 12744 + 1400 + 2800 + 1500 = 441\,216 \text{ руб.}$$

Расчет цены программного продукта

Установление определенной цены на программный продукт служит для последующей его продажи и получения прибыли. Очень важно назначить цену таким образом, чтобы она не оказалась слишком высокой или слишком низкой.

Для определения минимальной цены, ниже которой разработчику будет невыгодно продавать программный продукт, используется следующая формула:

$$Ц_{п.п} = C_{п.п} \cdot (1 + H_{пр}), \quad (14)$$

где $Ц_{п.п}$ – цена программного продукта, руб.; $C_{п.п}$ – себестоимость программного продукта, руб.; $H_{пр}$ – норматив прибыли (20 %, в формуле $H_{пр} = 0,2$).

$$Ц_{п.п} = 441216 \text{ руб.} \cdot (1 + 0,2) = 529\,459,2 \text{ (руб)}$$

Расчет экономической эффективности

В расчет экономической эффективности входит определение эксплуатационных расходов и капитальных затрат потребителя, годовой экономии эксплуатационных расходов у одного потребителя, срока окупаемости программного продукта, годового экономического эффекта.

Расходы потребителя, связанные с эксплуатацией программы, определяются по следующей формуле:

$$P_{э.п} = Bp_{п.п} \cdot C_{м.вд} + Ц_{п.п} / C_{сл} \quad (15)$$

где $P_{э.п}$ – эксплуатационные расходы потребителя, руб.;

$V_{р.п}$ – объем машинного времени в течение года, необходимый для решения данной задачи с использованием программы, ч;

$C_{м.вр}$ – стоимость одного часа машинного времени, руб./ч;

$\Pi_{п.п}$ – цена программного продукта, руб.;

$C_{сл}$ – срок службы программного продукта, год. Обычно составляет 1 – 2 года, затем выпускается новая версия программного продукта.

$$C_{м.вр} = 250 \cdot 8 = 2000 \text{ ч.}$$

$$P_{э.п} = 2000 \cdot 10,62 + 529459,2/2 = 285\,969,5 \text{ руб.}$$

На момент внедрения программного продукта у потребителя все работы выполнялись вручную. Следовательно, капитальные затраты рассчитываются по формуле:

$$P_{\text{кап}} = \frac{V_{р.п} \cdot K_{\text{ЭВМ}}}{\Phi_{\text{вр}}} + \Pi_{п.п}, \quad (16)$$

где $P_{\text{кап}}$ – капитальные расходы потребителя, руб.;

$V_{р.п}$ – объем машинного времени в течение года, необходимый для решения данной задачи с использованием программы, ч;

$\Phi_{\text{вр}}$ – полезный годовой фонд времени работы вычислительной техники, принимается условно 2000 ч в год;

$K_{\text{ЭВМ}}$ – капитальные затраты на вычислительную технику, для которой предназначена программа, руб.

Капитальные затраты на вычислительную технику рассчитываются по формуле:

$$K_{\text{ЭВМ}} = \Pi_{\text{ЭВМ}} + P_{п.п}, \quad (17)$$

где $\Pi_{\text{ЭВМ}}$ – цена вычислительной техники, руб.; $P_{п.п}$ – прочие расходы потребителя, связанные с помещением (отопление, освещение, уборка и т.д.),

принимаются в размере 10 % от стоимости помещения потребителя (или его аренды), руб.

$$K_{\text{эвм}} = 69900 + 1000 = 70900 \text{ руб.}$$

$$P_{\text{кап}} = (2000 * 70900) / 2000 + 529459,2 = 600\,359,2 \text{ руб.}$$

Для расчета годовой экономии эксплуатационных расходов потребителя вычисляются эксплуатационные затраты потребителя при решении задачи вручную:

$$P_{\text{э,руч}} = 1,21 * \text{ФЗП} * 12 \quad (18)$$

где $P_{\text{э,руч}}$ – эксплуатационные расходы потребителя при решении задачи вручную, руб.;

ФЗП – фонд заработной платы персонала, обслуживающего решение задачи вручную, руб.;

12 – количество месяцев в году;

1,21 – поправочный коэффициент.

$$P_{\text{э,руч}} = 1.21 * 50000 * 12 = 726\,000 \text{ руб.}$$

Тогда годовая экономия эксплуатационных расходов у одного потребителя рассчитывается по формуле:

$$\text{Э} = P_{\text{э,руч}} - P_{\text{э.п.}} \quad (19)$$

$$\text{Э} = 726000 - 285969,5 = 440\,030,5 \text{ руб.}$$

Срок окупаемости программного продукта рассчитывается по формуле:

$$T_{\text{ок}} = \frac{P_{\text{кап}}}{\text{Э}}. \quad (20)$$

$$T_{\text{ок}} = 600\,359,2 / 440\,030,5 = 1,36 \text{ год.}$$

Годовой экономический эффект, получаемый одним потребителем, рассчитывается по формуле:

$$\text{ЭЭ} = \text{Э} - E_{\text{н}} \cdot P_{\text{кап}}, \quad (21)$$

где $E_{\text{н}}$ – нормативный коэффициент эффективности дополнительных капитальных вложений, равный 0,15.

$$\text{ЭЭ} = 440\,030,5 - 0,15 \cdot 600\,359,2 = 349\,976,62 \text{ руб.}$$

Основные риски проекта

Риски в рамках реализуемого проекта представлены в табл. П1.6.

Таблица П1.6

Риски проекта

№ п/п	Риски проекта	Мероприятия по управлению рисками (по предотвращению риска / пореагированию при возникновении риска)
1.	Не своевременная сдача проекта в установленные сроки.	Своевременный анализ сроков и успеваемости работ по срокам
2.	Ошибка в оценке бюджета проекта	Анализ оценки бюджета с заказчиком, пересмотр бюджета при необходимости.
3.	Ошибки при проектировании системы.	Проектирование системы согласно общепринятым стандартам. Анализ моделей, полученных в ходе проектирования, с заказчиком и наставником
4.	Использование неактуальных технологий.	Анализ используемых технологий, сравнение их с аналогами, изучение их лицензий использования.
5.	Использование не лицензируемых проектов/программных модулей для решения задач проекта	

Реальная бизнес-ситуация, служащая обоснованием проекта с данными о прибыли на инвестиции.

Проект не является коммерческим, доходы по проекту в период его реализации не предусмотрены.

Текст программы

```

@Controller
public class RestController {
    @Value("${spring.application.name}")
    private String appName;
    @Autowired
    private IPostService postService;
    @Autowired
    private IStatisticService statisticService;

    @GetMapping("/")
    public String homePage(Model model) {
        model.addAttribute("appName", appName);
        List<CommentDTO> commentDTOS =
            statisticService.calculateNewComments();
        List<PostDTO> postDTOS =
            statisticService.calculateNewPosts();
        List<UserDTO> userDTOS =
            statisticService.calculateNewUsers();
        model.addAttribute("newPosts", postDTOS);
        model.addAttribute("newComments",
            commentDTOS);
        model.addAttribute("newUsers", userDTOS);
        return "home";
    }

    @GetMapping("/activeCommentator")
    public String
        findActiveCommentator(@ModelAttribute("selectedUser")
            UserDTO user, Model model) {
        UserDTO mostActiveCommentator =
            statisticService.getMostActiveCommentator(user.getId());
        return "redirect:/users/ById/" +
            mostActiveCommentator.getId();
    }

    @GetMapping("/commentedPost")
    public String
        findPopularPost(@ModelAttribute("selectedUser")
            UserDTO user, Model model) {
        PostDTO mostCommentedPostOfUser =
            statisticService.getMostCommentedPostOfUser(user.getId());
        return "redirect:/posts/ById?postId=" +
            mostCommentedPostOfUser.getId();
    }

    @GetMapping("/all")
    public String getAllPosts(Model model) {
        List<PostDTO> all = postService.getAll();
        model.addAttribute("posts", all);
        return "posts";
    }

    @PostMapping("/delete/{postId}")
    public String deleteById(@PathVariable Long
        postId, Model model){
        try {
            postService.delete(postId);
            return "redirect:/posts/all";
        } catch (Exception e) {
            return "redirect:/error";
        }
    }

    @GetMapping("/ById")
    public String
        getPostById(@ModelAttribute("selectedPost")
            PostDTO post, @ModelAttribute("selectedComment")
            CommentDTO comment, Model model) {
        try {
            PostDTO postDTO =
                postService.getById(post.getId());
            model.addAttribute("post", postDTO);
            model.addAttribute("selectedComments",
                postDTO.getComments());
            return "post";
        } catch (Exception e) {
            return "redirect:/error";
        }
    }

    @PostMapping("/add")
    public String addPost(@RequestParam String
        message, @RequestParam String imageLink,
        @RequestParam Long userId, Model model) {
        try {
            PostDTO postDTO = new PostDTO();
            postDTO.setMessage(message);
            postDTO.setImageLink(imageLink);

            postDTO.setPublishDate(Date.from(Instant.now()));
            postService.create(postDTO, userId);
            return "redirect:/posts/all";
        } catch (Exception e) {
            return "redirect:/error";
        }
    }

    @PostMapping("/update/{postId}")
    public String updatePost(@PathVariable Long
        postId, @RequestParam String message,
        @RequestParam String imageLink,
        @RequestParam("publishDate")
        @DateTimeFormat(pattern = "yyyy-MM-dd") Date
        publishDate, Model model) {
        try {
            PostDTO postDTO = new PostDTO();
            postDTO.setMessage(message);
            postDTO.setImageLink(imageLink);
            postDTO.setPublishDate(publishDate);
            postDTO.setPostId(postId);
            postService.update(postDTO);
            model.addAttribute("selectedPost", postDTO);
            return "redirect:/posts/all";
        } catch (Exception e) {
            return "redirect:/error";
        }
    }
}

```

Рис. П2.1. Модуль «RestController.java»

```

@Entity
@Table(name = "posts")
public class Post {
    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private Long postId;
    @ManyToOne @JoinColumn(name="user_id",
    nullable=false) private User user;
    @OneToMany(mappedBy="post", cascade =
    CascadeType.ALL)
    private Set<Comment> comments;
    private String message;
    private String imageLink;
    private Date publishDate;
    public Post(Long postId, User user, String message,
    String imageLink, Date publishDate, Set<Comment>
    comments) {
        this.postId = postId;
        this.user = user;
        this.message = message;
        this.imageLink = imageLink;
        this.publishDate = publishDate;
        this.comments = comments; }
    public Long getPostId() {
        return postId; }
    public void setPostId(Long postId) {
        this.postId = postId; }

```

```

    public User getUser() {
        return user; }
    public void setUser(User user) {
        this.user = user; }
    public String getMessage() {
        return message; }
    public void setMessage(String message) {
        this.message = message; }
    public String getImageLink() {
        return imageLink; }
    public void setImageLink(String imageLink) {
        this.imageLink = imageLink; }
    public Date getPublishDate() {
        return publishDate; }
    public void setPublishDate(Date publishDate) {
        this.publishDate = publishDate; }
    public Set<Comment> getComments() {
        return comments; }
    public void setComments(Set<Comment>
    comments) { this.comments = comments; }
    @Override public String toString() {
        return "Post{" +
            "postId=" + postId +
            ", message=" + message + "\n" +
            ", imageLink=" + imageLink + "\n" +
            ", publishDate=" + publishDate + '>'; }

```

Рис. П2.2. Модуль «Post.java»

```

@Service public class PostService implements
IPostService {
    @Autowired private PostMapper postMapper;
    @Autowired private PostRepository postRepository;
    @Autowired private UserRepository
    userRepository;
    public Post create(PostDTO post, Long userId){
        Optional<User> byId =
        userRepository.findById(userId);
        if(byId.isEmpty()){
            throw new IllegalArgumentException("No such
            user with id: " + userId); }
        Post entity = postMapper.toEntity(post);
        entity.setUser(byId.get());
        Set<Comment> comments =
        entity.getComments();
        if(comments != null) {
            comments.forEach(comment -> {
                comment.setPost(entity);
                comment.setUser(entity.getUser());}); }
        Post saved = postRepository.save(entity);
        return saved; }
    @Override public void delete(Long postId){
        postRepository.deleteById(postId); }
    @Override public void update(PostDTO post){

```

```

        if (postRepository.existsById(post.getPostId())) {
            Post entity =
            postRepository.getById(post.getPostId());
            entity.setMessage(post.getMessage());
            entity.setImageLink(post.getImageLink());
            entity.setPublishDate(post.getPublishDate());
            Post save = postRepository.save(entity); } }
    @Override public List<PostDTO> getAll(){
        List<Post> all = postRepository.findAll();
        List<PostDTO> allDto = all.stream().map(entity -
        >.toDto(entity)).collect(Collectors.toList());
        return allDto; }
    @Override
    public PostDTO getById(Long postId){
        Optional<Post> postById =
        postRepository.findById(postId);
        if(postById.isEmpty()){return null; }
        PostDTO result =
        postMapper.toDto(postById.get());
        return result; }
    public Post getEntityById(Long postId){
        Optional<Post> postById =
        postRepository.findById(postId);
        if(postById.isEmpty()){return null; }
        return postById.get(); }

```

Рис. П2.3. Модуль «PostService.java»

```

@Entity
@Table(name = "users")
public class User { @Id
@GeneratedValue(strategy=GenerationType.AUTO)
    private Long userId;
    private String avatar;
    private String firstName;
    private String secondName;
    private Date registrationDate;
    @OneToMany(mappedBy="user", cascade =
CascadeType.ALL)
    private Set<Post> posts;
    @OneToOne(mappedBy = "user")
    private Recommendation recommendation;
    public User() {}
    public User(Long userId, String avatar, String
firstName, String secondName, Date registrationDate,
Set<Post> posts, Recommendation recommendation) {
        this.userId = userId;
        this.avatar = avatar;
        this.firstName = firstName;
        this.secondName = secondName;
        this.registrationDate = registrationDate;
        this.posts = posts;
        this.recommendation = recommendation; }
    public Long getUserId() {
        return userId; }
    public void setUserId(Long userId) {
        this.userId = userId; }
    public String getFirstName() {
        return firstName; }

    public void setFirstName(String firstName) {
        this.firstName = firstName; }
    public String getSecondName() {
        return secondName; }
    public void setSecondName(String secondName) {
        this.secondName = secondName; }
    public Date getRegistrationDate() {
        return registrationDate; }
    public void setRegistrationDate(Date
registrationDate) {
        this.registrationDate = registrationDate; }
    public String getAvatar() { return avatar; }
    public void setAvatar(String avatar) {
        this.avatar = avatar; }
    public Set<Post> getPosts() { return posts; }
    public void setPosts(Set<Post> posts) {this.posts =
posts; }
    public Recommendation getRecommendation() {
        return recommendation; }
    public void setRecommendation(Recommendation
recommendation) {
        this.recommendation = recommendation; }
    @Override public String toString() { return "User{"
+ "userId=" + userId +
        ", avatar=" + avatar + "\" +
        ", firstName=" + firstName + "\" +
        ", secondName=" + secondName + "\" +
        ", registrationDate=" + registrationDate +
        ", posts=" + posts +
        ", recommendation=" + recommendation +
        "'; } }

```

Рис. П2.4. Модуль «User.java»

```

@Service
public class UserService implements IUserService {
    @Autowired private UserMapper userMapper;
    @Autowired private UserRepository
userRepository;
    @Override public User create(UserDTO user){
        User entity = userMapper.toEntity(user);
        Set<Post> posts = entity.getPosts();
        if(posts != null) {
            posts.forEach(post -> post.setUser(entity));}
        User saved = userRepository.save(entity);
        return saved; }
    @Override public void delete(Long userId){
        userRepository.deleteById(userId); }
    @Override public void update(UserDTO user){
        if (userRepository.existsById(user.getUserId())) {
            User entity =
userRepository.findById(user.getUserId());
            entity.setAvatar(user.getAvatar());
            entity.setFirstName(user.getFirstName());
            entity.setSecondName(user.getSecondName());
            entity.setRegistrationDate(user.getRegistrationDate());
            User save = userRepository.save(entity); } }
    public User getEntityById(Long userId){
        Optional<User> userById =
userRepository.findById(userId);
        if(userById.isEmpty()){
            return null; } return userById.get();}
    @Override public List<UserDTO> getAll(){
        List<User> all = userRepository.findAll();
        List<UserDTO> allDto = all.stream().map(entity -
>
userMapper.toDto(entity)).collect(Collectors.toList());
        return allDto; }
    @Override public UserDTO getById(Long userId){
        Optional<User> userById =
userRepository.findById(userId);
        if(userById.isEmpty()){ return null; }
        UserDTO result =
userMapper.toDto(userById.get());
        return result; } }

```

Рис. П2.5. Модуль «UserService.java»

```

import pandas as pd
from surprise import Dataset
from surprise import Reader

posts_content_dict = getPostContentTags()
user_posts_dict = getUserPostTags()
user_follows_dict = getUserFollowsTags()
postContentEntries = posts_content_dict.items()
postsList = posts_content_dict.keys()
userPosttEntries = user_posts_dict.items()
userList = user_posts_dict.keys()
userFollowsEntries = user_follows_dict.items()
#calcing userPostContetFeatures
userPostContetFeatures = dict()
for u_p_entry in userPosttEntries:
    userName = u_p_entry[0]
    userPosts = u_p_entry[1]
    content_set = set()

    for postId in userPosts:
        post_content = posts_content_dict.get(postId)

        for content in post_content:
            content_set.add(content)
        userPostContetFeatures.update({userName:
content_set})
    print(userPostContetFeatures)

#calcing userFollowsContetFeatures
userFollowsContetFeatures = dict()
for u_f_entry in userFollowsEntries:
    userName = u_f_entry[0]
    userFollows = u_f_entry[1]
    content_set = set()
    for followId in userFollows:
        follow_content =
userPostContetFeatures.get(followId)
        for content in follow_content:
            content_set.add(content)

        userFollowsContetFeatures.update({userName:
content_set})

    print(userFollowsContetFeatures)

# calcing userAllContentFeatures
userAllContentFeatures = dict()
for entries in userPostContetFeatures.items():
    user = entries[0]
    currPostContent = entries[1]
    currFollowsContent =
userFollowsContetFeatures.get(user)
    allContentFeatures = set(currPostContent) |
set(currFollowsContent)

userAllContentFeatures.update({user:allContentFeatur
es})
print(userAllContentFeatures)

userRecommendations = dict()
for entry_u_c in userAllContentFeatures.items():
    user = entry_u_c[0]
    contentList = entry_u_c[1]
    # filteredItterable = filter(lambda postContentEntry:
any(postContentEntry[1] in contentList),
postContentEntries)
    notUserPosts = filter(lambda postContentEntry:
postContentEntry[0] not in user_posts_dict.get(user),
postContentEntries)
    filteredItterable = filter(lambda postContentEntry:
any(filter(lambda postContent: postContent in
contentList, postContentEntry[1])), notUserPosts)
    # print(list(filteredItterable))
    mappedList=list(map(lambda entry: entry[0],
filteredItterable))
    userRecommendations.update({user:mappedList})
print(userRecommendations)

```

Рис. П2.6. Модуль «ContentFiltering.py»

Спецификации

Данное приложение содержит спецификацию на разработанную программную документацию и компоненты программного обеспечения. Спецификации на разработанную программную документацию приведена в табл. ПЗ.1.

Таблица ПЗ.1

Спецификации

Обозначение	Назначение	Примечание
1	2	3
Документация		
Викторов_ДА_ВКР.pdf	Расчетно-пояснительная записка к магистерской диссертации на тему «Программное обеспечение для анализа контента и формирования рекомендаций социальной сети на основе искусственных нейронных сетей».	
Приложение 2	Текст программы.	
Приложение 4	Руководство пользователя.	
Приложение 5	Данные эксперимента	
Компоненты		
Request.java	Абстрактный класс, описывающий запрос между модулями	
Response.java	Абстрактный класс, описывающий ответ между модулями	
User.java	Класс-сущность, описывающий пользователя социальной сети	
Post.java	Класс-сущность, описывающий публикацию социальной сети	
Comment.java	Класс-сущность, описывающий комментарий социальной сети	
UserRequest.java	Класс, описывающий клиентский запрос пользователей/пользователя	
PostRequest.java	Класс, описывающий клиентский запрос публикации/публикации	
CommentRequest.java	Класс, описывающий клиентский запрос комментариев/комментария	
RecommendationRequest.java	Класс, описывающий клиентский запрос получения рекомендаций для конкретного пользователя	

1	2	3
PostResponse.java	Класс, описывающий ответ с данными о публикациях/ публикации	
UserResponse.java	Класс, описывающий ответ с данными о пользователях/пользователе	
CommentResponse.java	Класс, описывающий ответ с данными о комментариях/комментарии	
UserService.java	Класс-сервис, содержащий логику взаимодействия с сущностью пользователь	
PostService.java	Класс-сервис, содержащий логику взаимодействия с сущностью публикация	
CommentService.java	Класс-сервис, содержащий логику взаимодействия с сущностью комментариев	
RestController.java	Объект данного класса принимает и отвечает на запросы расчёта формирования рекомендаций	
InputData.java	Объект данного класса описывает входные данные для расчёта рекомендаций пользователю	
Service.java	Объект данного класса описывает сервисную логику обработки данных: сервисы осуществляющие фильтрацию, тегирующие изображения, определяющие интересы пользователя.	
ModelConfiguration.py	Объект данного класса описывает конфигурацию модели машинного обучения	
FilteringResult.py	Объект данного класса описывает входные результат формирования рекомендаций пользователю	
Features.py	Объект данного класса описывает тэги (содержимое изображения, интересы пользователя)	
Request.py	Объект данного класса описывает запрос на расчёт фильтрации рекомендаций	
Response.py	Объект данного класса описывает ответ расчёта фильтрации рекомендаций	
RestController.py	Объект данного класса принимает и отвечает на запросы расчёта формирования рекомендаций	

1	2	3
InputData.py	Объект данного класса описывает входные данные для расчёта рекомендаций пользователю	
Service.py	Объект данного класса описывает сервисную логику обработки данных: сервисы осуществляющие фильтрацию, тегирующие изображения, определяющие интересы пользователя.	
Model.py	Объект данного класса описывает модель машинного обучения	
GetCollaborativeRequest.py	Объект данного класса описывает запрос на расчёт коллаборативной фильтрации рекомендаций	
GetContentRequest.py	Объект данного класса описывает запрос на расчёт фильтрации рекомендаций по контенту	
GetCollaborativeResponse.py	Объект данного класса описывает ответ расчёта коллаборативной фильтрации рекомендаций	
GetContentResponse.py	Объект данного класса описывает ответ на расчёт фильтрации рекомендаций по контенту	
ContentData.py	Объект данного класса описывает входные данные о публикациях для расчёта рекомендаций пользователю	
UserData.py	Объект данного класса описывает входные данные о пользователях для расчёта рекомендаций пользователю	
TaggingService.py	Объект данного класса описывает сервисную логику тегирования изображений	
DataService.py	Объект данного класса описывает сервисную логику обработки входных данных	
ContentFilteringService.py	Объект данного класса осуществляет сервисную логику формирования рекомендаций по контенту	
CollaborativeFilteringService.py	Объект данного класса осуществляет сервисную логику формирования рекомендаций методом коллаборативной фильтрации	

1	2	3
FeatureService.py	Объект данного класса описывает сервисную логику выделения интересов пользователя или публикации	
UserFeatures.py	Объект данного класса описывает интересы пользователя	
ContentFeatures.py	Объект данного класса описывает тэги изображения (содержимое изображения)	
EntityService.java	Абстрактный класс, описывающий сервисную логику обработки данных социальной сети	
Entity.java	Абстрактный класс, для описания сущности социальной сети	
RecommendationsApiService.java	Объект данного класса описывает сервис, в котором содержится логика взаимодействия с пакетом/модулем Recommendations, отправляет запросы и получает результат по персонализированным рекомендациям для пользователей.	
MainFlowService.java	Основной класс сервис, в котором описана логика обработки всех входных сообщений-запросов из пакета UI. В ходе разбора сообщений взаимодействует с остальными сервисами, формирует ответ с данными.	

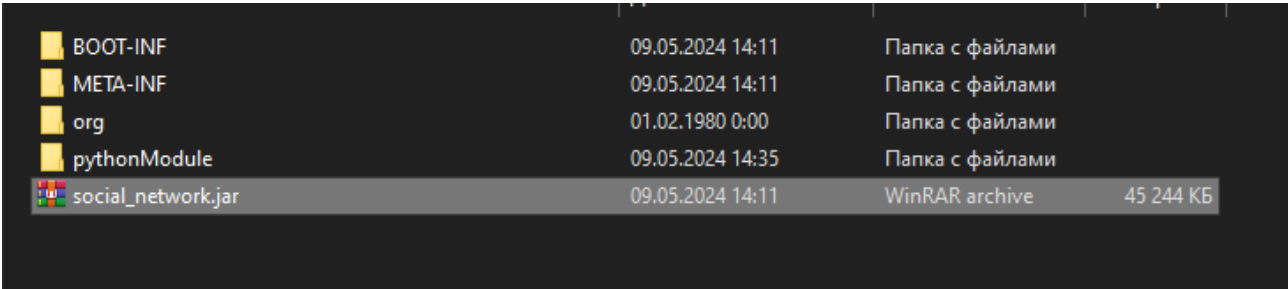
Руководство пользователя

Для установки программного продукта необходимо выполнить представленные ниже шаги:

1. Скачать архив с файлами запуска.
2. Установить необходимые драйверы для работы ПО (JRE 11+ версии, Python 3.9).
3. Распаковать архив с программным продуктом;

Для запуска программного продукта необходимо запустить «social_network.jar» (рис. П4.1). Запустить можно любым возможным способом запуска файлов формата «.jar», например, двойным нажатием левой кнопки мыши или через консоль.

По умолчанию сервер запускается, используя порт «8081». Для смены адреса в параметрах запуска этого файла указать параметр –port, запуская через консоли.



BOOT-INF	09.05.2024 14:11	Папка с файлами	
META-INF	09.05.2024 14:11	Папка с файлами	
org	01.02.1980 0:00	Папка с файлами	
pythonModule	09.05.2024 14:35	Папка с файлами	
social_network.jar	09.05.2024 14:11	WinRAR archive	45 244 КБ

Рис. П4.1. Файлы для запуска

Модули могут быть запущены на разных машинах, но при этом необходимо что бы был доступ между ними для обмена сообщениями (запрос/ответ). Модуль формирования рекомендаций запускается путём старта файла «starter.py» (рис. П4.2).

Имя	Дата изменения	Тип	Размер
Model	09.05.2024 14:20	Папка с файлами	
Services	09.05.2024 14:20	Папка с файлами	
starter.py	09.05.2024 14:36	Python File	4 КБ

Рис. П4.2. Каталог pythonModule

Остальные файлы данного модуля располагаются в этом же каталоге, в подкаталогах. Например, сервисная логика формирования рекомендаций описана в файлах по пути “.../pythonModule/Services/Recommendations/” (рис. П4.3).

Имя	Дата изменения	Тип	Размер
CollaborativeFiltering.py	09.05.2024 14:32	Python File	2 КБ
ContentFiltering.py	09.05.2024 14:31	Python File	4 КБ
ImageTagging.py	09.05.2024 14:28	Python File	2 КБ

Рис. П4.3. Подкаталог «Recommendations»

Возможности конфигурации весов алгоритмов, моделей указаны в п.2.3 и п.4.2.

Для начала работы с приложением, необходимо в окне браузера перейти на адрес приложения, вводя IP или DNS адрес сервера и порт (по умолчанию «localhost:8081»).

После успешного запуска и начала работы, пользователь попадает на стартовую страницу, содержащая приветствие и область навигации (рис. П4.4).

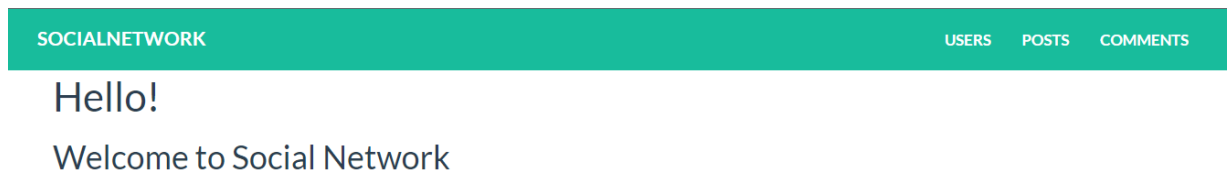


Рис. П4.4. Стартовая страница

В шапке стартовой страницы расположено навигационное меню, с помощью которого, пользователь может перейти к странице просмотра списков пользователей, публикаций и комментариев (рис. П4.5 – П4.7).

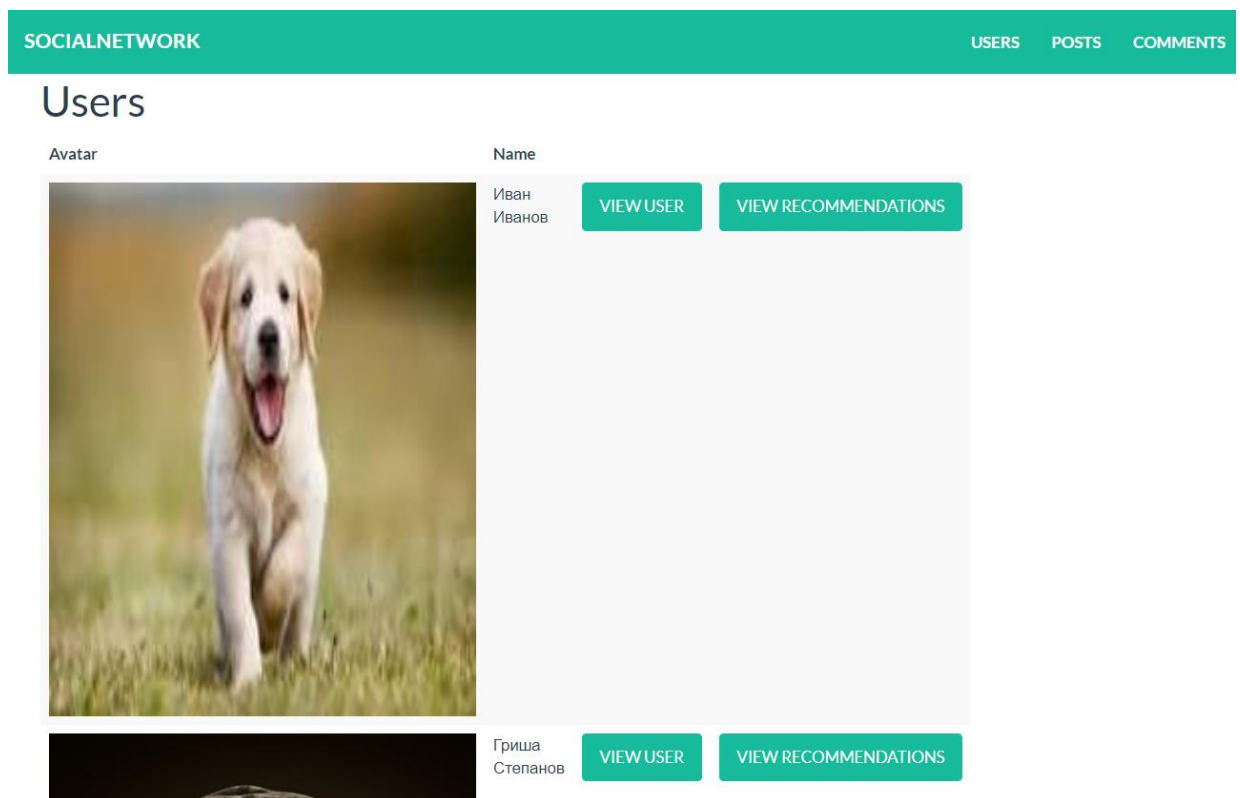


Рис. П4.5. Страница просмотра пользователей

На странице просмотра пользователей социальной сети содержатся кнопки для просмотра профиля конкретного пользователя из списка, а также кнопка

просмотра персонализированных рекомендаций для этого пользователя (см. рис. П4.5).

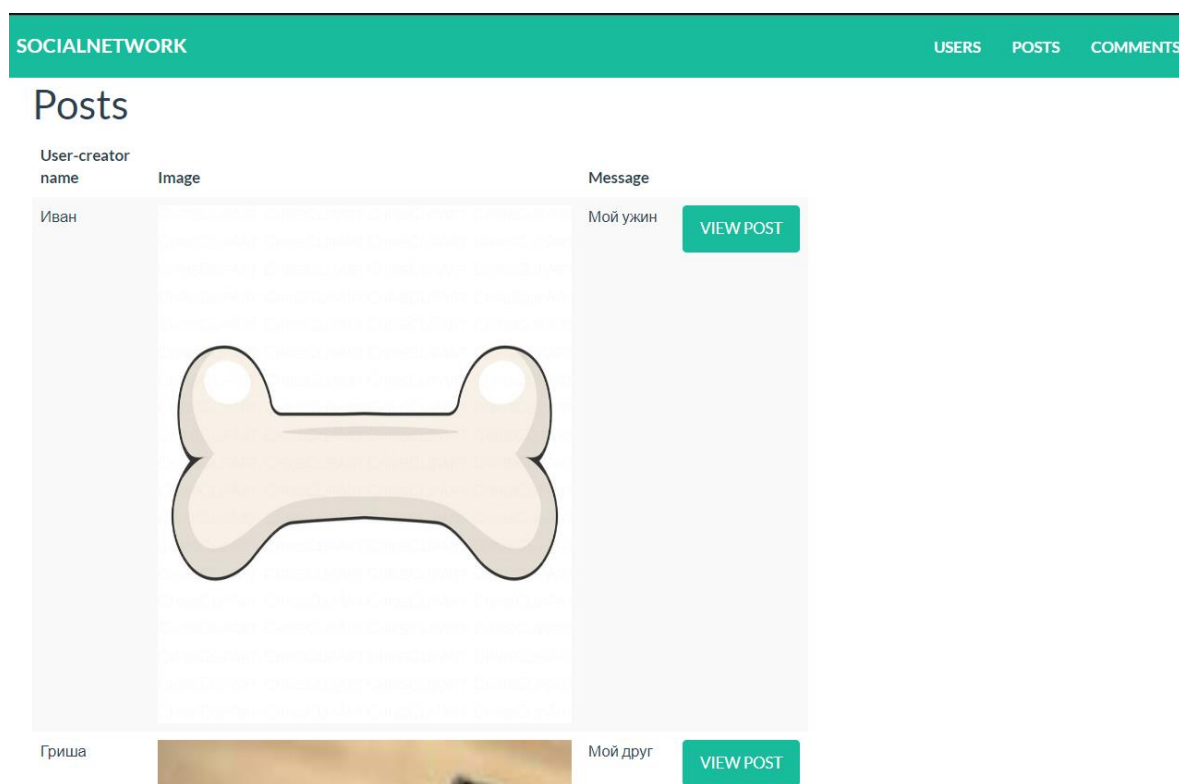


Рис. П4.6. Страница просмотра публикаций

На странице просмотра всех публикаций, пользователю админ-панели доступен просмотр подробной информации о публикации с помощью соответствующей кнопки (см. рис. П4.6).

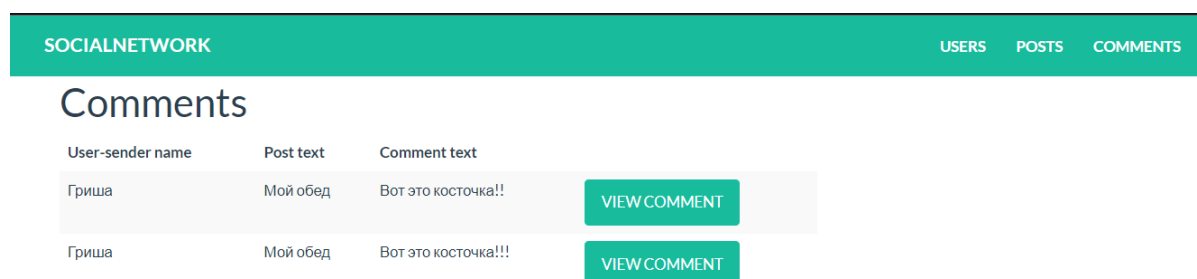


Рис. П4.7. Страница просмотра комментариев

На странице просмотра всех комментариев может быть осуществлён переход по кнопке к просмотру подробностей о комментарии (см. рис. П4.7).

Страницы подробного просмотра сущностей отображены на рис. П4.8 – П4.10.

The screenshot shows the 'User' page of a social network. At the top is a green header with 'SOCIALNETWORK' on the left and 'USERS', 'POSTS', and 'COMMENTS' on the right. Below the header, the title 'User' is displayed. The main content area contains a form with the following fields: 'Id' (value: 4), 'Name' (value: Иван), 'Second name' (value: Иванов), 'Avatar link' (value: https://encrypted-tbn0.gstatic.com/images?q=tnb), and 'Registration date' (value: 2023-09-21 11:41:33.06). Below these fields are two buttons: 'UPDATE USER' (grey) and 'DELETE USER' (red). At the bottom, there is a 'Posts' section with a dropdown menu showing 'Мой обед' and a 'VIEW POST' button (green).

Рис. П4.8. Страница просмотра пользователя

The screenshot shows the 'Post' page of a social network. At the top is a green header with 'SOCIALNETWORK' on the left and 'USERS', 'POSTS', and 'COMMENTS' on the right. Below the header, the title 'Post' is displayed. The main content area contains a form with the following fields: 'Id' (value: 10), 'Message' (value: Мой ужин), 'Image link' (value: https://kartinkof.club/uploads/posts/2022-09/166), and 'Publish date' (value: 2023-09-18 11:41:33.061). Below these fields are two buttons: 'UPDATE POST' (grey) and 'DELETE POST' (red). At the bottom, there is a 'Comments' section with a dropdown menu and a 'VIEW COMMENT' button (green).

Рис. П4.9. Страница просмотра публикации

SOCIALNETWORK

USERS POSTS COMMENTS

Comment

Id	8
Text	Вот это косточка!!
User-sender id	1
Post id	6
Publish date	2023-09-18 11:41:33.061

UPDATE COMMENT

DELETE POST

Рис. П4.10. Страница просмотра комментария

На каждой из страниц просмотра сущности расположены текстовые поля для ввода, в которых отображаются не редактируемые и редактируемые параметры сущностей. Для подтверждения изменений данных создана кнопка «UPDATE COMMENT». Администратор также может удалить сущность с помощью кнопки «DELETE POST». При просмотре пользователя существует список его публикаций, к которым можно перейти при помощи соответствующей кнопки. Аналогично для публикаций и комментариев.

Данные эксперимента

Далее будет приведено несколько примеров из тестовой выборки входных данных на определённом этапе обработки системой.

Пользователи (их интересы):

userId, interestTags

1, ['music', 'social', 'hobbies', 'life']

2, ['fashion', 'style']

3, ['sports']

4, ['animals', 'nature', 'dogs']

5, ['travel', 'adventure']

Публикации (изображения их описания):

PublicationId, Tags, userId, Image117

1, ['food', 'dining'], 16, image1

2, ['film', 'tv', 'video'], 16, image2

3, ['diaries', 'daily', 'life'], 14, image3

4, ['music'], 1, image4

5, ['cats', 'wild'], 3, image5

Изображения из публикаций приведены на рис. П5.1 - П5.5.



Рис. П5.1. Изображение публикации 1 (image1)



Рис. П5.2. Изображение публикации 2 (image2)

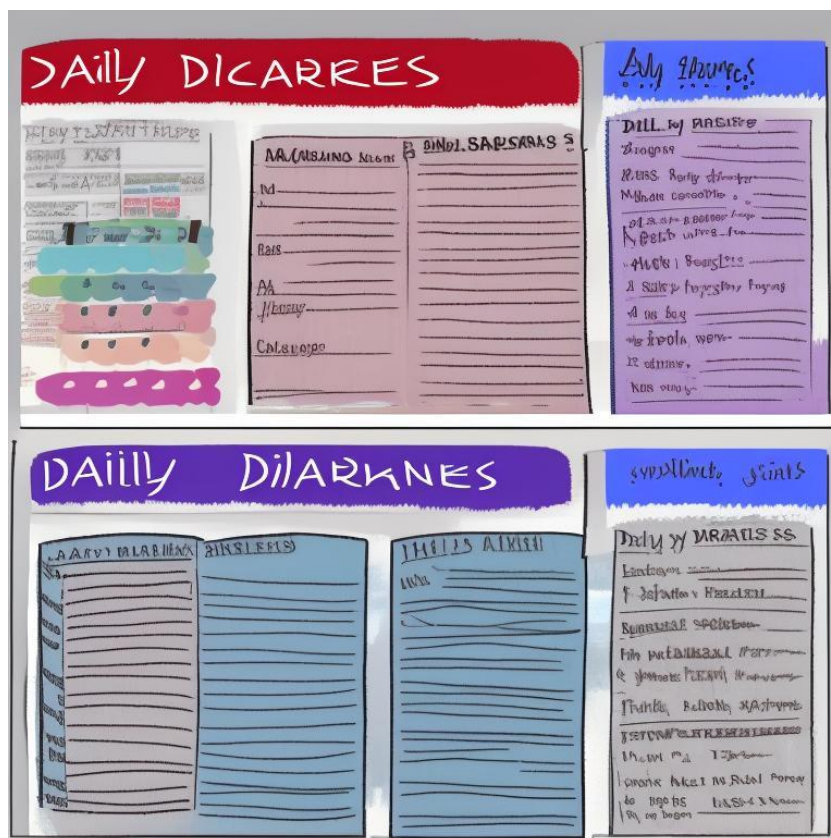


Рис. П5.3. Изображение публикации 3 (image3)



Рис. П5.4. Изображение публикации 4 (image4)

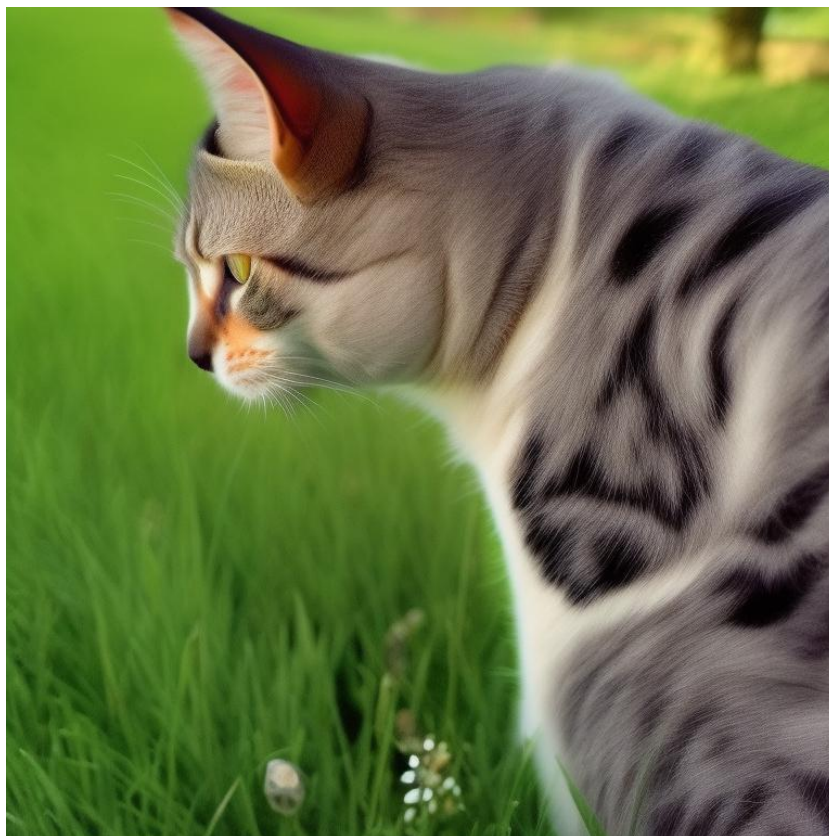


Рис. П5.5. Изображение публикации 5 (image5)

Далее будут отображены несколько матриц, рассчитываемых в ходе анализа контента, интереса пользователей и их схожести между собой.

В табл. П5.1 показана матрица оценок рекомендуемых публикаций разработанным алгоритмом для пользователей на основе анализа контента и схожести пользователей между собой.

Матрица оценок публикаций-кандидатов для рекомендаций
разработанным алгоритмом

Publicatio	Publicatio	Publicatio	Publicatio	Publicatio	Publicatio	Publicatio	Publicatio	Publicatio	Publicatio	Publicatio
0,3	0,7	0,7	1	1	1	0,7	0,7	1	0,7	0
0,7	0,7	1	0	0,7	0,7	1	1	0,7	0,7	0
0,7	0,7	0,7	0,7	0	1	0,3	0,7	1	0,7	0
0,7	0,7	0,7	1	1	1	0,7	0,7	1	0,7	0
0,7	0,7	0	0	0,7	0,7	0	1	0,7	0,7	0
0,7	0,7	0,7	0,7	1	1	0,7	0,7	1	0,7	0
0,7	0,7	1	0	0,7	0,7	1	1	0,7	0,7	0
0,7	1	1	0,7	1	1	1	0,7	1	1	0
1	0,7	0,7	0,7	1	1	0,7	1	1	0,7	0
0,7	0,7	0,7	0,7	1	1	0,7	0,7	1	0,7	0
0,7	0,7	0,7	0,7	1	1	0,3	0,7	1	0,3	0
0,7	0,7	0,7	0,7	1	1	0,3	0,7	1	0,7	0
1	0,7	0,7	0	0,7	0,7	0,7	1	0,7	0,3	0
0,7	0,7	1	0,7	1	1	1	1	1	0,7	0
0,7	0,7	1	0	0,7	0,7	0	1	0,7	0,7	0
0,7	0,7	0,7	0,7	1	1	0,7	0,7	1	0,7	1
0	0,7	0,7	0	0,3	0,7	0,3	1	0,7	0,7	1
1	1	0,7	0,7	1	1	0,7	0,7	1	1	0
1	0,3	0,7	0,7	0,7	0,7	0,7	0,7	0,7	0,7	0

Так же в рамках эксперимента рассматривался действующий алгоритм в целях сравнения его с разработанным, матрица его оценок для тех же публикаций отображена в табл. П5.2.

Матрица оценок публикаций кандидатов действующим алгоритмом

Publicatio	Publicatio	Publicatio	Publicatio	Publicatio	Publicatio	Publicatio	Publicatio	Publicatio
0	0	0	1	1	1	0	0	1
0	0	1	0	0	0	1	1	0
0	0	0	0	1	1	0	0	1
0	0	0	1	1	1	0	0	1
0	0	1	0	0	0	1	1	0
0	0	0	0	1	1	0	0	1
0	0	1	0	0	0	1	1	0
0	1	1	0	1	1	1	0	1
1	0	0	0	1	1	0	1	1
0	0	0	0	1	1	0	0	1
0	0	0	0	1	1	0	0	1
0	0	0	0	1	1	0	0	1
1	0	0	0	0	0	0	1	0
0	0	1	0	1	1	1	1	1
0	0	1	0	0	0	1	1	0

На рис. П5.1 отображён код на языке Python для расчёта метрик оценки алгоритмов.

```

def compute_mrr(users_publications_matrix,
predicted_ratings):
    mrr_sum = 0
    for user_id in
range(predicted_ratings.shape[0]):
        relevant_items =
np.where(users_publications_matrix[user_id] == 1)[0]
        if len(relevant_items) > 0:
            ranked_items =
np.argsort(predicted_ratings[user_id])[:-1] #
Ранжированные предсказанные значения
            first_relevant_item_position =
np.where(np.isin(ranked_items, relevant_items))[0][0]
+ 1 # Позиция первого релевантного элемента
            mrr_sum += 1 /
first_relevant_item_position # Обратный ранг
первого релевантного элемента
        mrr = mrr_sum / predicted_ratings.shape[0] #
Средний обратный ранг
    return mrr

def dcg_at_k(relevances, k):
    relevances = np.asarray(relevances)[:k]
    discounts = np.log2(np.arange(len(relevances))
+ 2)
    return np.sum(relevances / discounts)

def idcg_at_k(relevances, k):
    relevances = np.sort(np.asarray(relevances))[:-
1][:k]
    discounts = np.log2(np.arange(len(relevances))
+ 2)
    return np.sum(relevances / discounts)

def ndcg_at_k(predictions, k):
    ndcg_values = []
    for row in predictions:
        dcg = dcg_at_k(row, k)
        idcg = idcg_at_k(row, k)
        if idcg == 0:
            ndcg_values.append(0.0)
        else:
            ndcg_values.append(dcg / idcg)
    return np.mean(ndcg_values)

def compute_recall(actual_positives,
predicted_positives):
    true_positives =
np.sum(np.logical_and(actual_positives == 1,
predicted_positives == 1))
    false_negatives =
np.sum(np.logical_and(actual_positives == 1,
predicted_positives == 0))
    if true_positives + false_negatives == 0:
        return 0
    else:
        recall = true_positives / (true_positives +
false_negatives)
    return recall

```

Рис. П5.1. Код расчёта метрик оценок рекомендательных алгоритмов

```

def compute_accuracy(users_publications_matrix,
predicted_ratings):
    # Общее количество правильно
    # предсказанных оценок
    total_correct =
np.sum(np.logical_and(users_publications_matrix,
predicted_ratings))

    # Общее количество оценок (все элементы
    # матрицы)
    total_predictions =
users_publications_matrix.size

    # Точность
    accuracy = total_correct / total_predictions

    return accuracy

```

```

def compute_precision(actual_positives,
predicted_positives):
    true_positives =
np.sum(np.logical_and(actual_positives == 1,
predicted_positives == 1))
    false_positives =
np.sum(np.logical_and(actual_positives == 0,
predicted_positives == 1))

    if true_positives + false_positives == 0:
        return 0
    else:
        precision = true_positives / (true_positives +
false_positives)
        return precision

    from sklearn.metrics import mean_absolute_error,
mean_squared_error, r2_score

```